



Lecture 13: Instance Segmentation & 6D Pose Estimation

Li Yi

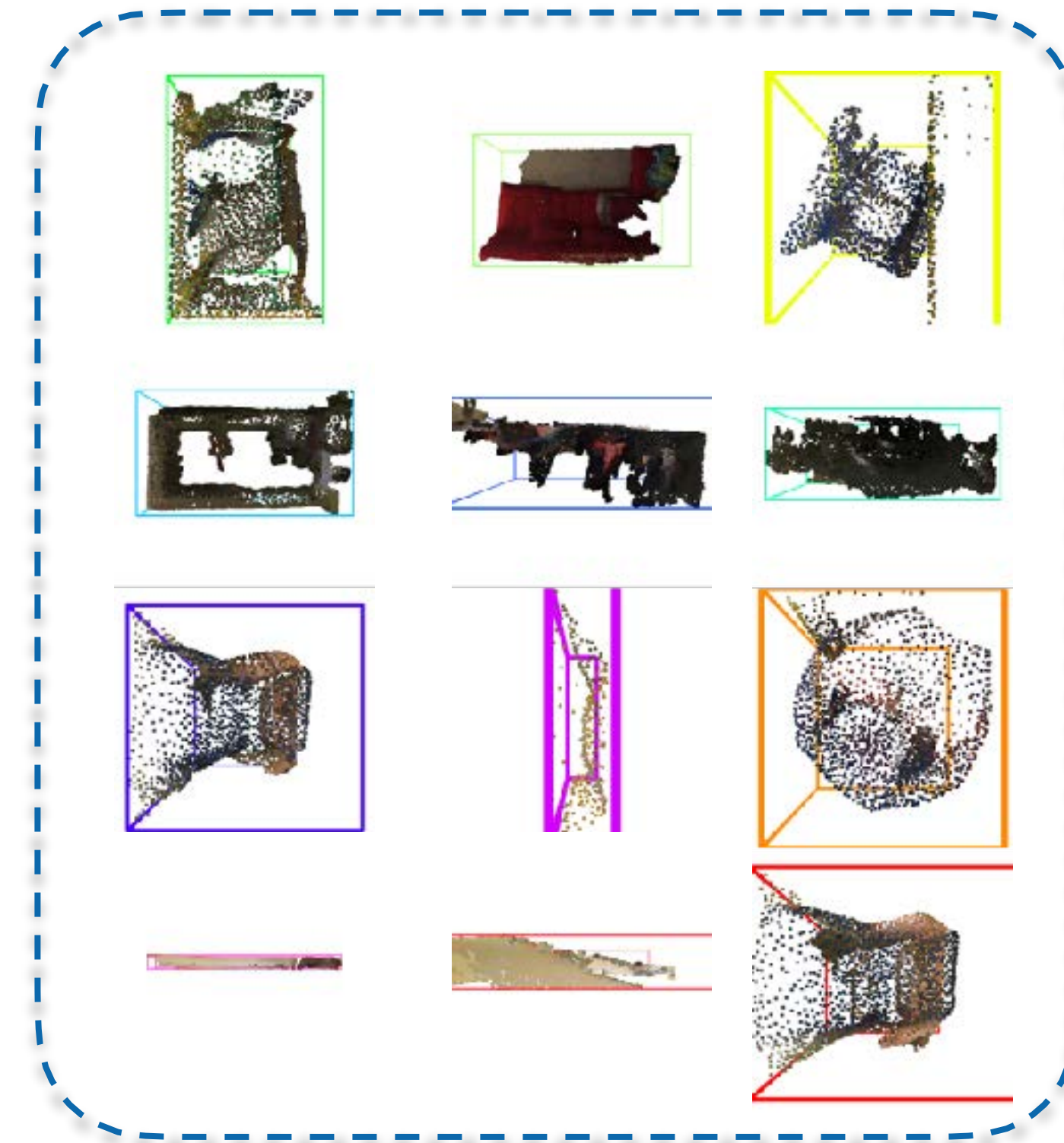
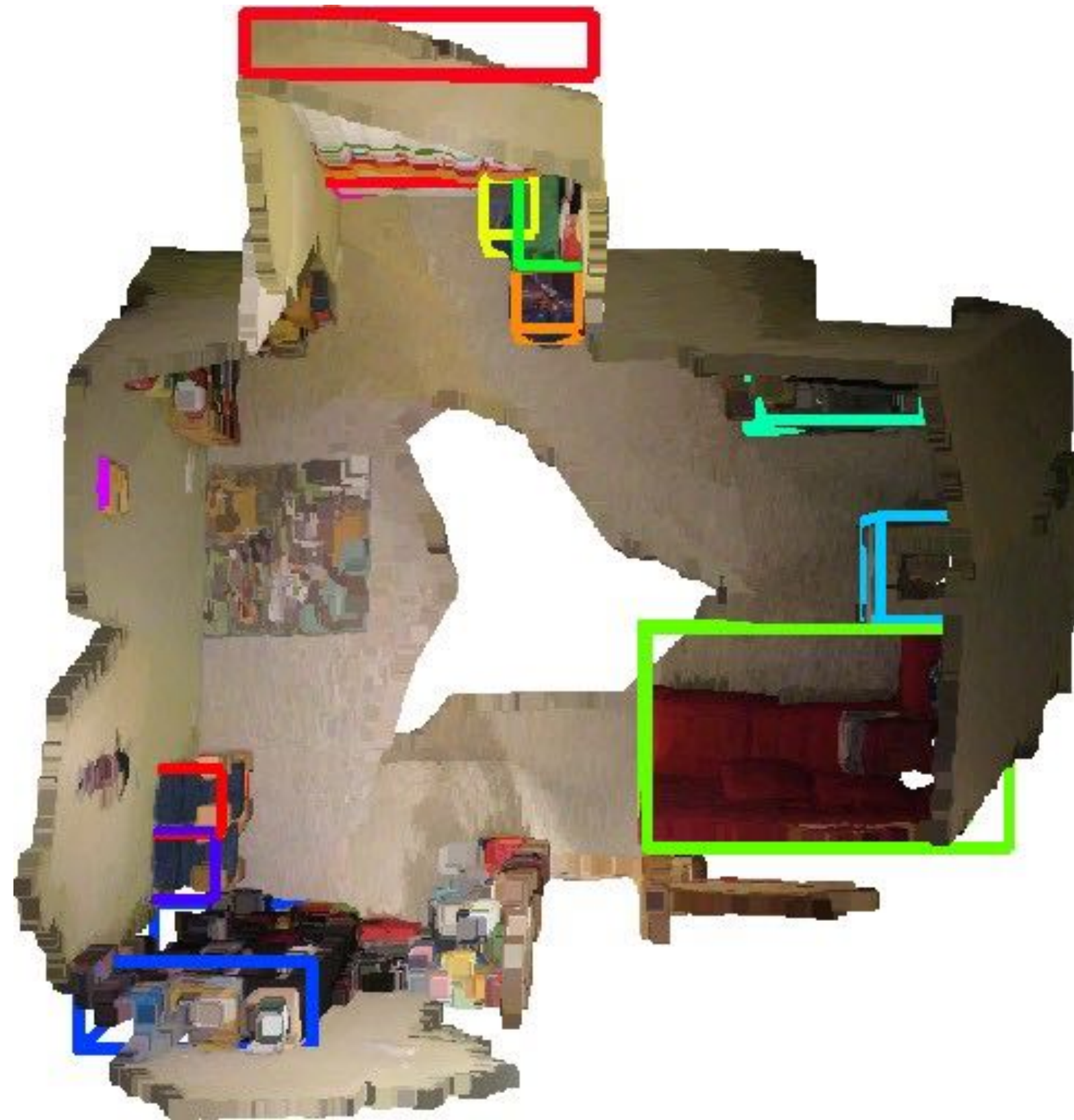
May 22, 2025

Recap

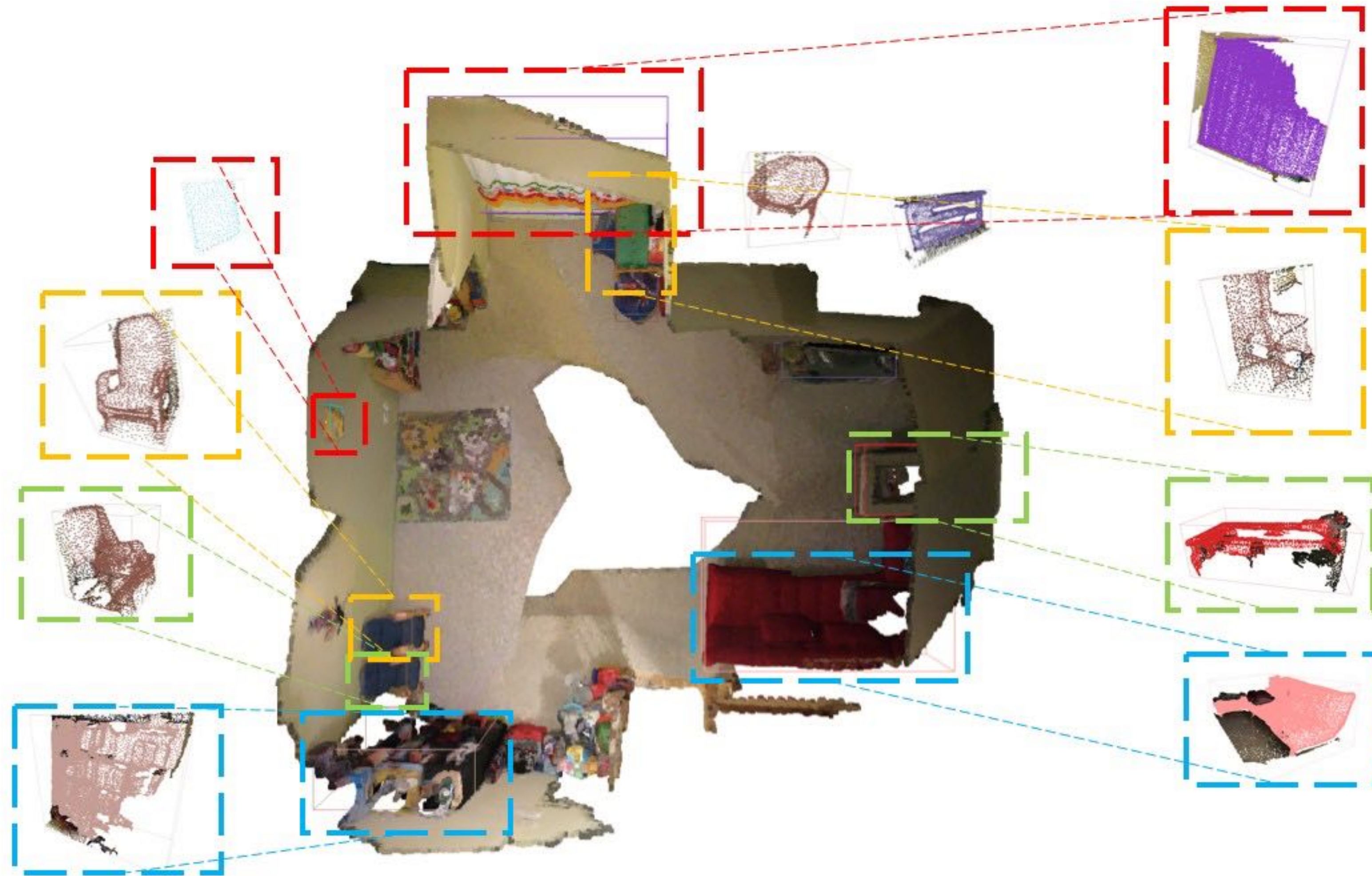
- 3D Object Detection
 - Background
 - The History and Recent Development
 - ✦ Frustum PointNets
 - ✦ PointPillars
 - ✦ VoteNet
- 3D Instance Segmentation
 - *Top-Down Approaches*
 - Bottom-Up Approaches

First Step: Generate Proposals (e.g., Bounding Boxes)

Proposals



Second Step: Associate Points with Proposals

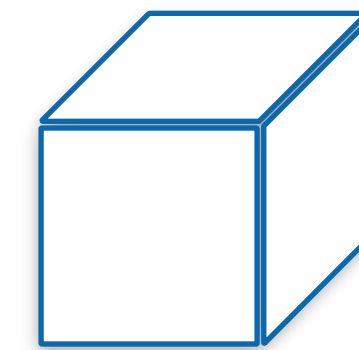


Two Key Questions:

- How to generate (instance) proposals?
- How to associate points with proposals?

First of all, what is a good proposal representation?

- Easy to parameterize and predict
- Easy to classify whether a point belongs to it
- Parameterization:
 - Primitive type
 - Parameters (position, rotation, ...)
- Common choices: 3D bounding box, spheres



Proposal Generation: Non-Learning

- **Sliding window:** The straightforward, heuristic method to generate proposals without learning
- Slide a (template) window over the input point cloud



stride=(0.5, 0.5)
size=(1.5, 1.5)



stride=(1.5, 1.5)
size=(1.5, 1.5)



stride=(1.5, 0.5)
size=(1.5, 1.0)

Proposal Generation: Learning-based

- To have a high recall, we need to densely slide a window
- However, too heavy burden for the association step

Examples of Learning-based Proposal Generation

- In 3D detection works:
 - 2D detection-based proposal (Frustum PointNet)
 - X-ray proposal (PointPillar)
 - Voting-based proposal (VoteNet)
- Other methods designed for instance segmentation:
 - Bounding box prediction proposal (3D-BoNet)
 - Shape generation proposal (GSPN)

Examples of Learning-based Proposal Generation

- In 3D detection works:
 - 2D detection-based proposal (Frustum PointNet)
 - X-ray proposal (PointPillar)
 - Voting-based proposal (VoteNet)
- Other methods designed for instance segmentation:
 - Bounding box prediction proposal (3D-BoNet)
 - Shape generation proposal (GSPN)

3D-BoNet Pipeline

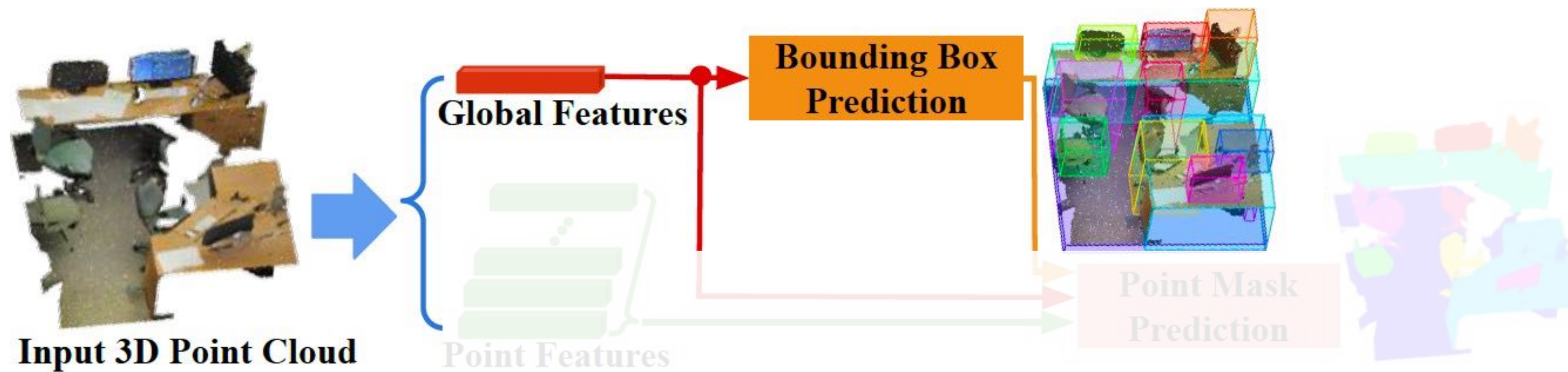


Figure 1: The 3D-BoNet framework for instance segmentation on 3D point clouds.

3D-BoNet Pipeline

“set prediction” task

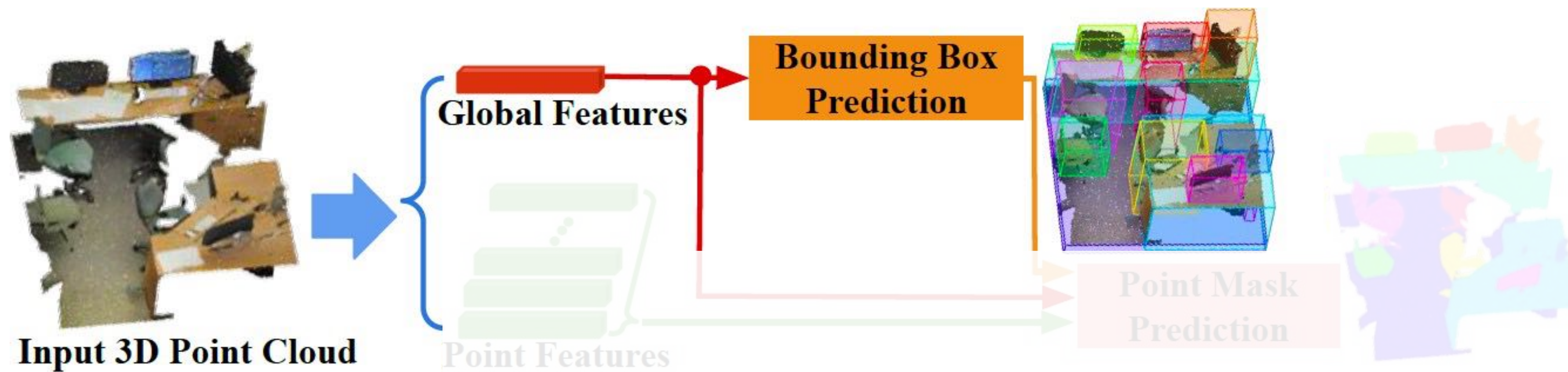
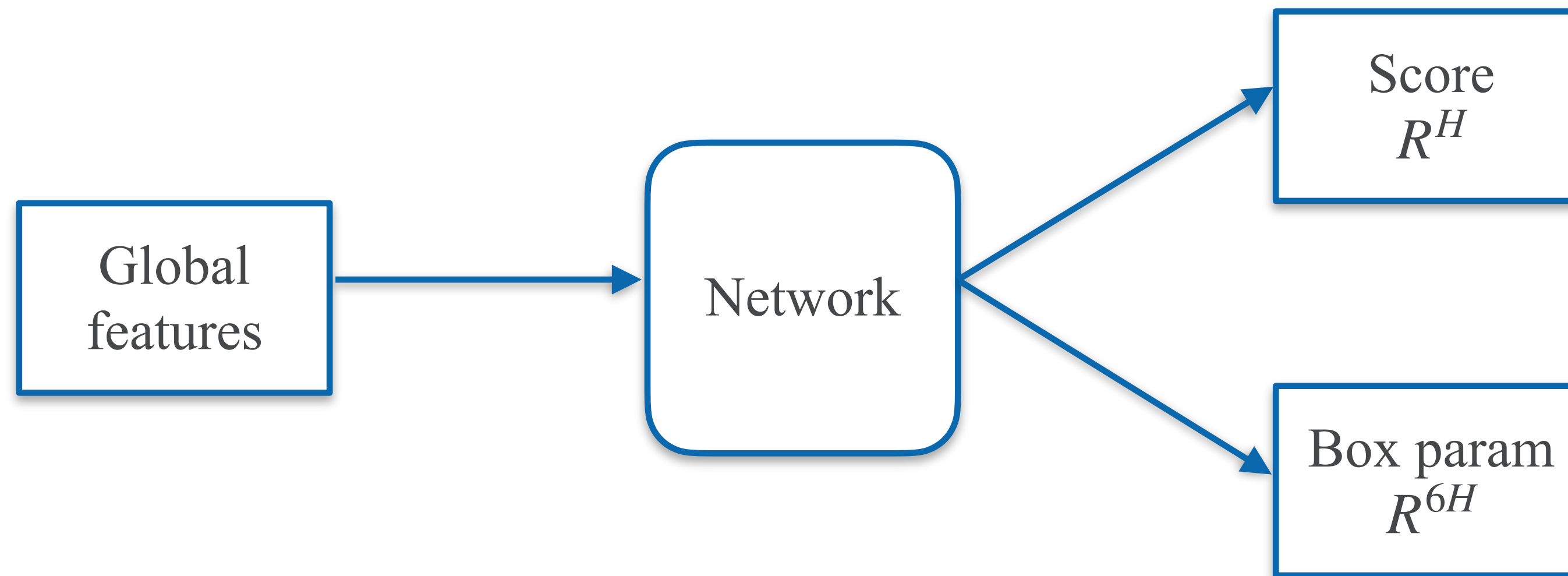


Figure 1: The 3D-BoNet framework for instance segmentation on 3D point clouds.

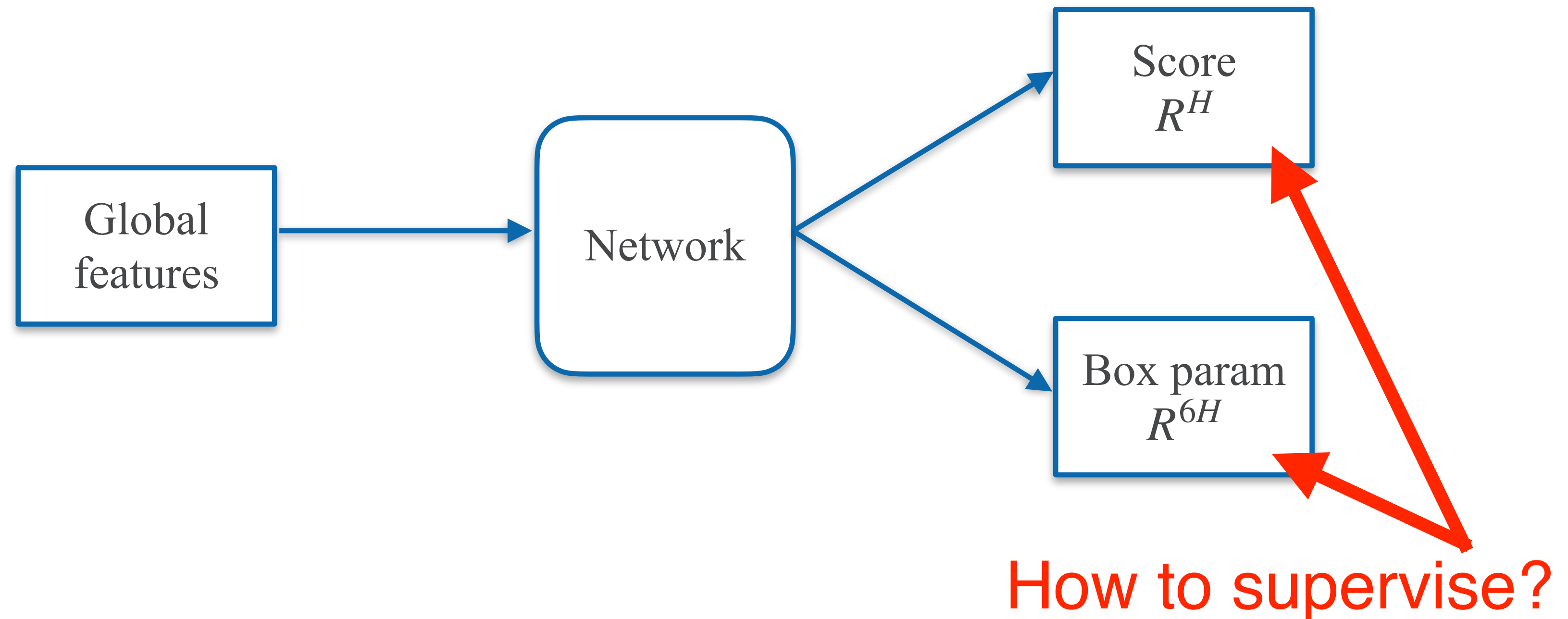
Bounding Box Prediction

- Bounding box parameterization:
 $\{x_{min}, y_{min}, z_{min}, x_{max}, y_{max}, z_{max}\}$
- Regress a predefined, fixed number (H) of bounding boxes



Bounding Box Prediction

- Bounding box parameterization:
 $\{x_{min}, y_{min}, z_{min}, x_{max}, y_{max}, z_{max}\}$
- Regress a predefined, fixed number (H) of bounding boxes

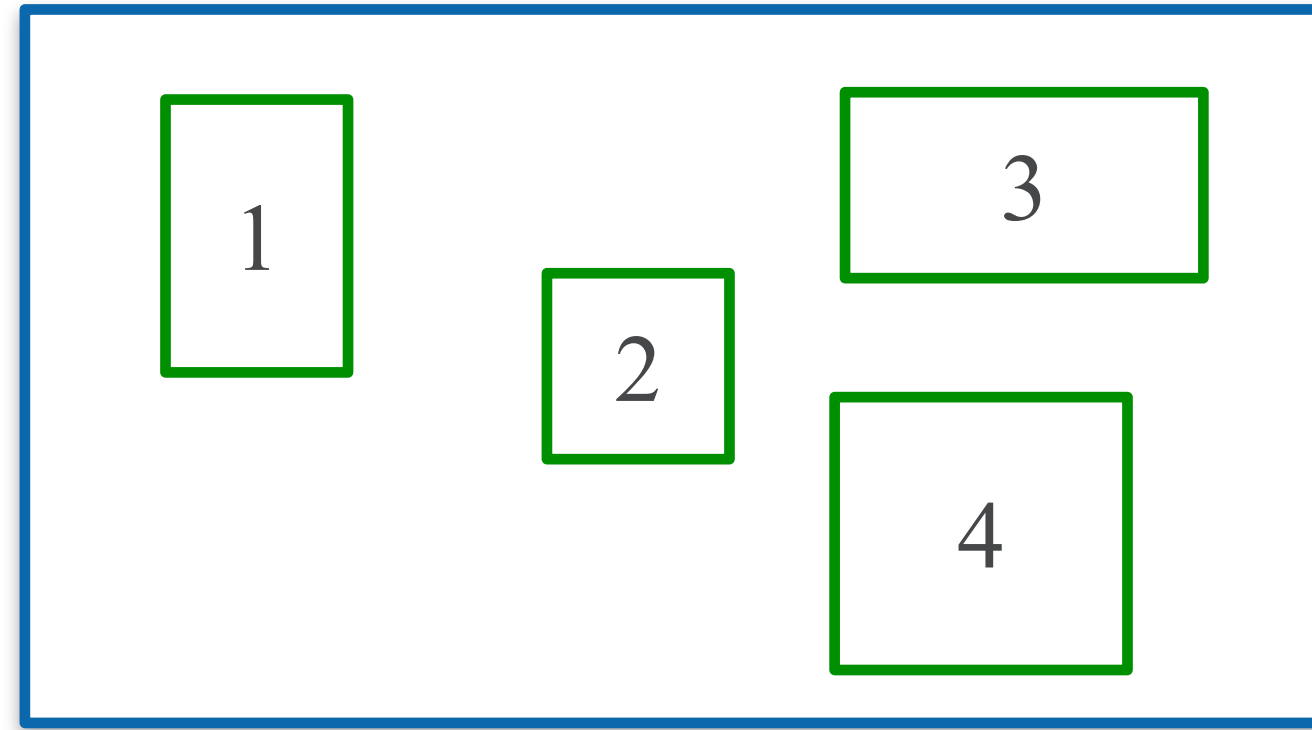


Loss: Bounding Box Association

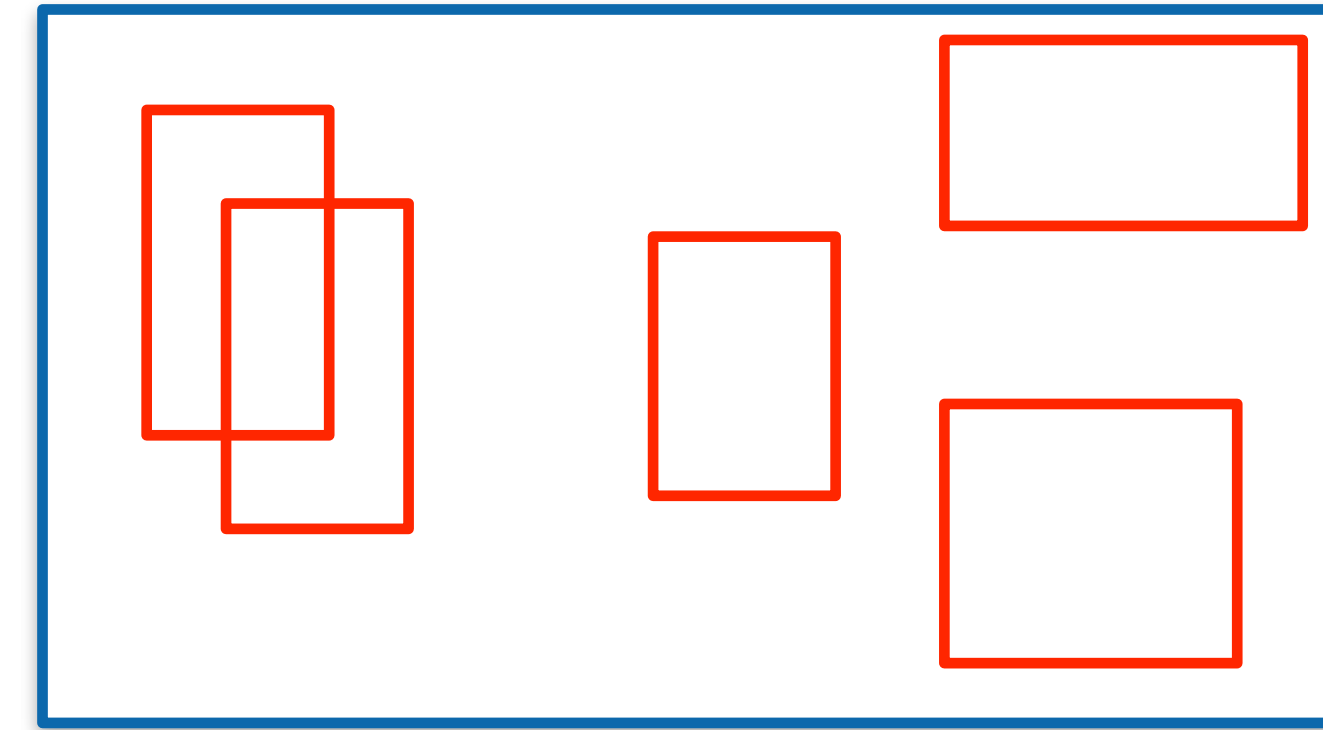
How to know the GT on-the-fly?

Find a match between the GT and predicted boxes

Optimal Association (2D case)

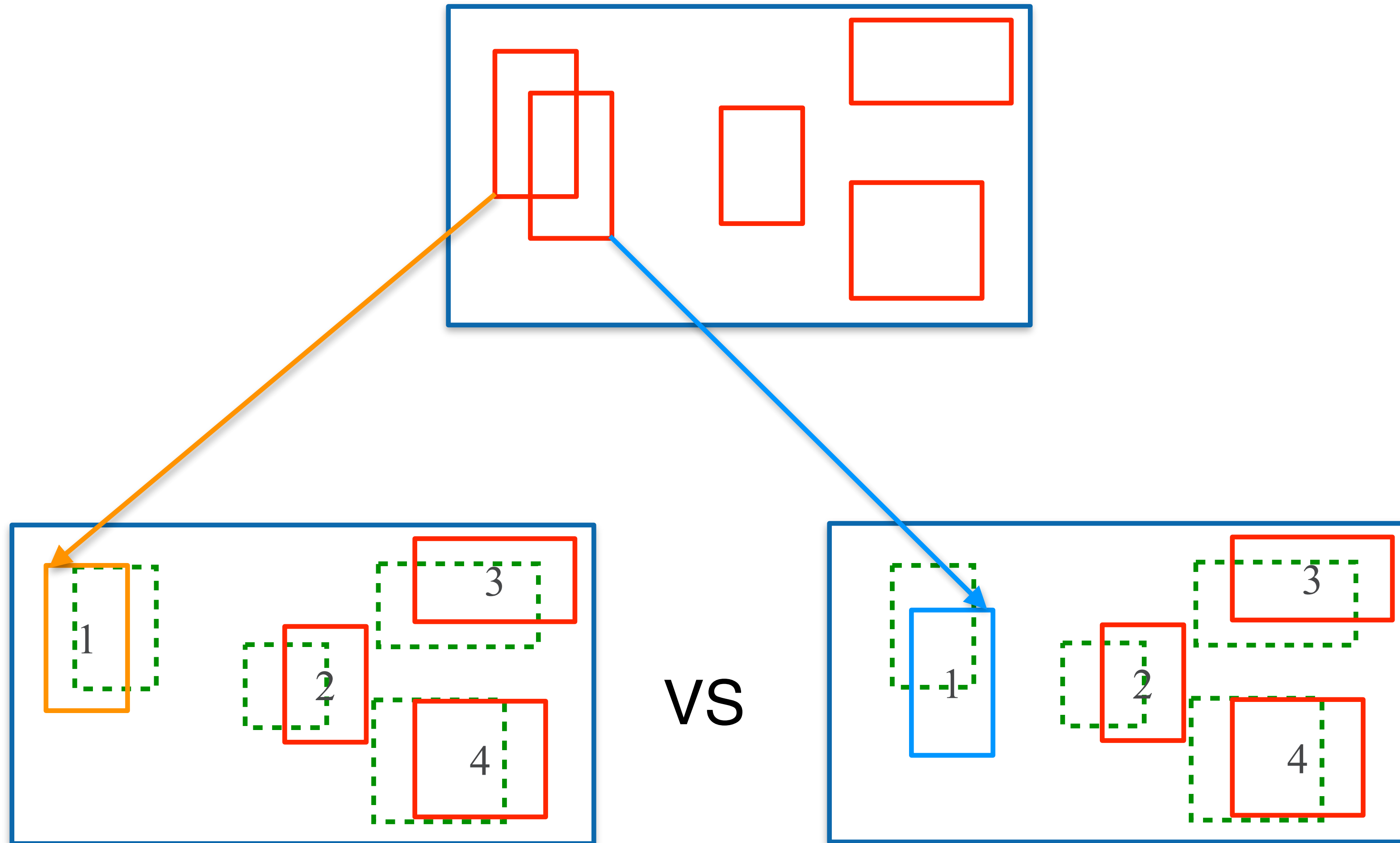


GT boxes



Prediction

Optimal Association (2D case)

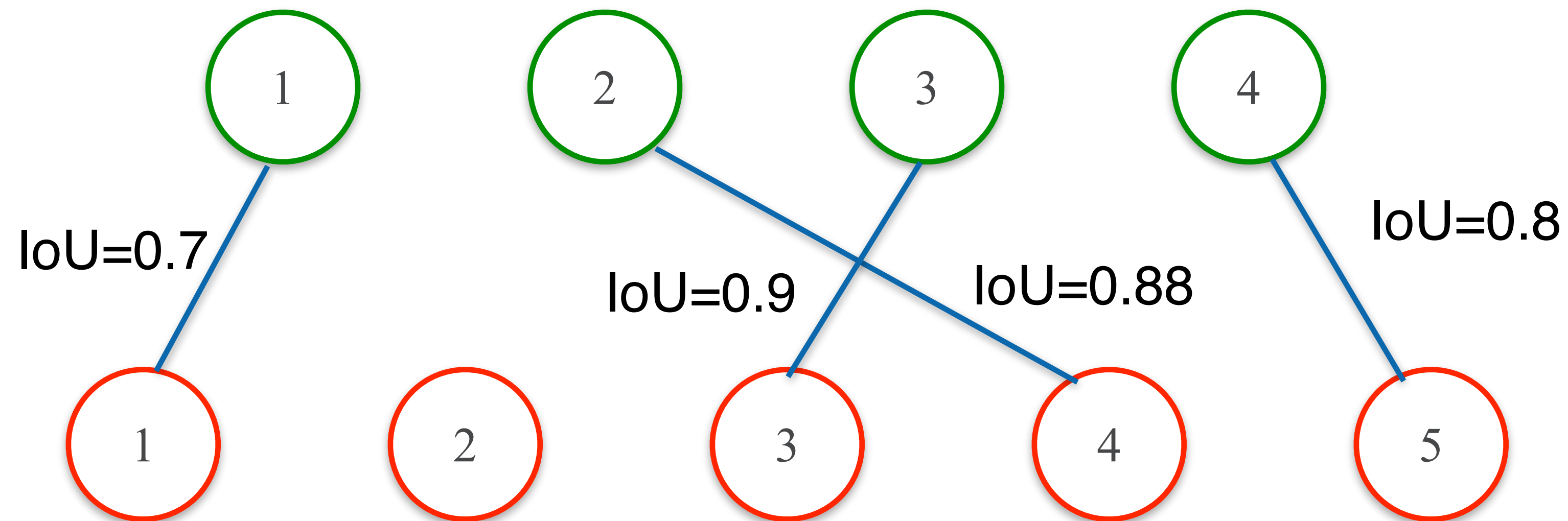


Matching 1

Matching 2

Optimal Association

- Objective: maximize the overall match gain
- Hungarian algorithm can solve this problem (similar to EMD)



The overall gain is $0.7 + 0.9 + 0.88 + 0.8$

- Gain $\Rightarrow$ cost, maximize $\Rightarrow$ minimize

Association Cost

- The cost (weight of bipartite graph) should evaluate the similarity between the predicted box and GT box (e.g., L_2 over b.box vertices offset)

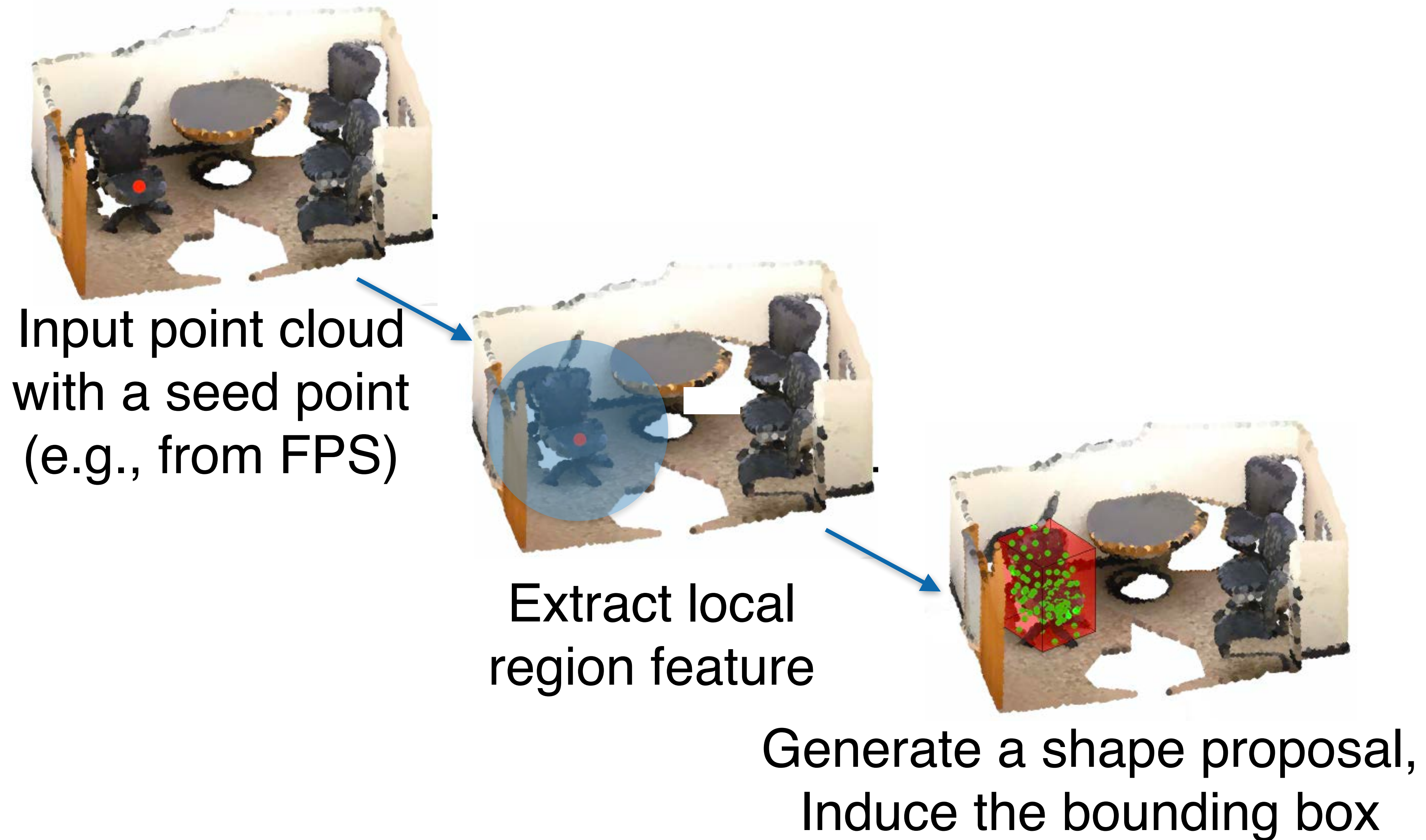
$$C_{i,j}^{ed} = \frac{1}{6} \sum (B_i - \bar{B}_j)^2$$

- Other criteria
 - Soft IoU
- The cost can be used as the loss directly

Examples of Learning-based Proposal Generation

- In 3D detection works:
 - 2D detection-based proposal (Frustum PointNet)
 - X-ray proposal (PointPillar)
 - Voting-based proposal (VoteNet)
- Other methods designed for instance segmentation:
 - Bounding box prediction proposal (3D-BoNet)
 - Shape generation proposal (GSPN)

GSPN Pipeline



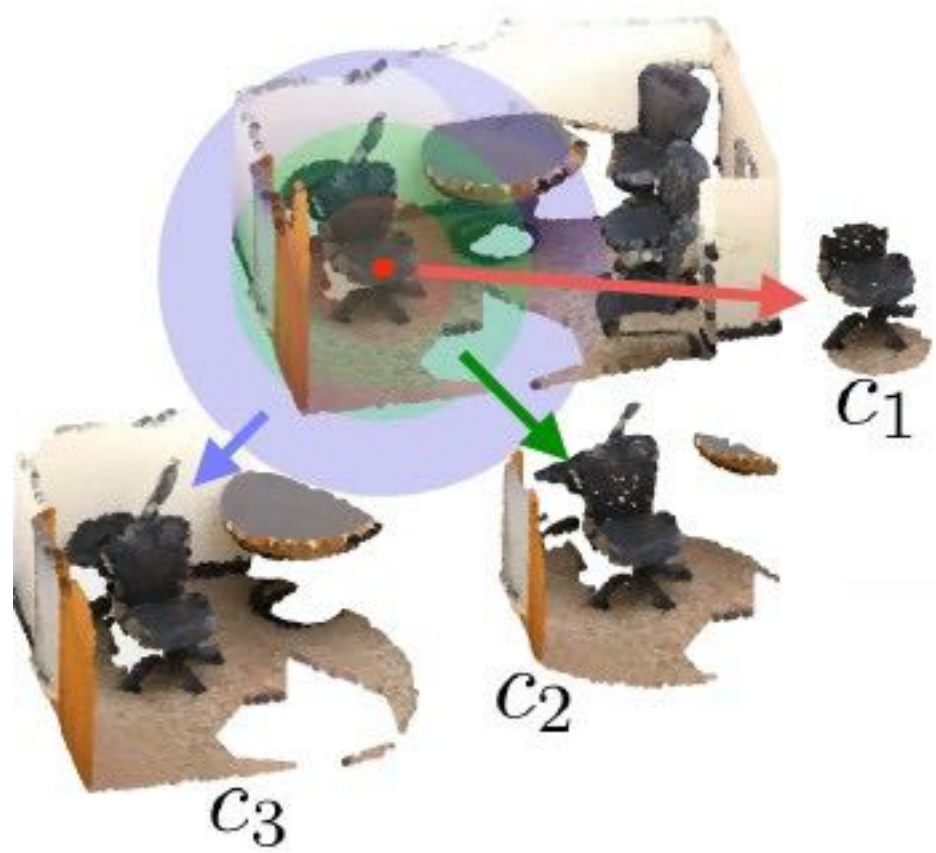
Point Cloud as Object Proposal

- Unlike primitive-based proposals, it is possible to **generate a point cloud** as a proposal (recall the single image to point cloud work)



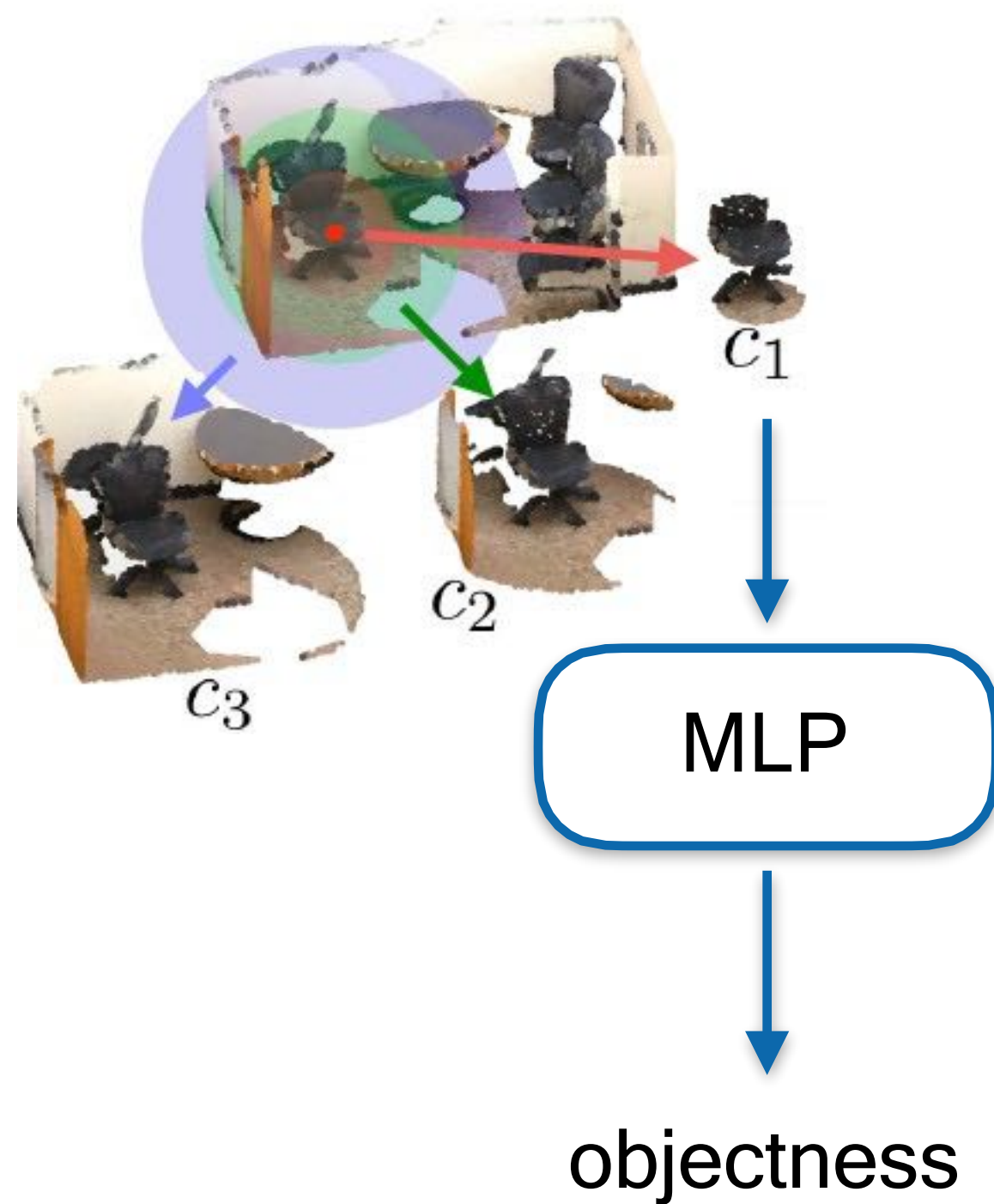
Generate Proposal as a Point Cloud

- Take a seed point and local context of different scales



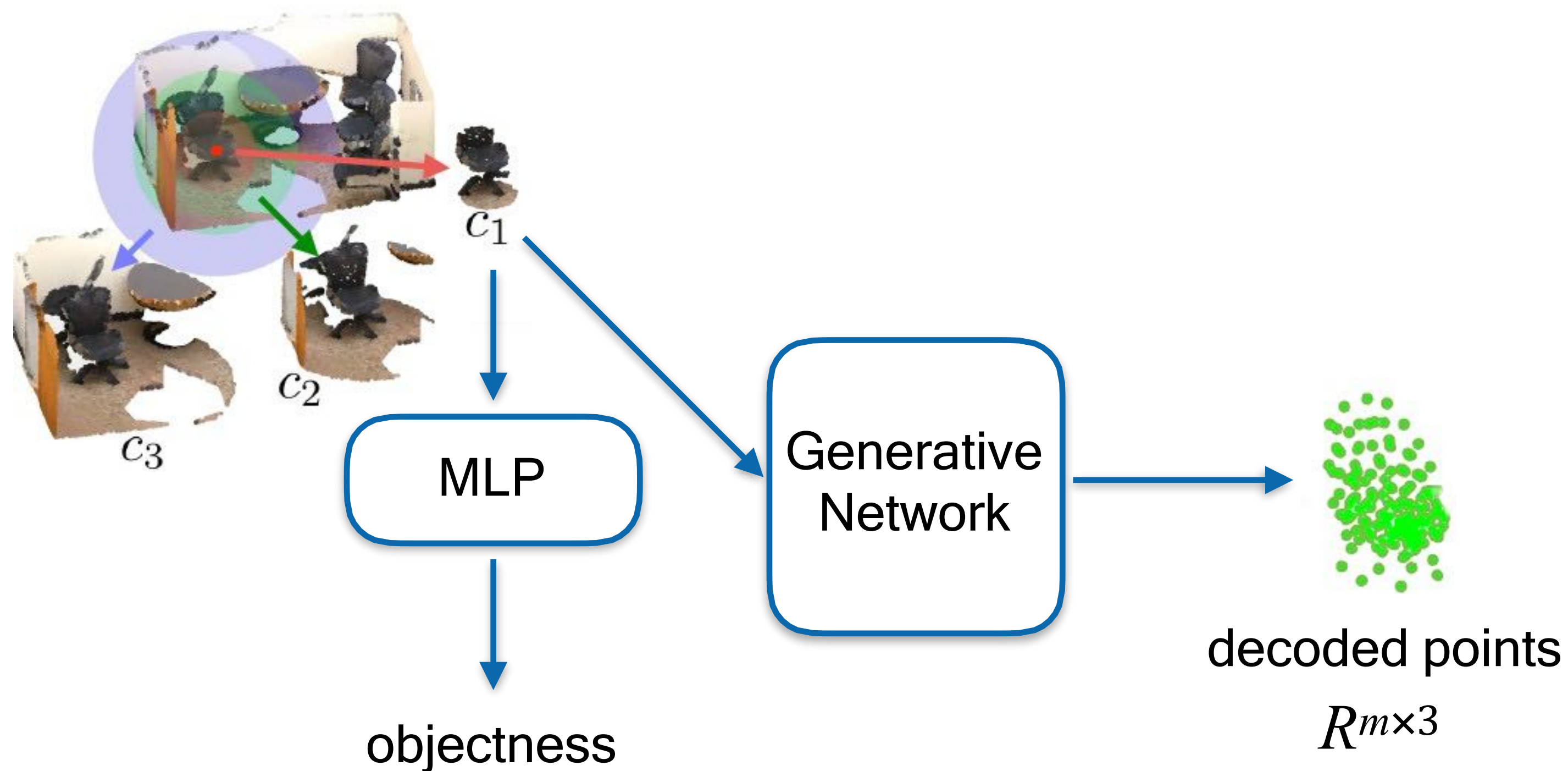
Generate Proposal as a Point Cloud

- Predict “objectness” (object v.s. non-object)



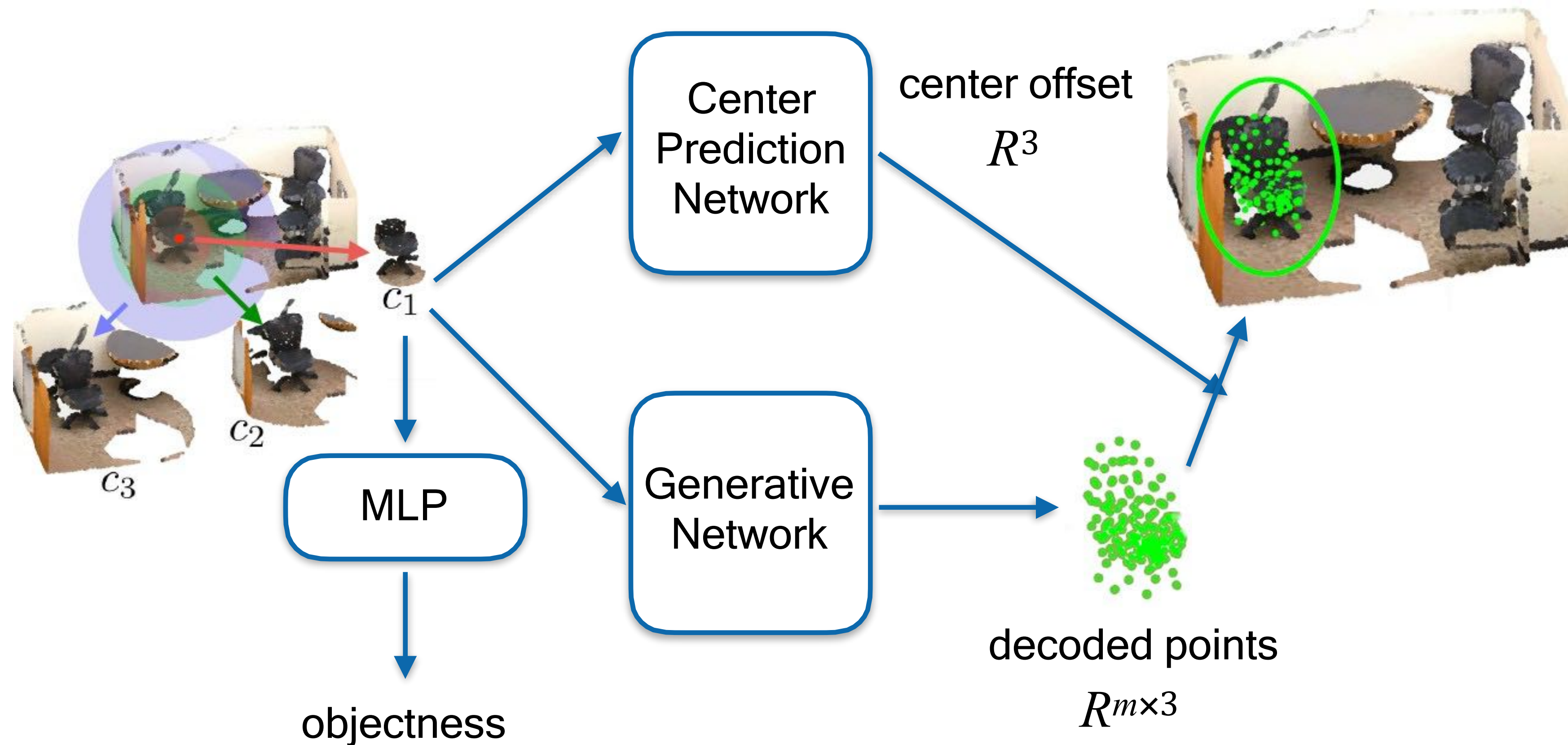
Generate Proposal as a Point Cloud

- Decode points, e.g., by a fully-connected network, as in single-image to point cloud work



Generate Proposal as a Point Cloud

- Predict a center offset from the seed point to the center of the instance



Losses for Point Cloud Proposals

- Only for positive proposals
 - Center prediction loss: huber loss (smooth l1)
 - Shape generation loss: chamfer distance
- For all the proposals
 - Objectness loss: cross-entropy

**How to associate points
with proposals?**

Basic Idea

- Given the proposal, predict a binary mask for each point whether the point belongs to the instance

Example: 3D-BoNet

- Steps:
 - Extract per-point features $\tilde{F}_l \in \mathbb{R}^{N \times D}$
 - Get instance-aware features $\hat{F}_l \in \mathbb{R}^{N \times (D+7)}$, e.g.,
 - point features (D dim)
 - bounding parameters (6 dim)
 - confidence (1 dim)
 - Predict point-wise mask $M_i \in \{0,1\}^N$

Point Label Generation and Loss

- Given the matched proposal and GT
 - For each proposal, we can induce a per-point binary mask given its corresponding GT



overall instance label



instance label for each proposal

Point Label Generation and Loss

- Given the matched proposal and GT
 - For each proposal, we can induce a per-point binary mask given its corresponding GT
 - We use a cross-entropy loss to do per-point binary classification



overall instance label



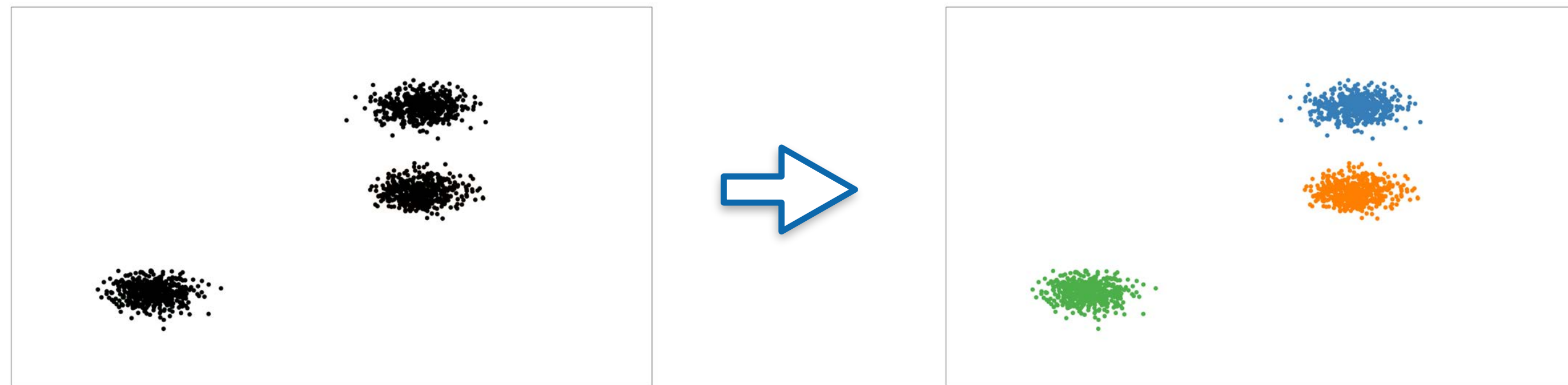
instance label for each proposal

Outline

- 3D Object Detection
 - Background
 - The History and Recent Development
 - ✦ Frustum PointNets
 - ✦ PointPillars
 - ✦ VoteNet
- 3D Instance Segmentation
 - Top-Down Approaches
 - *Bottom-Up Approaches*

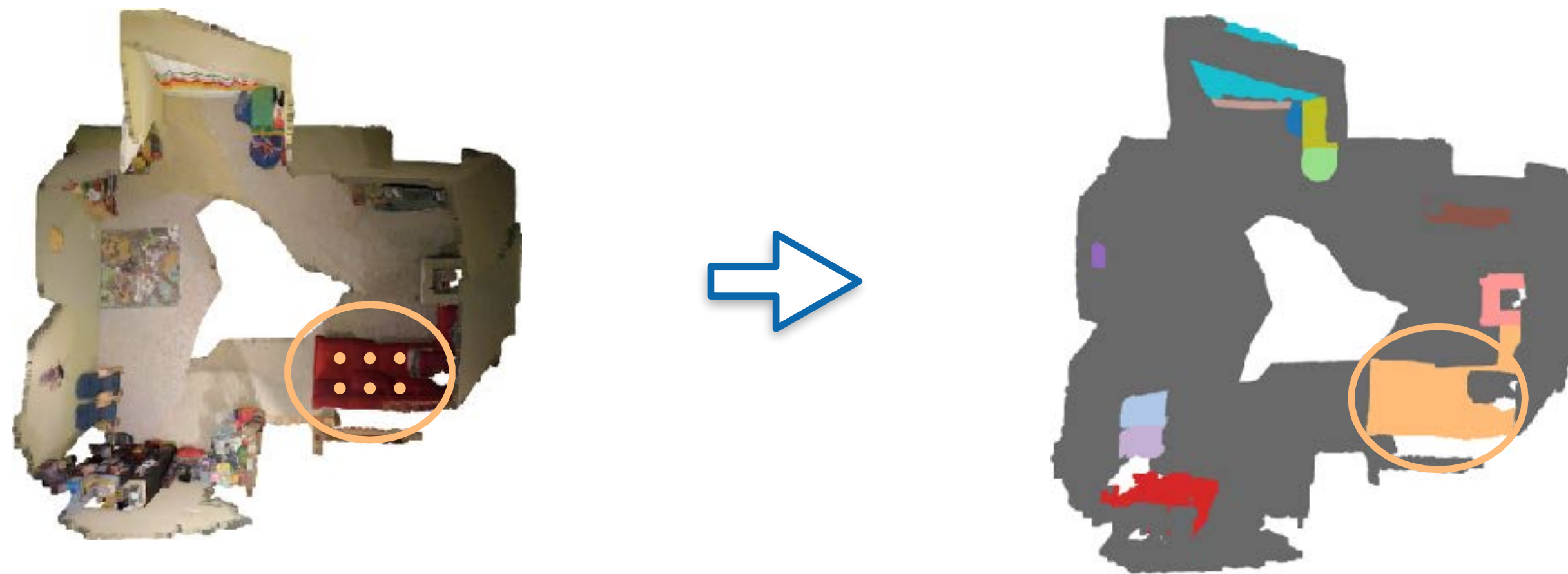
What is Bottom-up?

- A bottom-up approach is grouping the pieces of the points together to form an object.



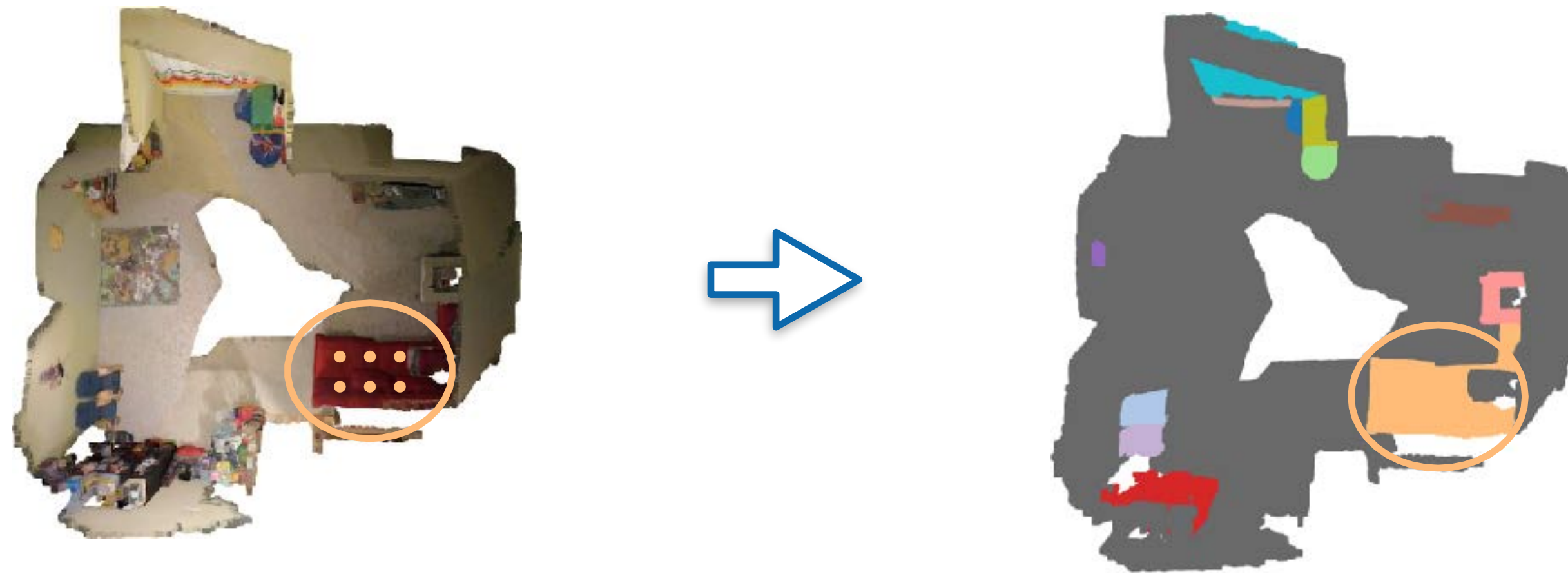
What is Bottom-up?

- A bottom-up approach is grouping the pieces of the points together to form an object.



What is Bottom-up?

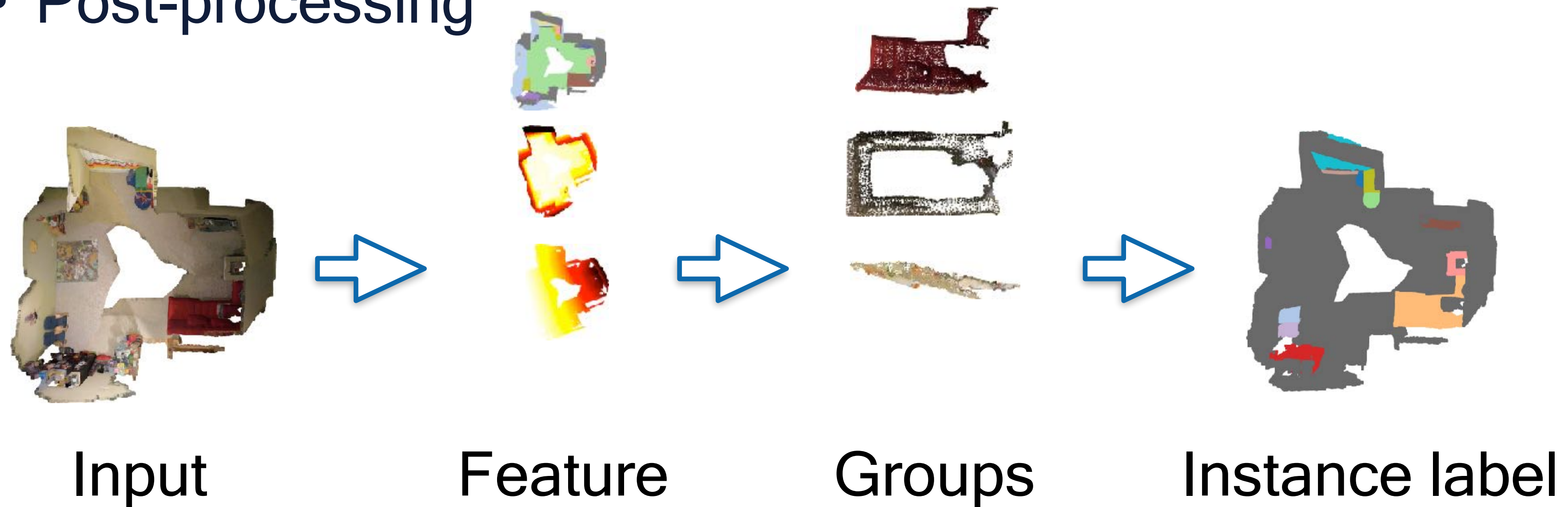
- A bottom-up approach is grouping the pieces of the points together to form an object.



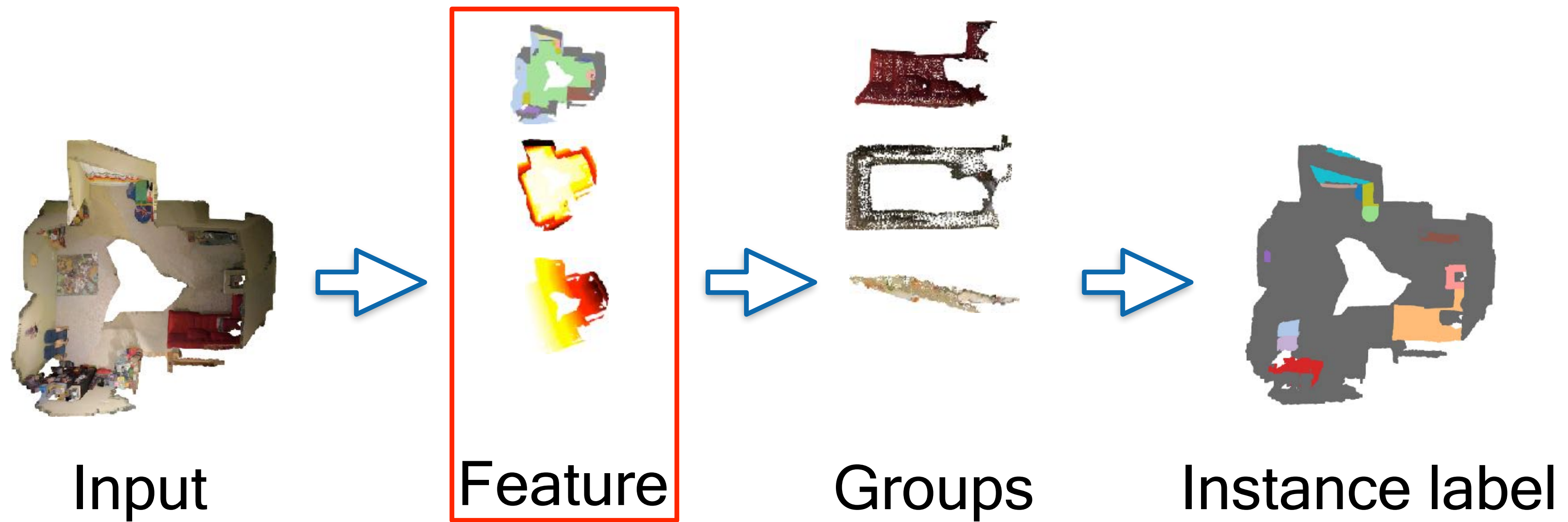
- In contrast, top-down: directly predict a proposal as object proxy and verify

Grouping-based Instance Segmentation

- Key Question: What points/fragments should be grouped?
 - Distance function
- Group procedure
 - Grouping/Clustering algorithm
- Post-processing

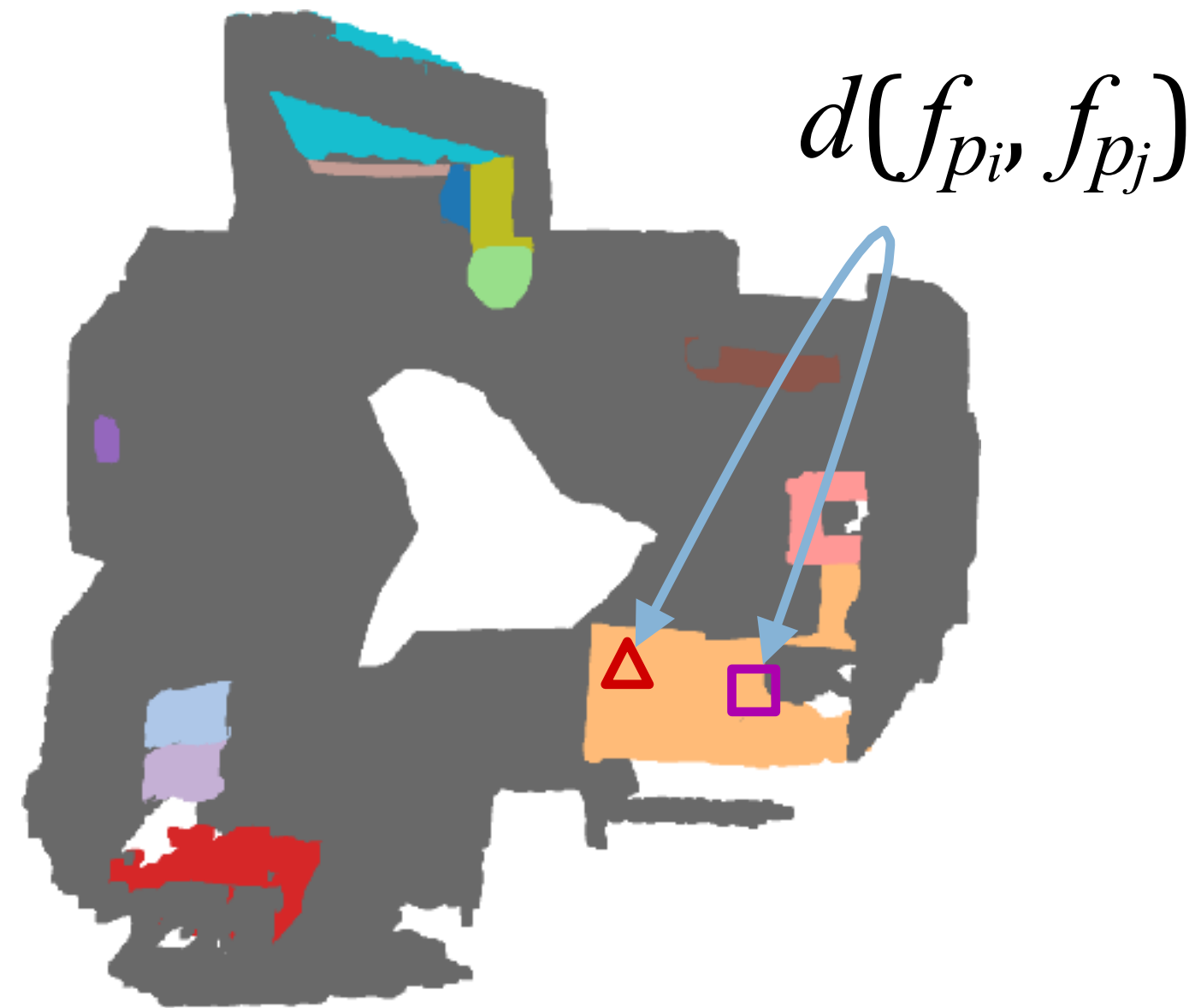


Grouping-based Instance Segmentation



Key Ideas

- Points in the same instance should be close in the **feature** space, such that clustering can be applied.



Distance in Feature Space

- Common choice: L_p -distance
 - e.g., L_1 -distance: $\|F_i - F_j\|_1$
- Potential features to consider:
 - Semantic features (about semantic label)
 - Spatial feature (about point location)
 - Instance feature (to distinguish instances)

Candidate I: Semantic Feature

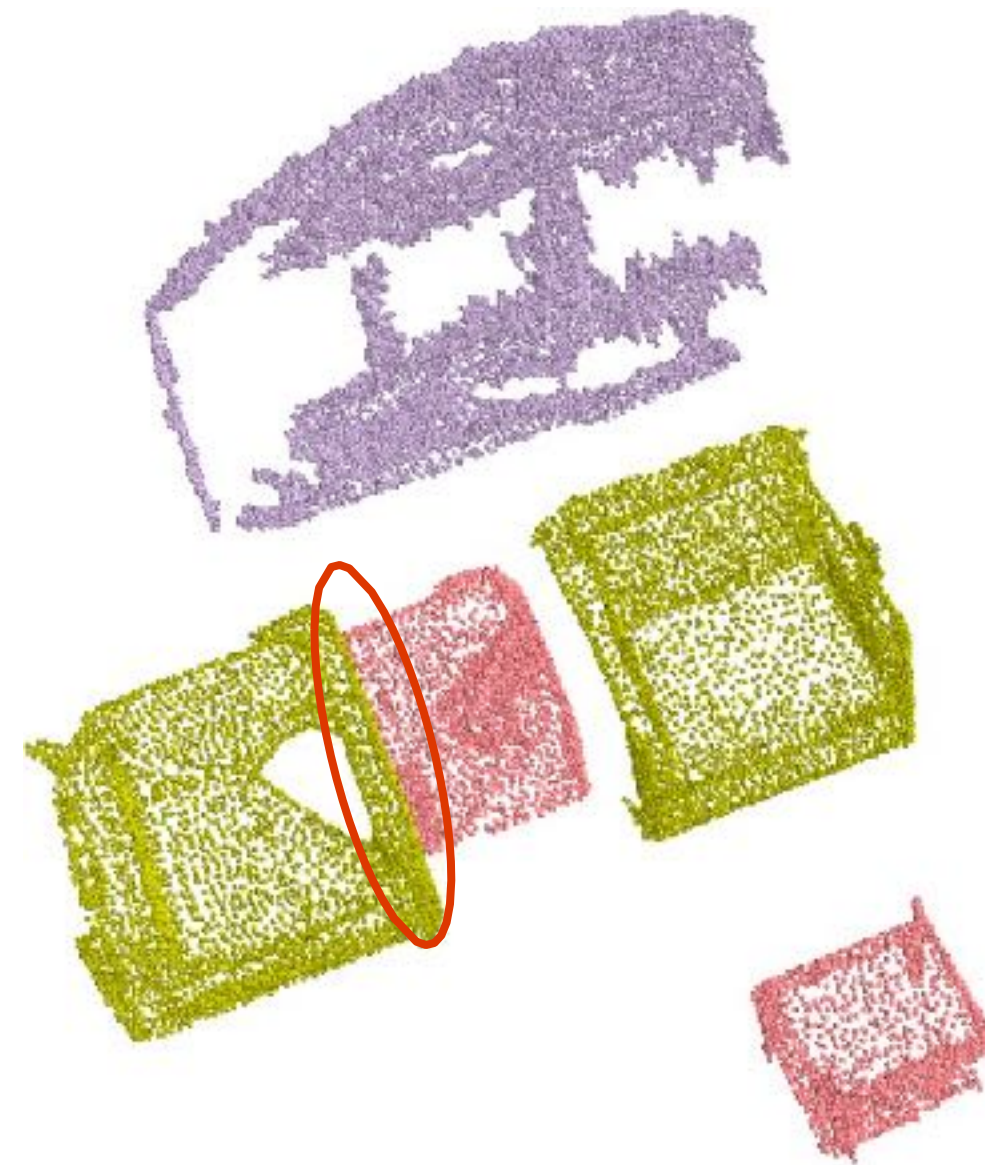
- Learn semantic feature for each point by point cloud segmentation loss.



MLAJiang, Li, et al. "Pointgroup: Dual-set point grouping for 3d instance segmentation." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020.

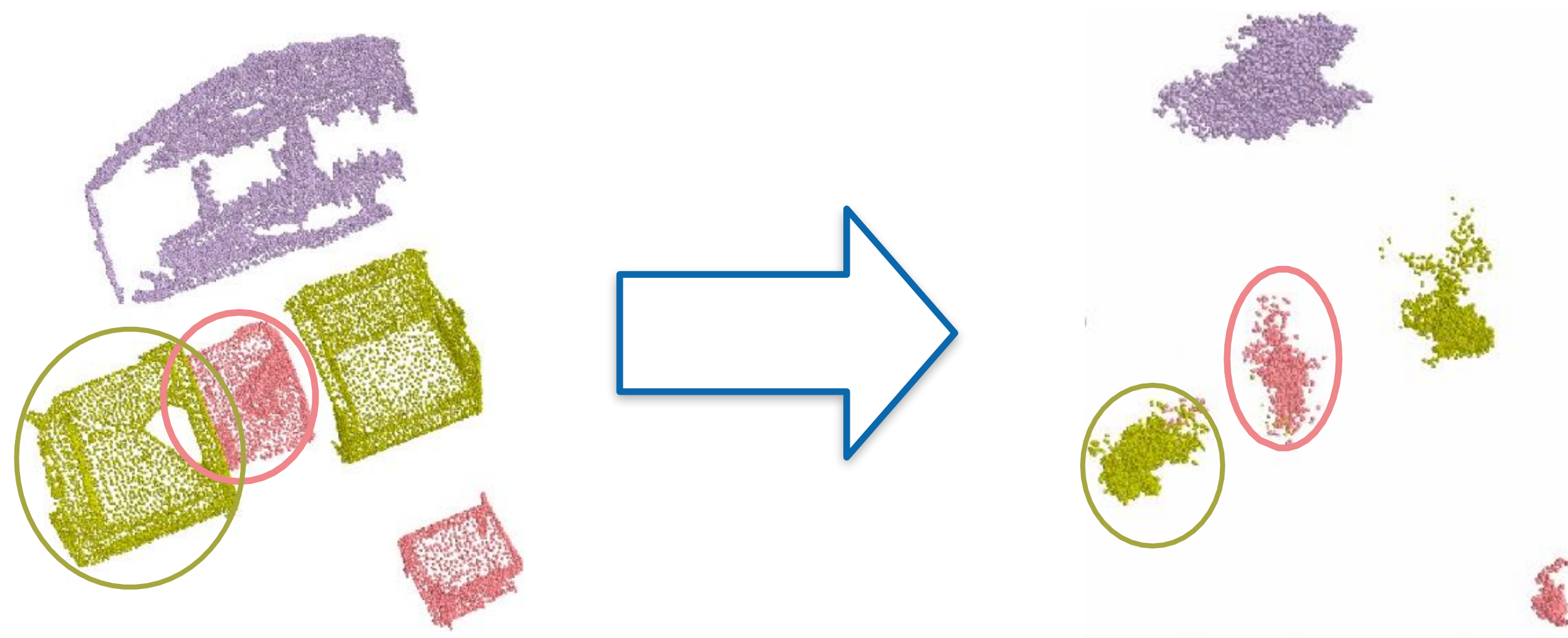
Candidate II: Spatial Feature

- Use 3D coordinates of points?
 - Reasonable, however,
 - Fails for points around object boundaries



Candidate II: Spatial Feature

- Learn to predict object center coordinates, and use the predicted object center as the spatial feature

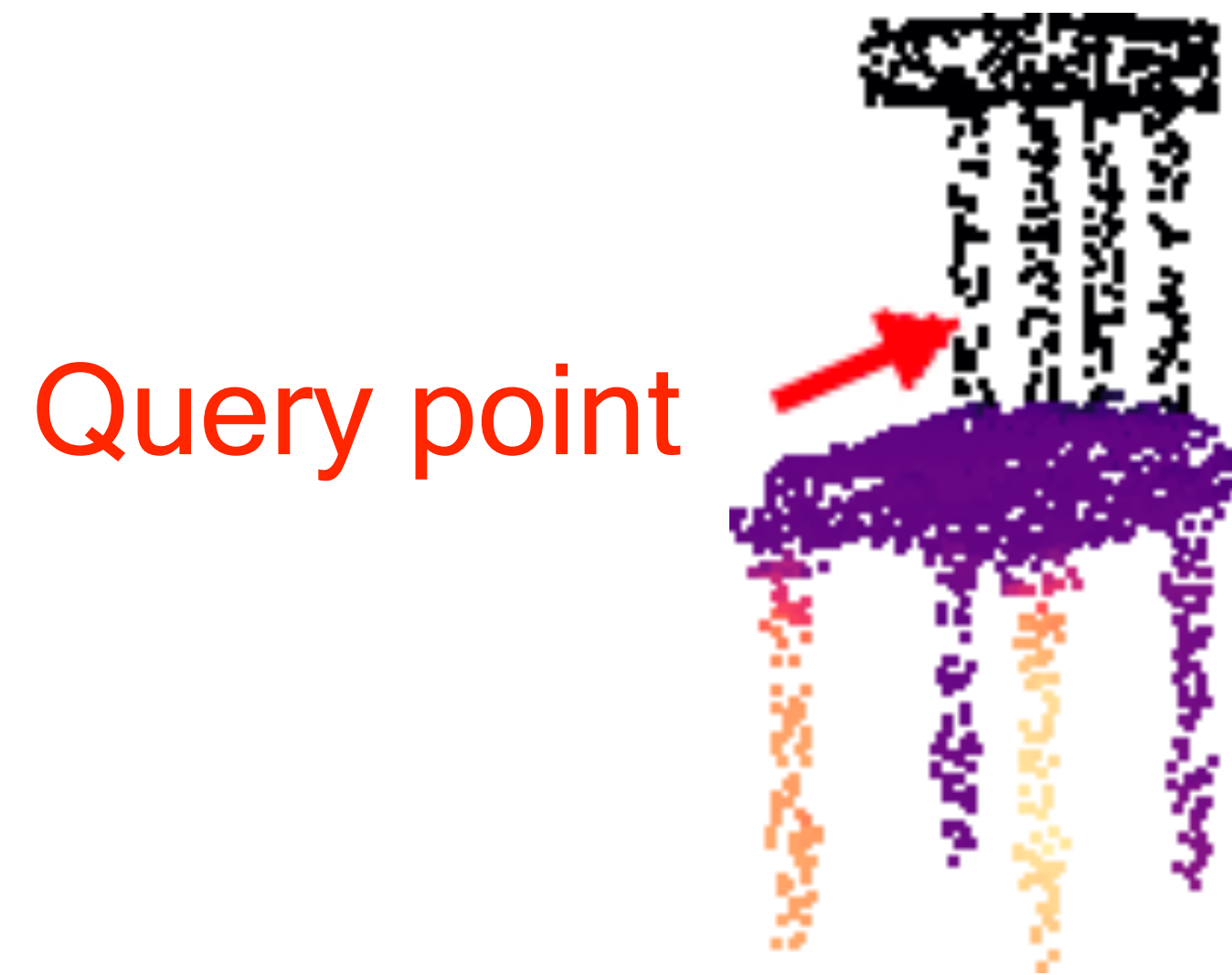


Predicted Object Centers

MLAJiang, Li, et al. "Pointgroup: Dual-set point grouping for 3d instance segmentation." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020.

Candidate III: Instance Features

- Fundamentally, we hope that the feature can be powerful enough to distinguish different instances
- Why not directly design a loss to learn it?!



Color map of distances between the given point and rest points (darker means closer)

Contrastive Loss

- Build loss for each **pair of points** to train point features.



Semantic label



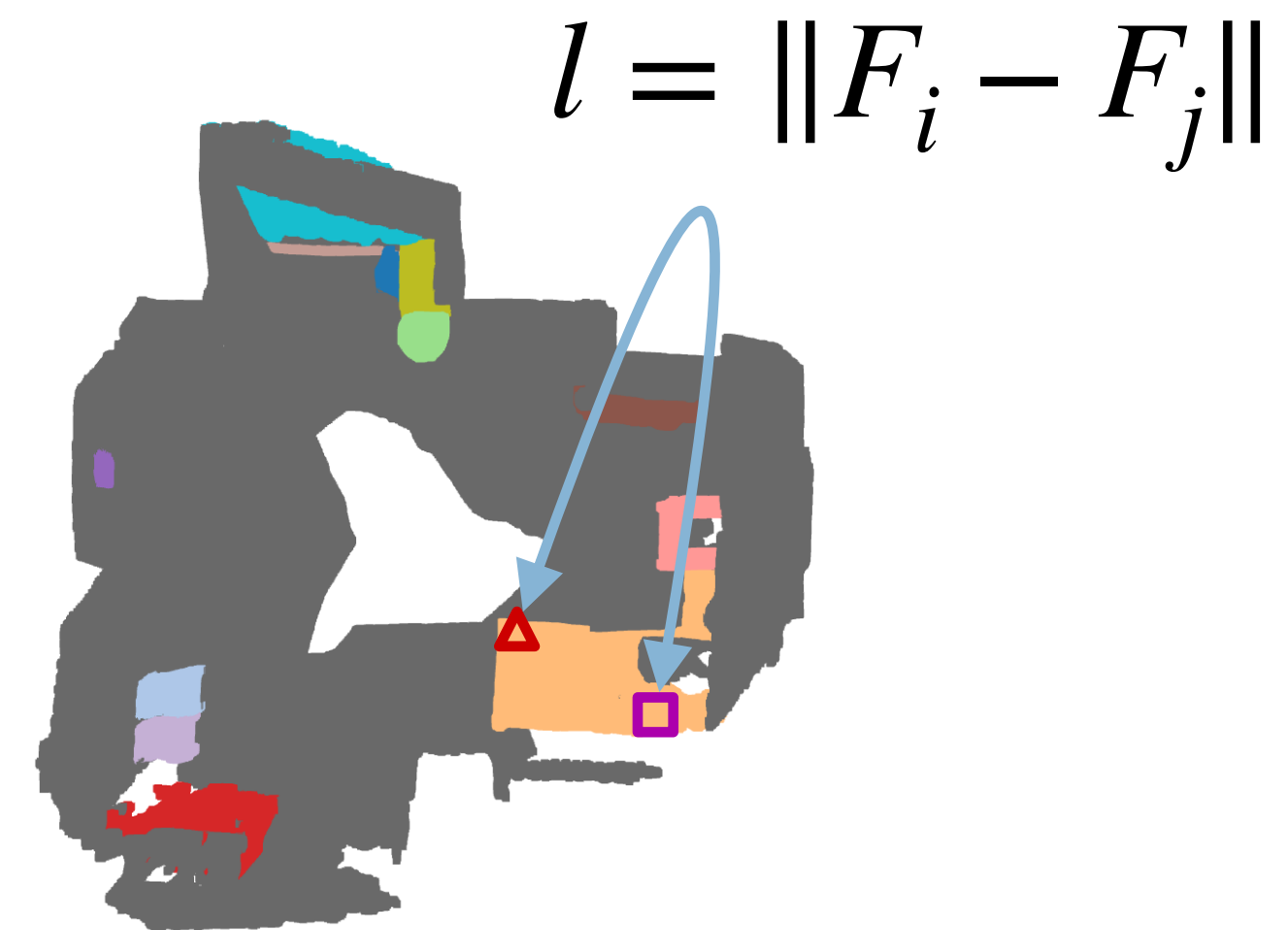
Instance label

Same Instance Case

- Point i and point j belongs to in the same instance.



Semantic label



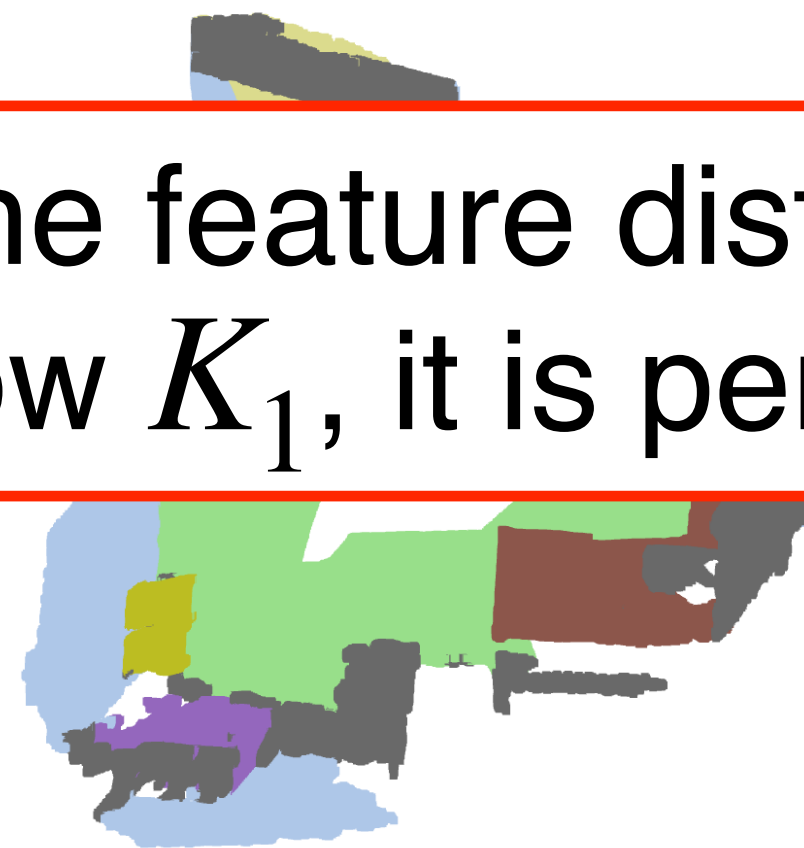
Instance label

Same Instance Case

- Point i and point j belongs to different instances with the same semantic label.

$$l(i, j) = \alpha \max(0, K_1 - \|F_i - F_j\|)$$

“If the feature distance is below K_1 , it is penalized”

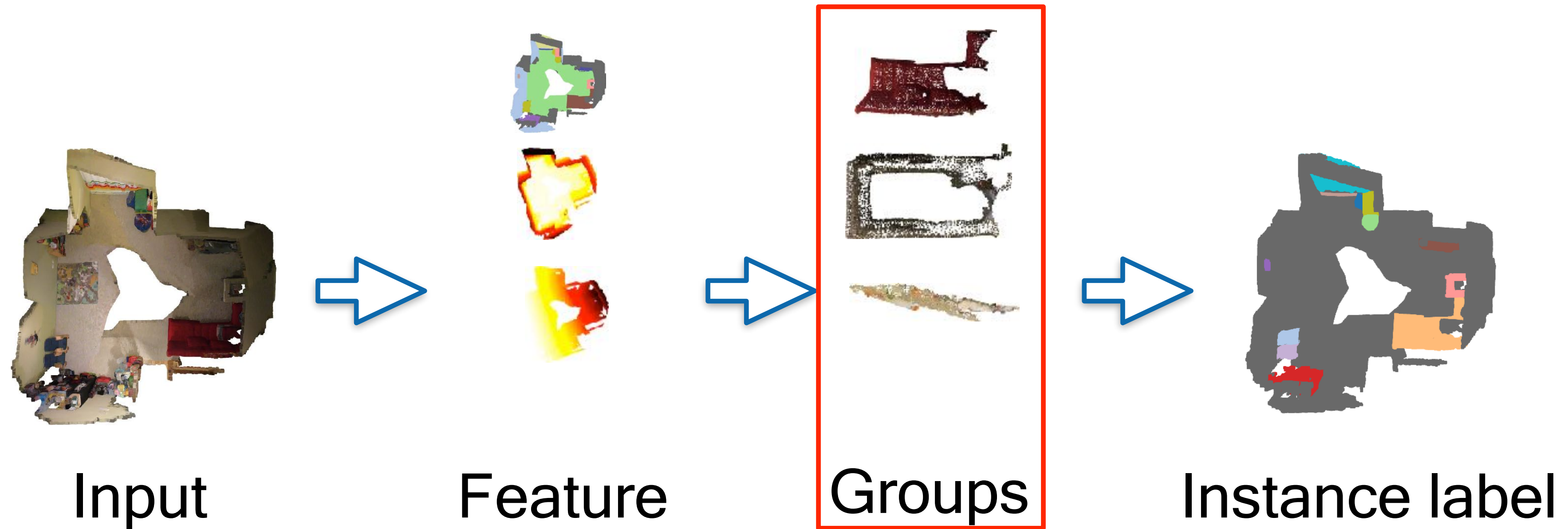


Semantic label



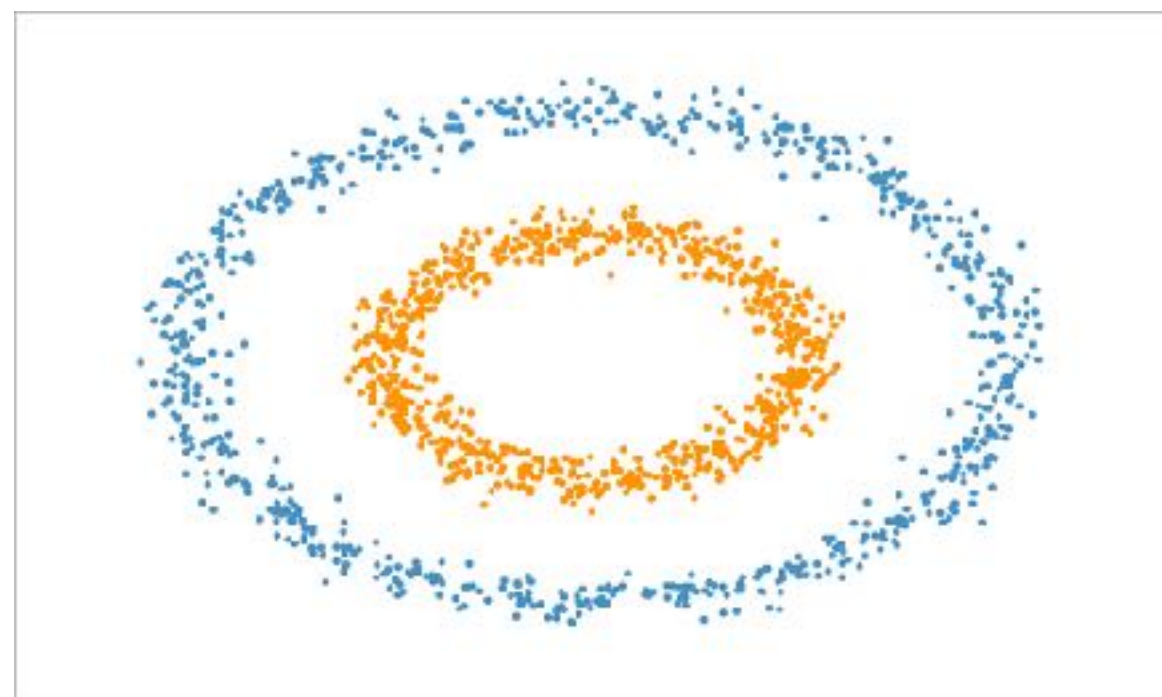
Instance label

Grouping-based Instance Segmentation

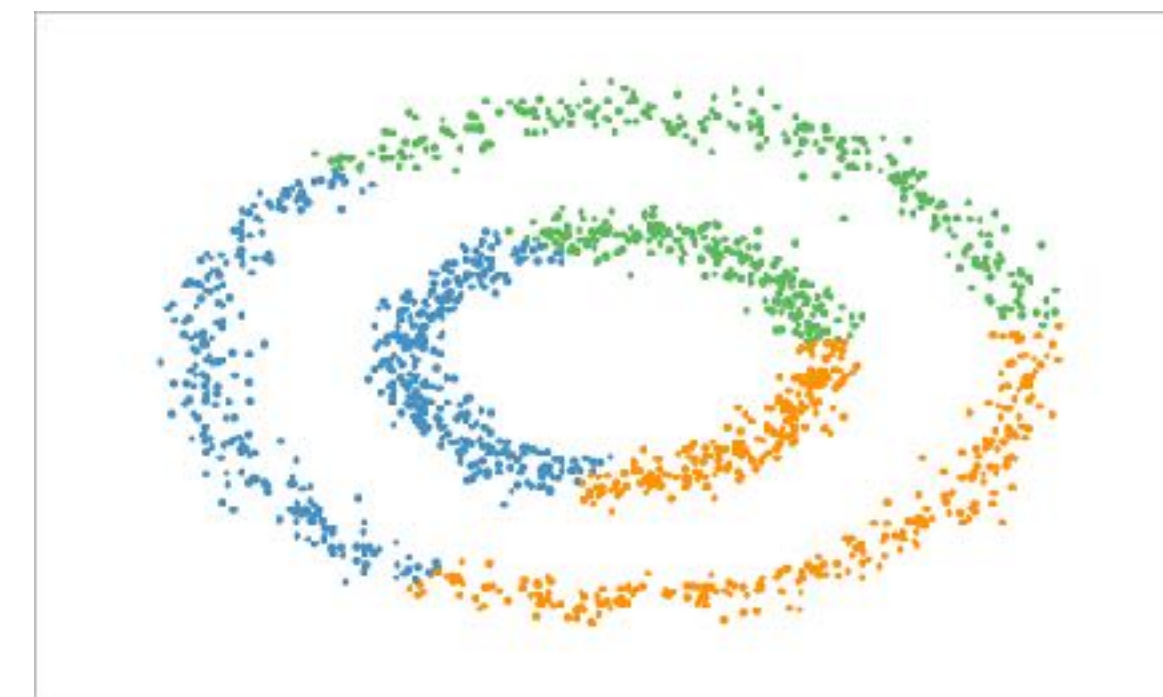


Grouping by Clustering Point Features

- Choose your favorable clustering algorithm
 - DBSCAN
 - Mean shift
 - ...



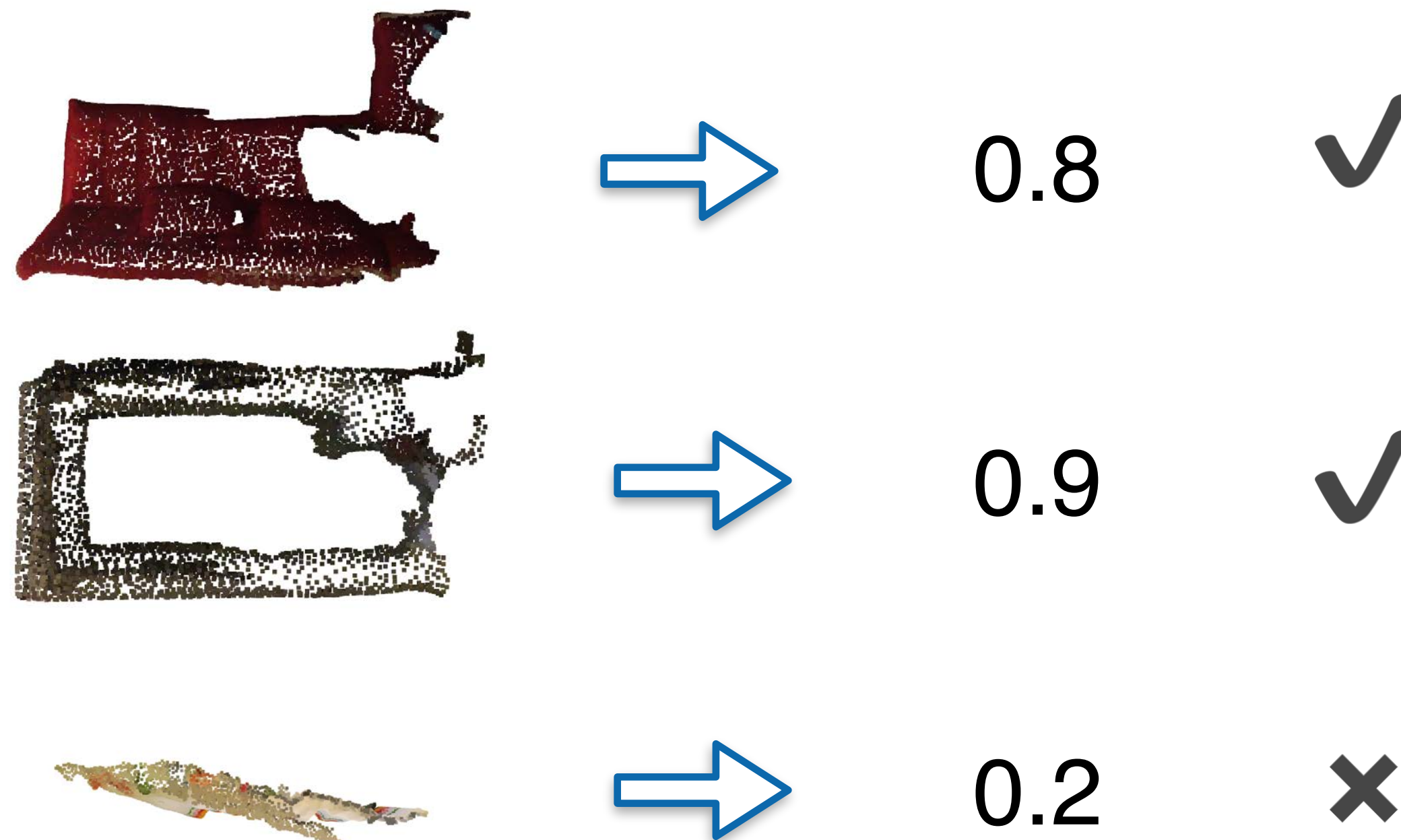
DBSCAN



Mean shift

Final Step: Post-Processing

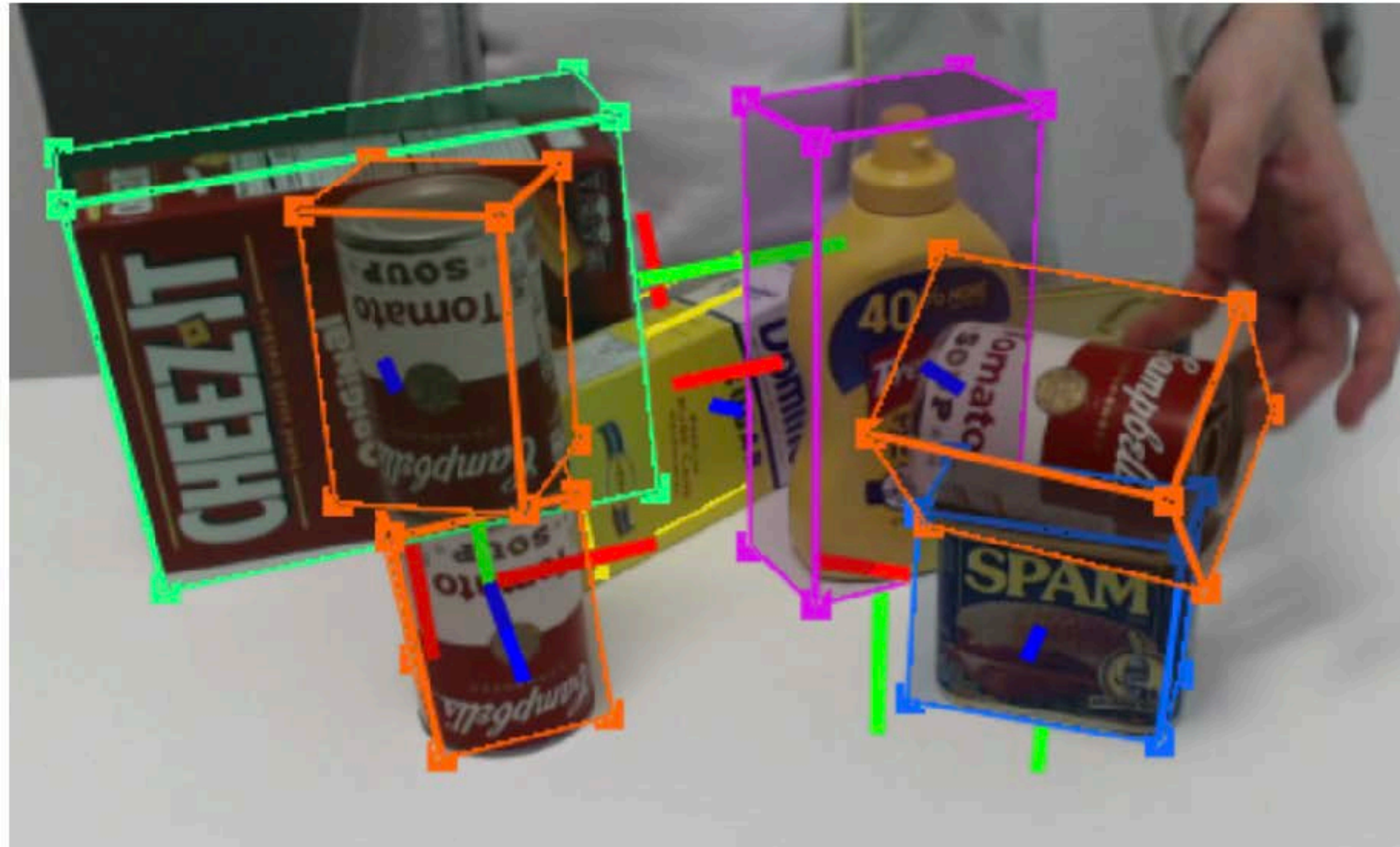
- May also be achieved by learning methods
- e.g., we use a network to predict a score which can represent the IoU between prediction and ground truth, and remove instances with low scores.



Summary

- 3D Object Detection
 - Background
 - The History and Recent Development
 - ✦ Frustum PointNets — Leveraging image proposal for 3D detection
 - ✦ PointPillars — Projecting 3D into 2D with a strong assumption
 - ✦ VoteNet — Clustering based proposal
- 3D Instance Segmentation
 - Top-Down Approaches — Proposal generation + point association
 - Bottom-Up Approaches — Careful feature learning for grouping purposes

6D Pose Estimation



Pose Estimation

We aren't talking about human pose

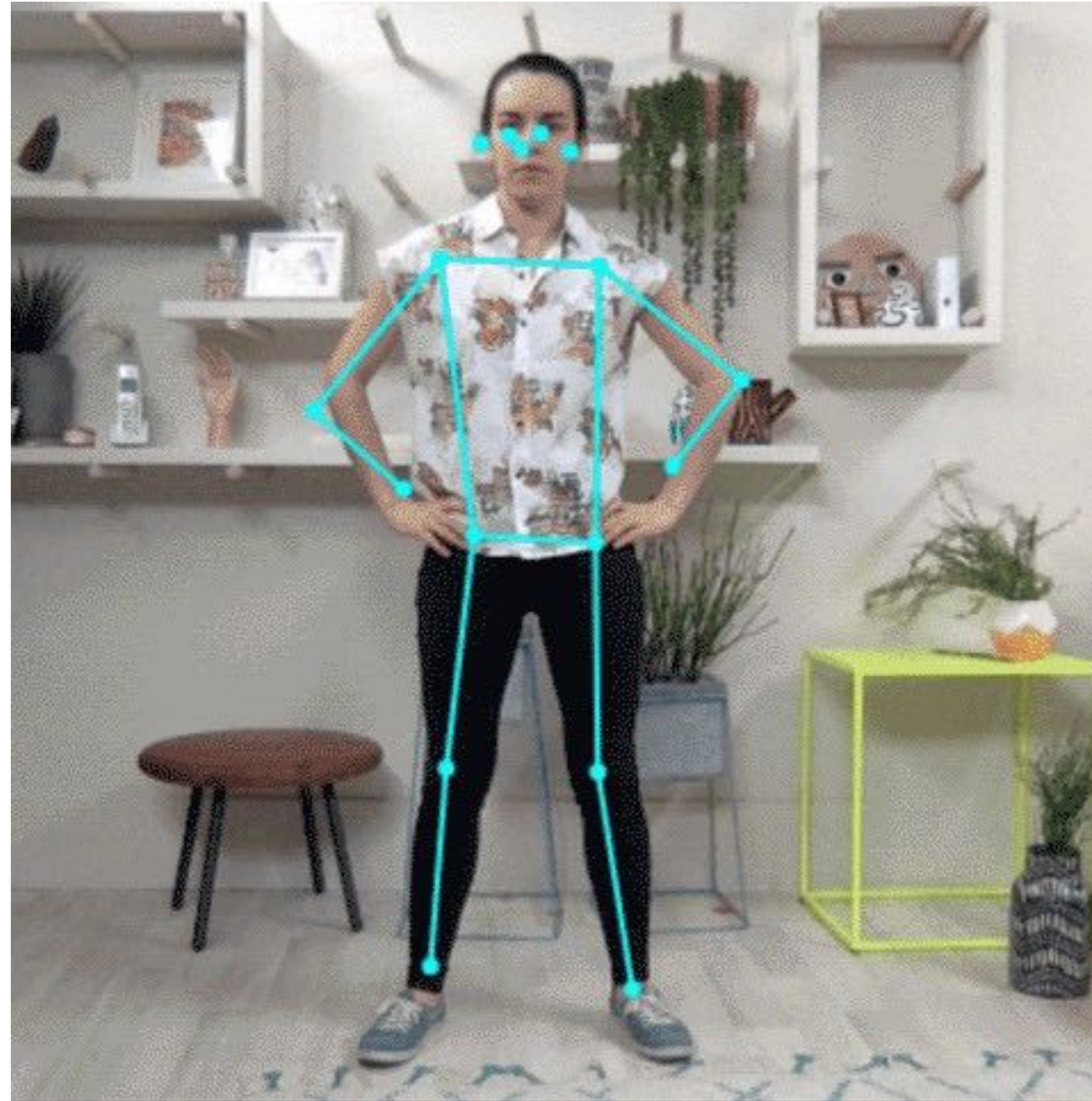


Figure from https://www.tensorflow.org/lite/models/pose_estimation/overview

We are talking about object pose

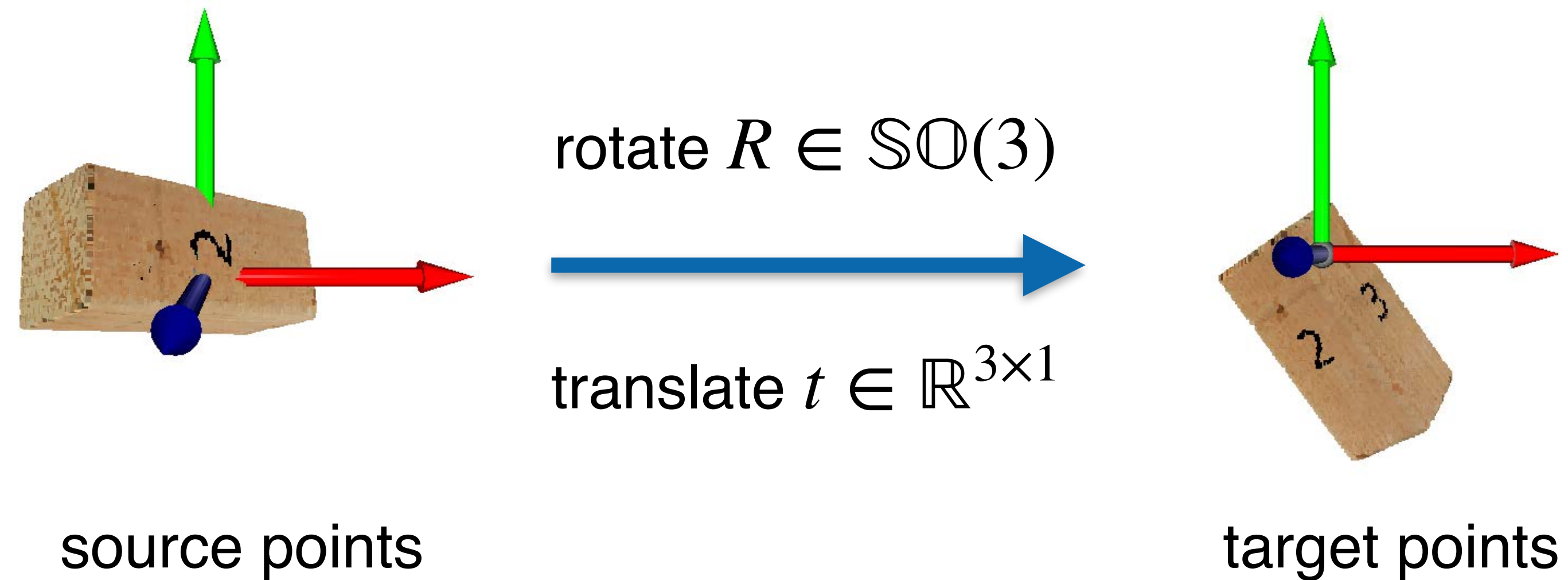


Figure from <https://paperswithcode.com/task/6d-pose-estimation>

Outline

- Introduction
- Rigid transformation estimation
 - Closed-form solution given correspondence
 - Iterative closet point (ICP)
- Learning-based approaches
 - Direct approaches
 - Indirect approaches

Rigid Transformation

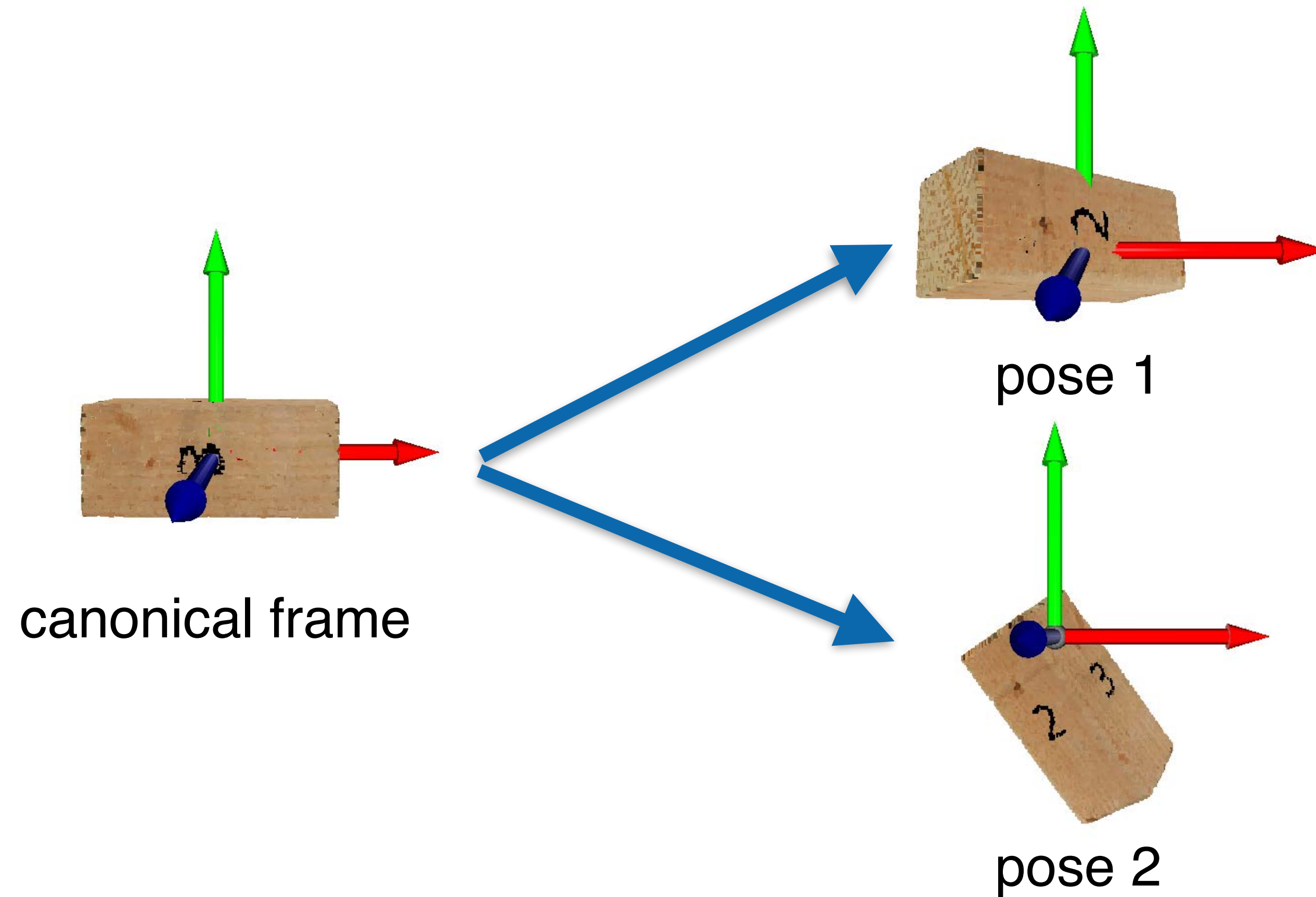


Transformation is relative!

Rigid Transformation

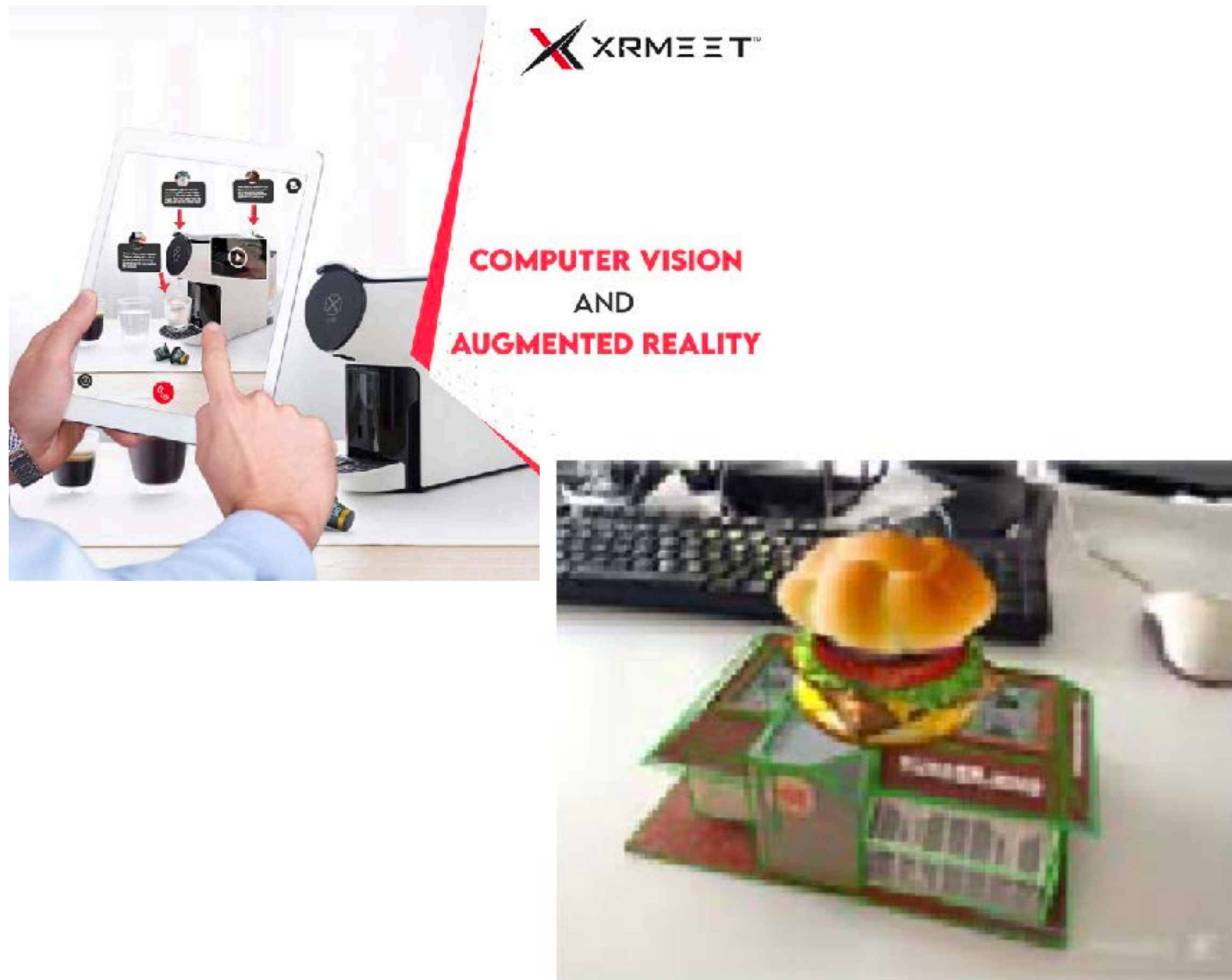
- Rigid transformation $T(p) = Rp + t$, where $p \in \mathbb{R}^{3 \times 1}$
- Represented by a rotation $R \in \mathbf{SO}(3)$, and a translation $t \in \mathbb{R}^{3 \times 1}$
- All the rigid transformations $\{T\}$ form the special Euclidean group, denoted by $\mathbf{SE}(3)$

6D Pose Estimation



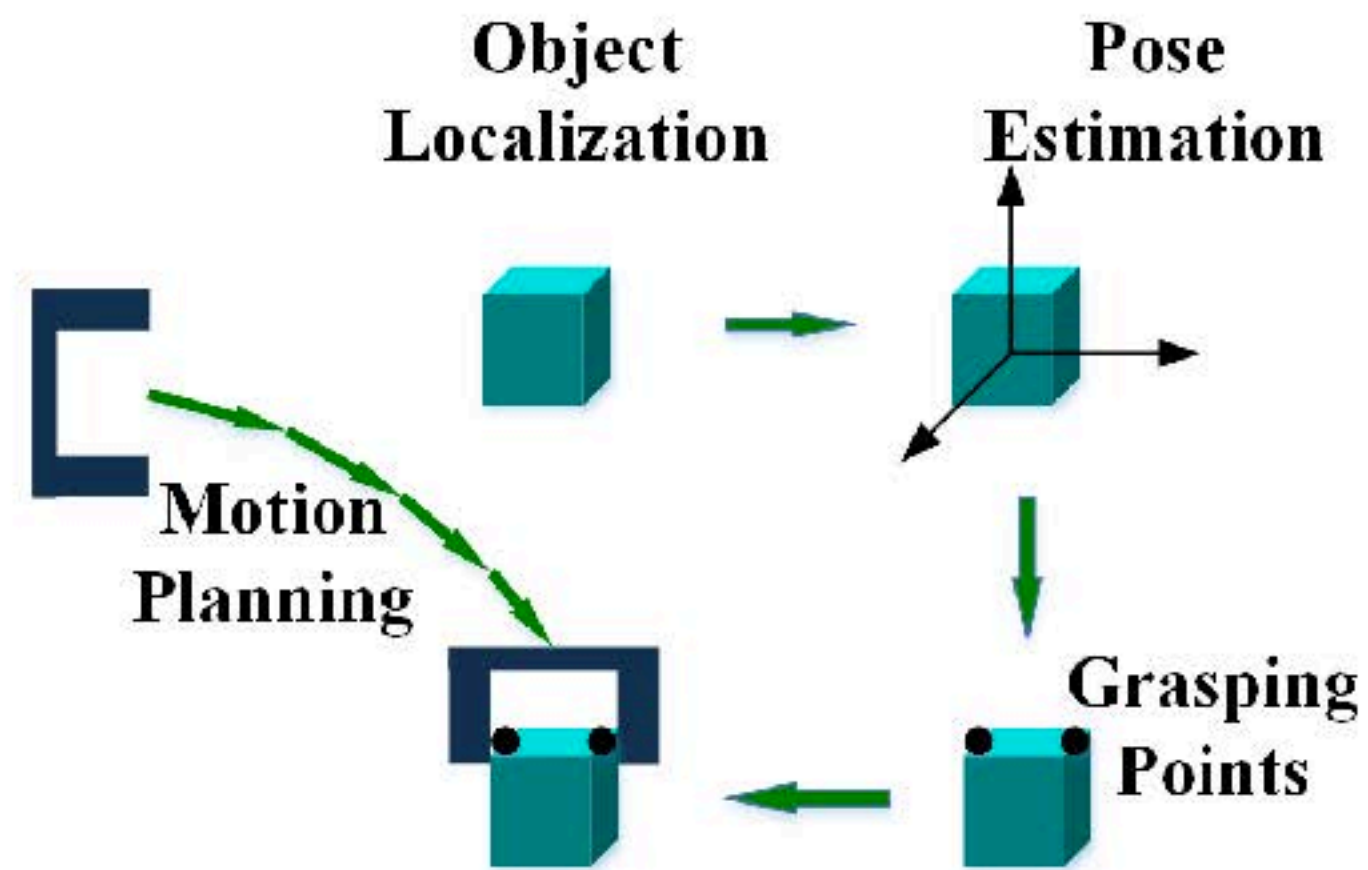
recognize the 3D location and orientation
of an object relative to a canonical frame

Applications: Augmented Reality



- **AR:** immersive interaction for **metaverse**
- visual guidance and E-learning
- mobile apps and entertainment

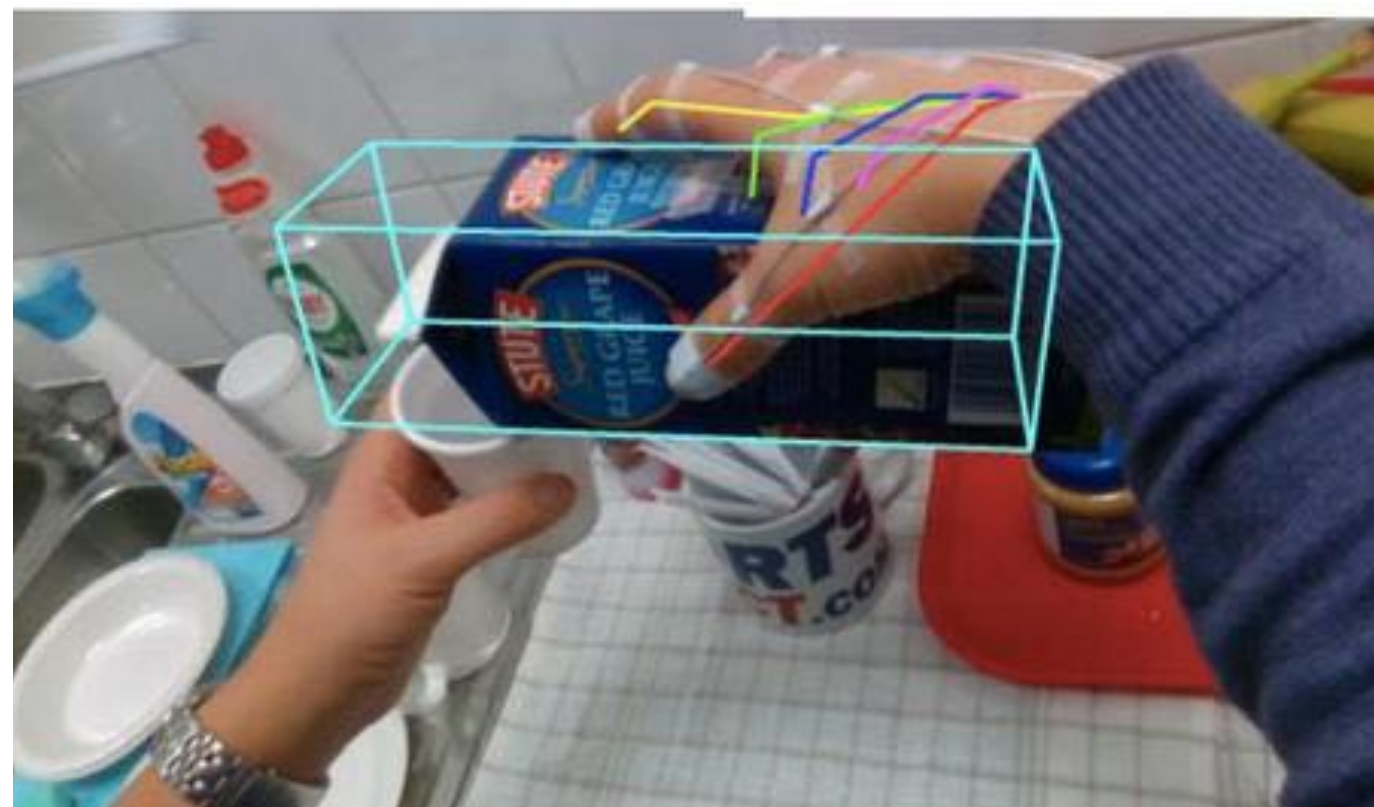
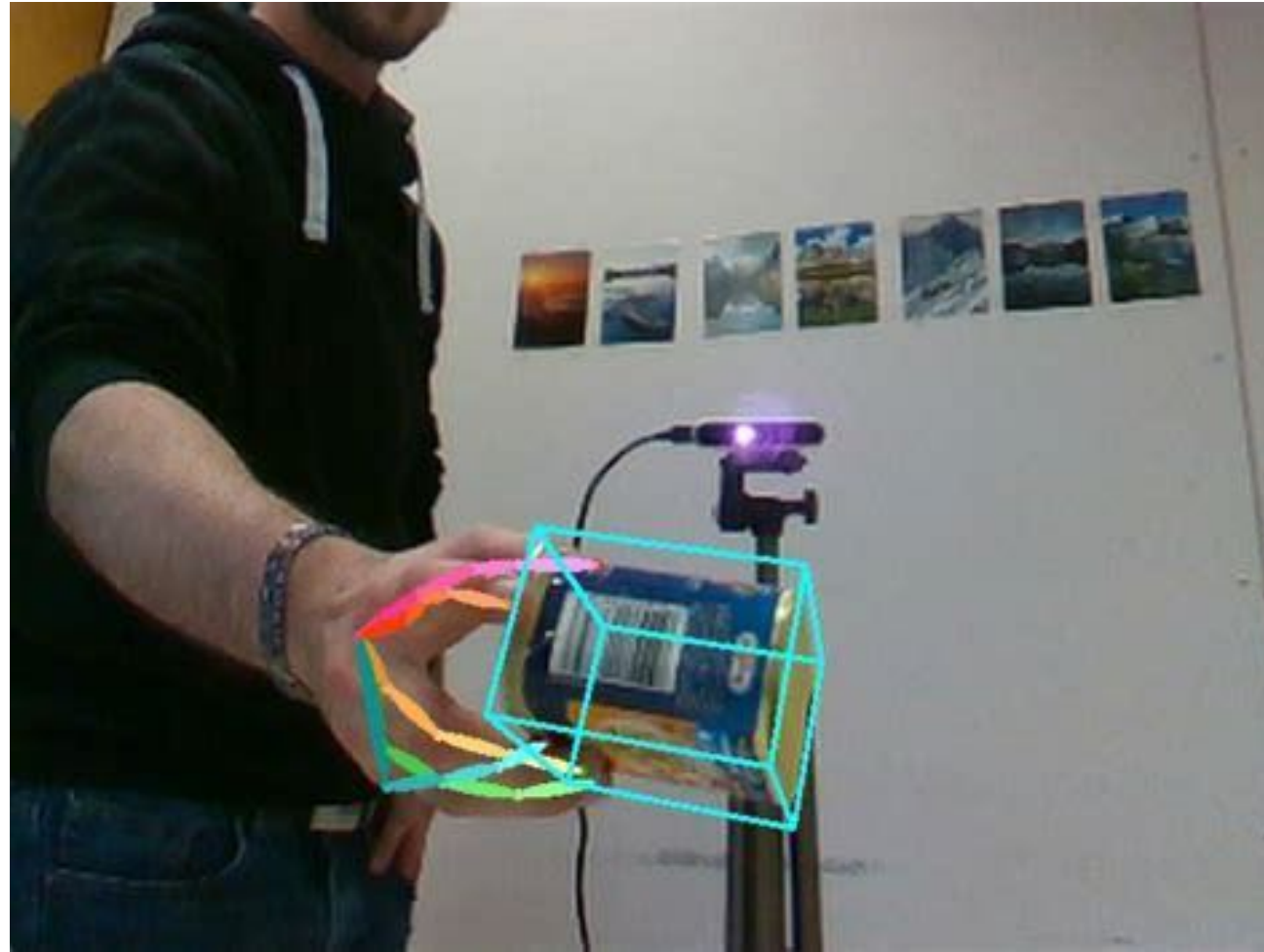
Applications: 6D Pose for Robotic Manipulation



- Object pose helps to transfer annotation from CAD model to the object in the camera space
- Pose tracking serves as visual closed-loop control



Applications: 6D Pose for Human Object Interaction



- **Vision** perspective: to perceive and understand human object interaction
- **Robotic** perspective: to extract pose trajectories for imitation learning

Hampali, S. , et al. "HOnnotate: A method for 3D Annotation of Hand and Objects Poses." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* , 2020.

Doosti, B. , et al. "HOPE-Net: A Graph-based Model for Hand-Object Pose Estimation." *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* , 2020.

6D Pose & Rigid Transformation

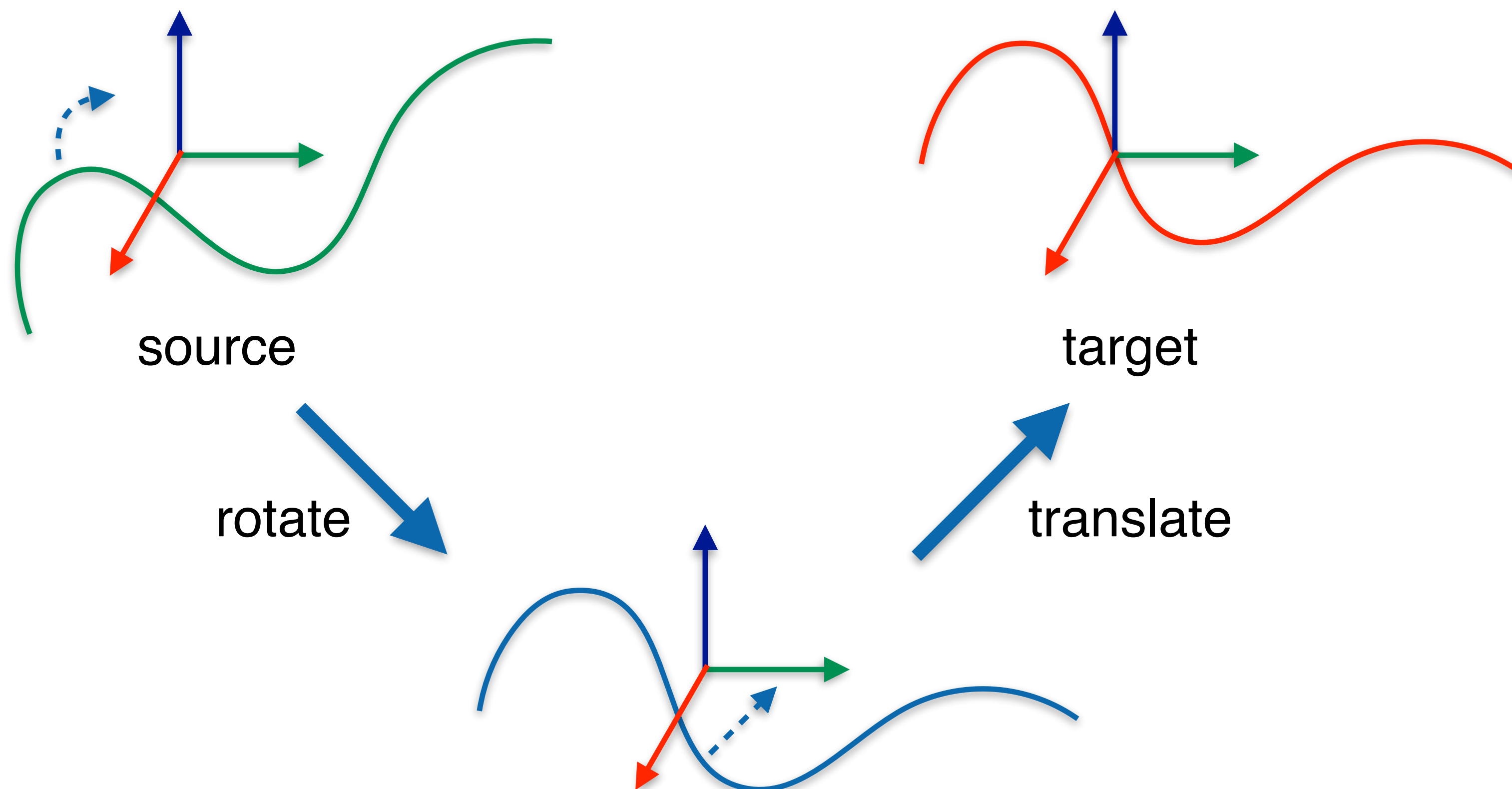
- 6D pose: object-level rigid transformation, associated with a canonical frame
- rigid transformation: can be object-level or scene-level, no predefined canonical frame

Outline

- Introduction
- Rigid transformation estimation
 - Closed-form solution given correspondence
 - Iterative closet point (ICP)
- Learning-based approaches
 - Direct approaches
 - Indirect approaches

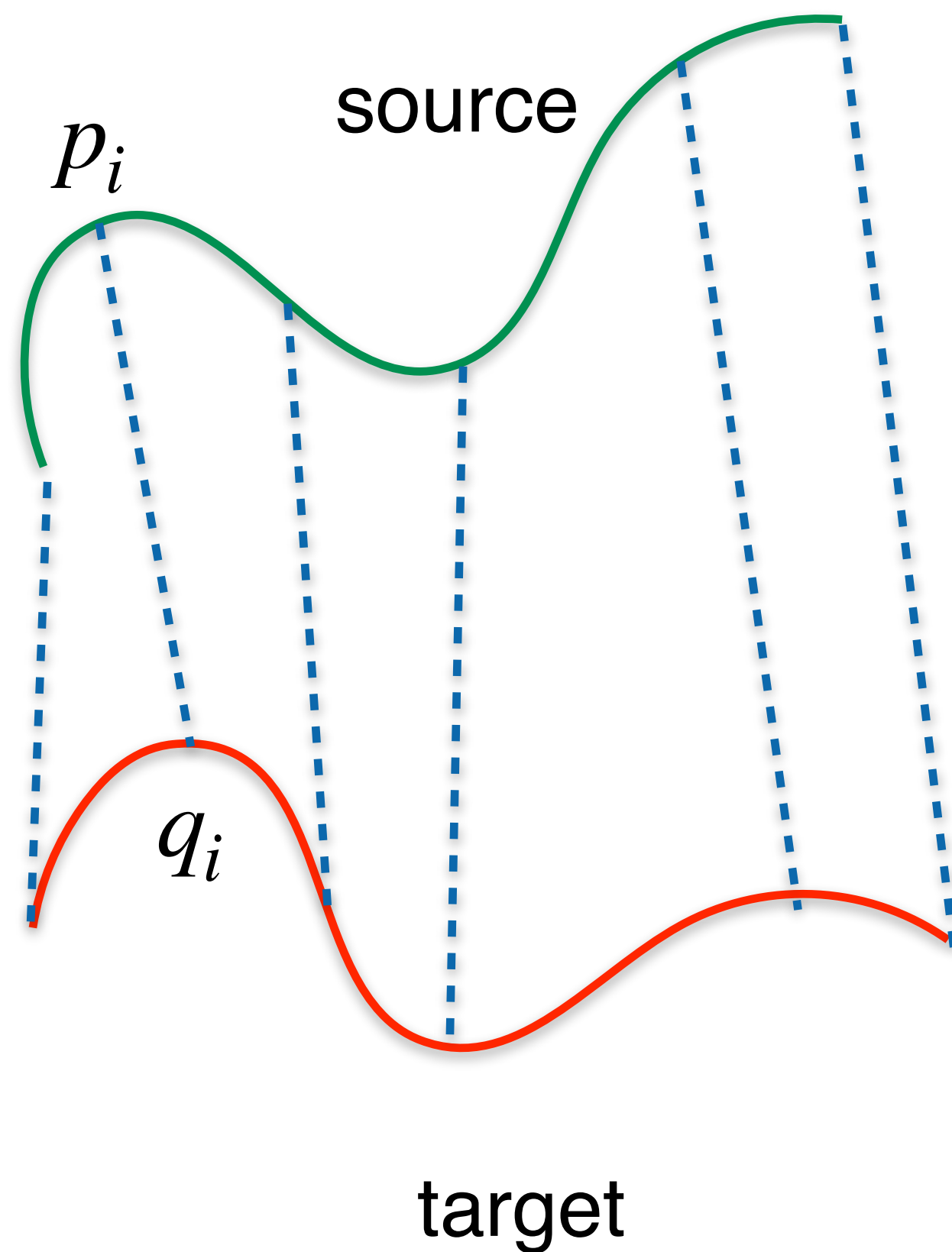
Rigid Transformation Estimation

$$\text{Rigid transformation } T(p) = Rp + t$$



Correspondence

Rigid transformation $T(p) = Rp + t$



$$q_1 = T(p_1) = Rp_1 + t$$

$$q_2 = T(p_2) = Rp_2 + t$$

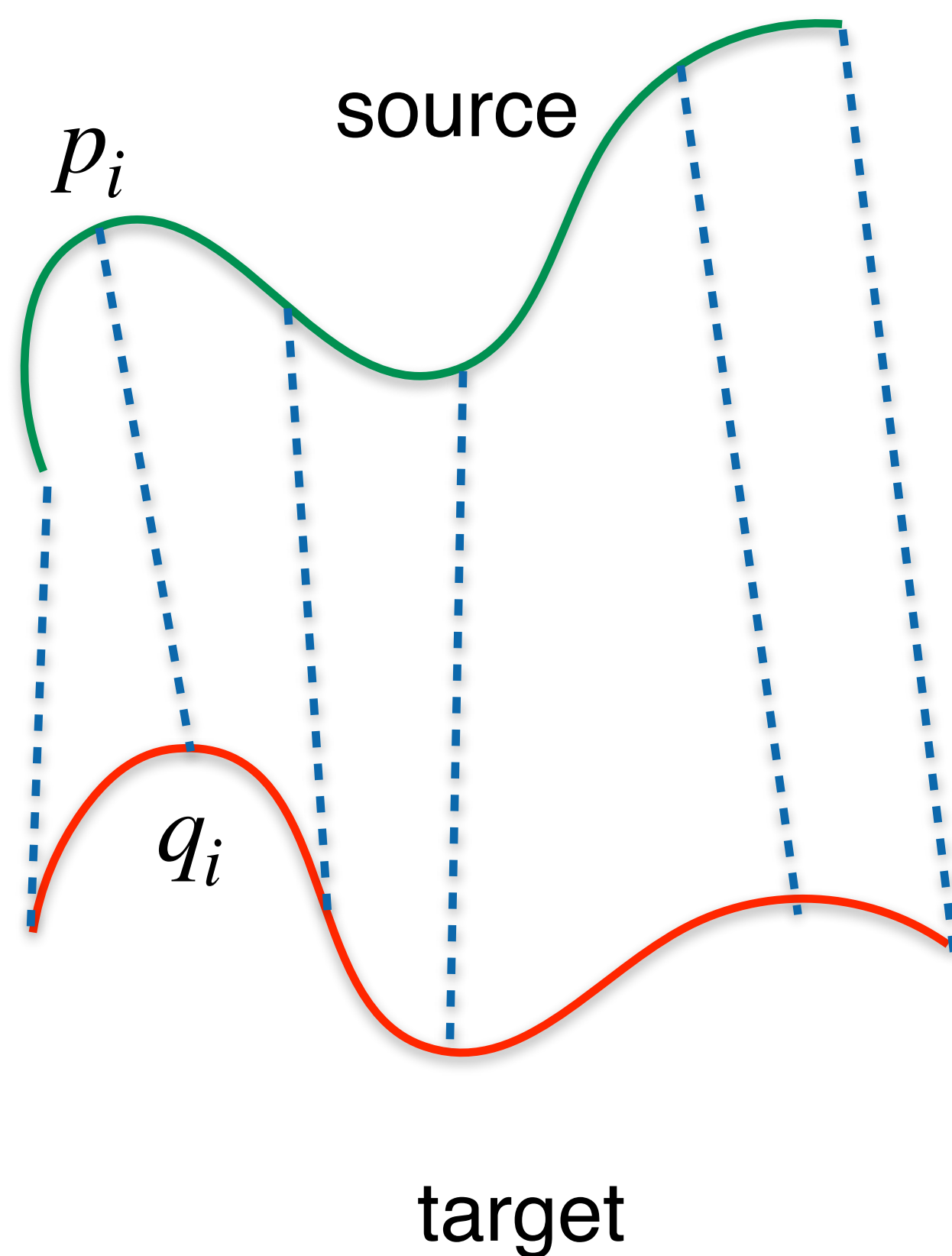
$\vdots$

$$q_n = T(p_n) = Rp_n + t$$

3n equations from
n pairs of points

Estimate from Correspondence

Rigid transformation $T(p) = Rp + t$



How many pairs of points are required to uniquely define a rigid transformation?

Two Key Steps

- Find the correspondence between source and target
 - combinatorial problem
 - greedy heuristic or exhaustive search
- Estimate the rigid transformation given the correspondence
 - constrained (for rotation) optimization
 - closed-form solution

Two Key Steps

- Find the correspondence between source and target
 - combinatorial problem
 - greedy heuristic or exhaustive search
- Estimate the rigid transformation given the correspondence
 - constrained (for rotation) optimization
 - closed-form solution

Math is coming

Least-square Estimation of Rigid Transformation

- Given source points $P = \{p_i\}$, and target points $Q = \{q_i\}$, the objective (least-square error) is:

$$L = \sum_{i=1}^n \|Rp_i + t - q_i\|^2$$

Optimization Problem

- Parameters: $R \in \mathbb{SO}(3)$ and $t \in \mathbb{R}^{3 \times 1}$
- Target point cloud: $Q = \{q_i\}$
- Source point cloud: $P = \{p_i\}$
- Objective: $\min_{R,t} \sum_{i=1}^n \|Rp_i + t - q_i\|^2$

Step I: Representing t by R

- Assuming $R \in \mathbb{SO}(3)$ is known,

- Recall the objective $L = \sum_{i=1}^n \|Rp_i + t - q_i\|^2$

- Calculate the gradient $\frac{\partial L}{\partial t} = \sum_{i=1}^n (Rp_i + t - q_i)$

- Solve $\frac{\partial L}{\partial t} = 0$:

$$t = \frac{\sum_{i=1}^n q_i}{n} - \frac{\sum_{i=1}^n Rp_i}{n}$$

Step I: Representing t by R

- Substituting the $t \in \mathbb{R}^{3 \times 1}$ expressed by R , the objective can be simplified as

$$L = \sum_{i=1}^n \|R\bar{p}_i - \bar{q}_i\|^2,$$

where

$$\bar{p}_i = p_i - \frac{\sum_{i=1}^n p_i}{n},$$
$$\bar{q}_i = q_i - \frac{\sum_{i=1}^n q_i}{n}$$

Step II: Solve R

Objective:
$$L = \sum_{i=1}^n \|R\bar{p}_i - \bar{q}_i\|^2$$

$$\Rightarrow R^* = \operatorname{argmin}_R \|RP - Q\|_F,$$

subject to

$$R^T R = \mathbf{I},$$

$$\det(R) = 1$$

where $P = [p_1, p_2, \dots, p_n] \in \mathbb{R}^{3 \times n},$

$$Q = [q_1, q_2, \dots, q_n] \in \mathbb{R}^{3 \times n}$$

Step II: Solve R

- **Orthogonal Procrustes Problem**

$$\operatorname{argmin}_R \|RP - Q\|_F, \text{ subject to } R^T R = \mathbf{I}$$

- Notice: No determinant constraint
- This problem has a closed form solution!
 - Project $M = QP^T$ to the space of orthogonal matrices
 - Numerically, the magical SVD comes!
 - If $M = U\Sigma V^T$,
 - then $R = UV^T$
- The proof can be found on the [wiki](#)

Step II: Solve R

- How to satisfy the determinant constraint?
- Assume $R = UV^T$
 - If $\det(R) = -1$, then flip the sign of the last column of V
- The proof can be found in [Umeyama's paper](#)

Summary of Closed-Form Solution (Known Correspondences)

- Known: $P = \{p_i\}$, $Q = \{q_i\}$
- Objective: $\min_{R,t} \sum_{i=1}^n \|Rp_i + t - q_i\|^2$
- Solution
 - $\sum_{i=1}^n (q_i - \bar{q})(p_i - \bar{p})^T = U\Sigma V^T$ (SVD)
 - $R = UV^T$ (flip the sign of the last column of V if $\det(R) = -1$)
 - $t = \frac{\sum_{i=1}^n q_i}{n} - \frac{\sum_{i=1}^n Rp_i}{n} := \bar{q} - R\bar{p}$

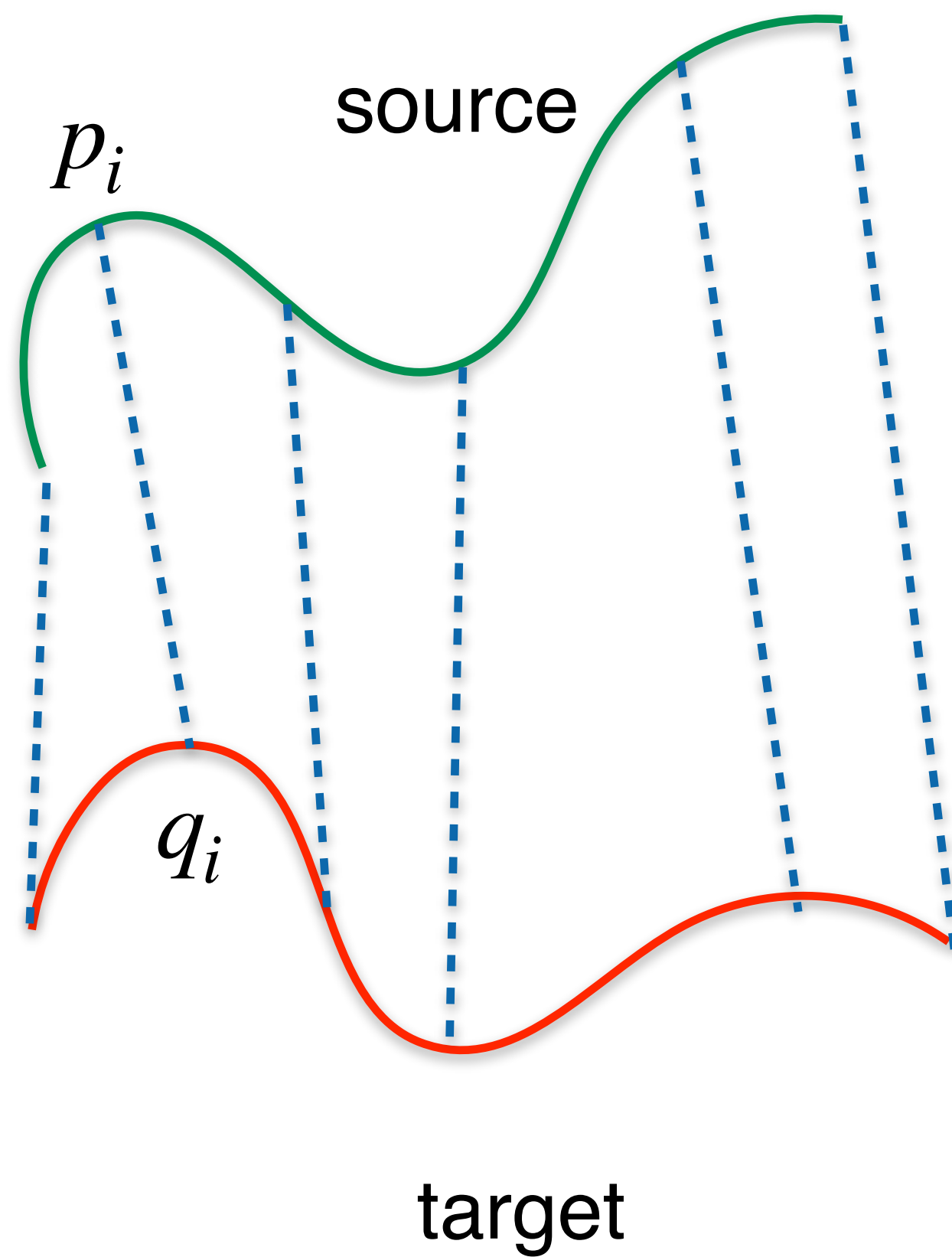
Two Key Steps

- Find the correspondence between source and target
 - combinatorial problem
 - greedy heuristic or exhaustive search
- Estimate the rigid transformation given the correspondence
 - constrained (for rotation) optimization
 - closed-form solution

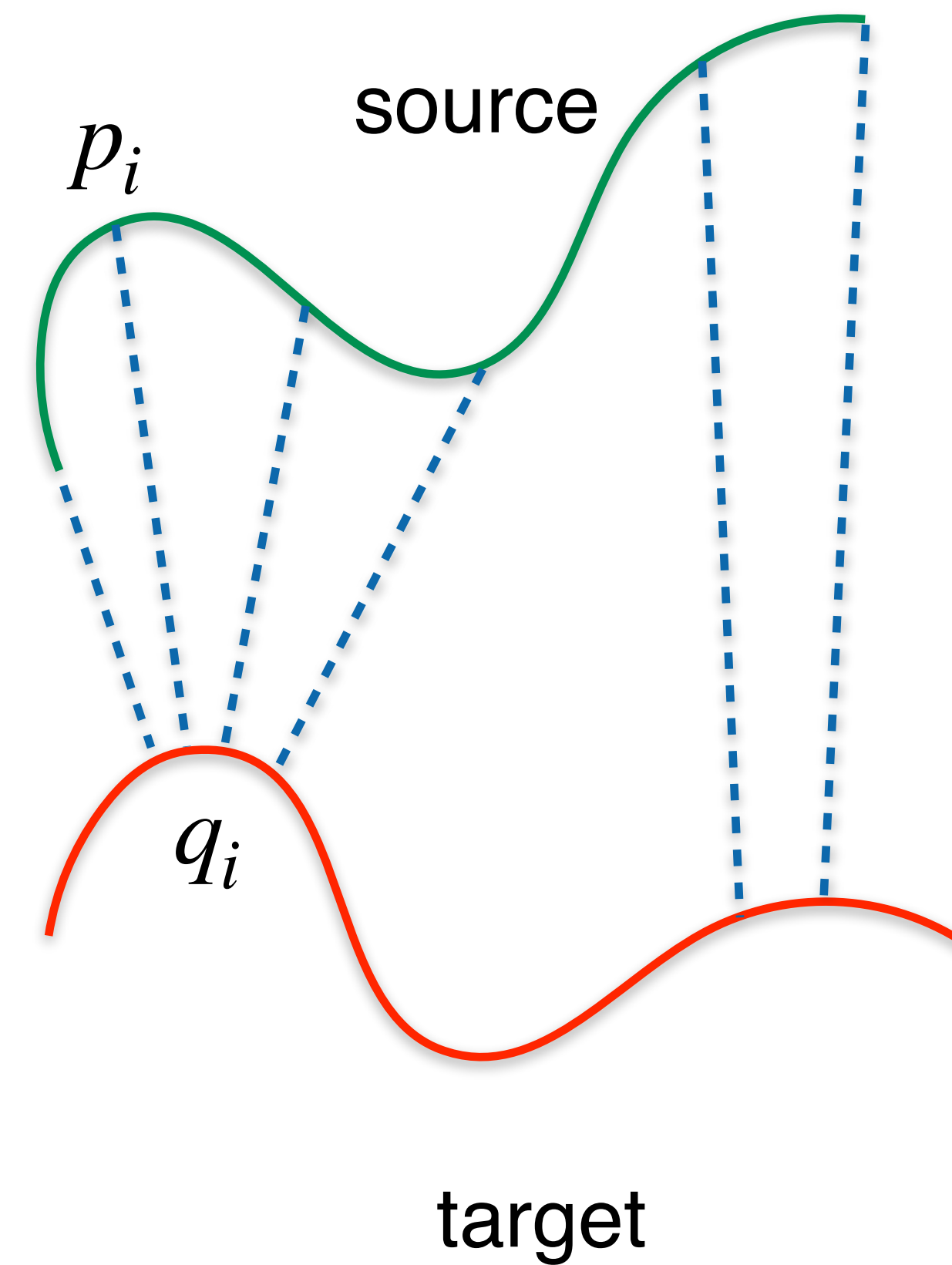
Heuristic

- The closest point might be the corresponding point
- If starting from a transformation close to the actual one, we can iteratively improve the estimation

Find Correspondence

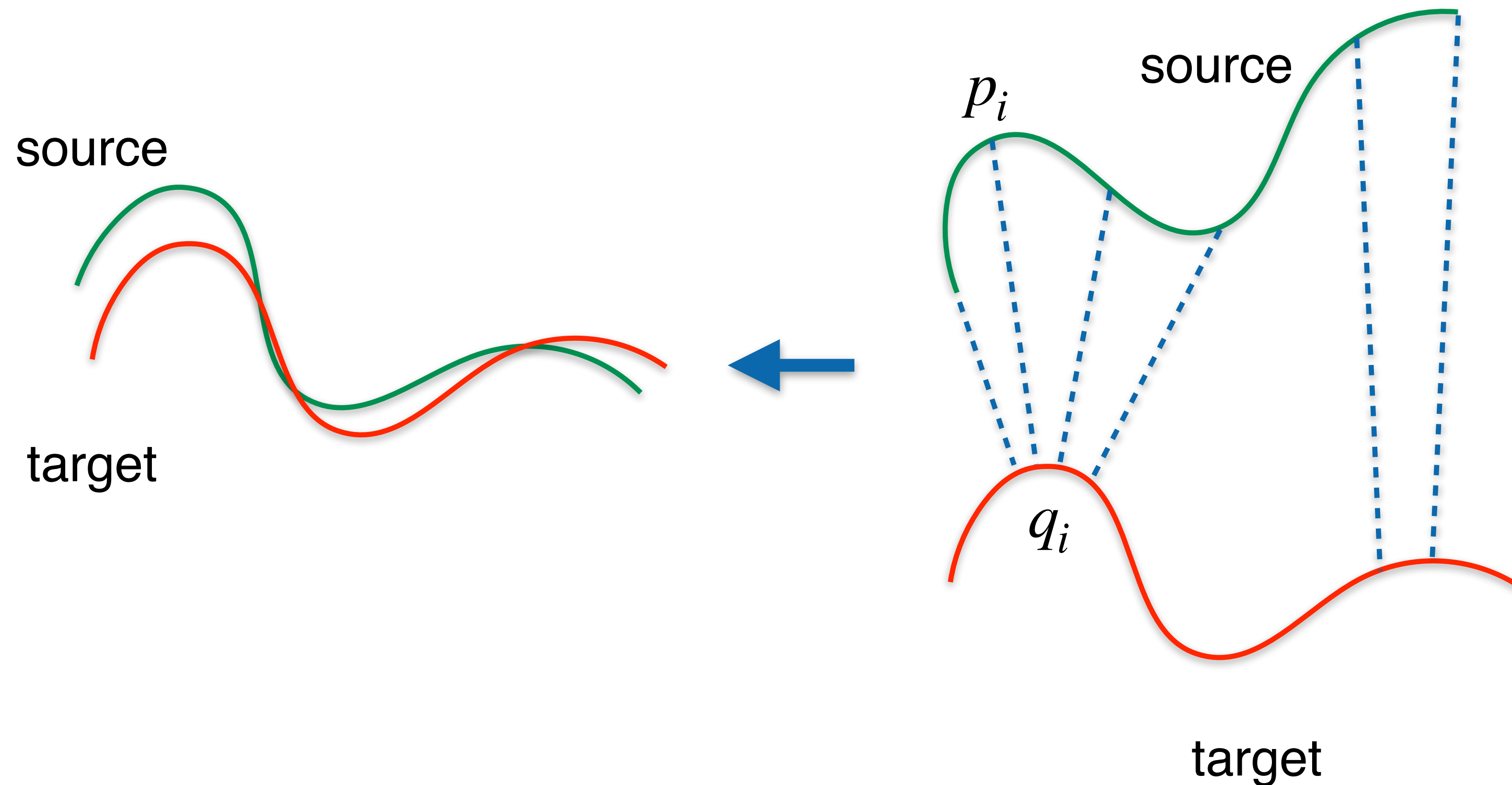


GT correspondence



ICP correspondence

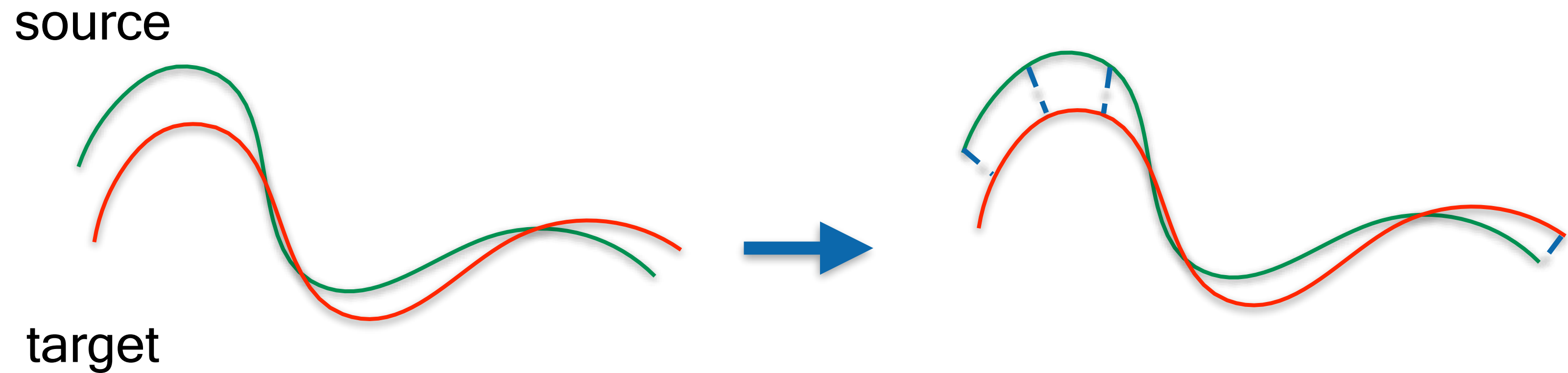
Update Transformation



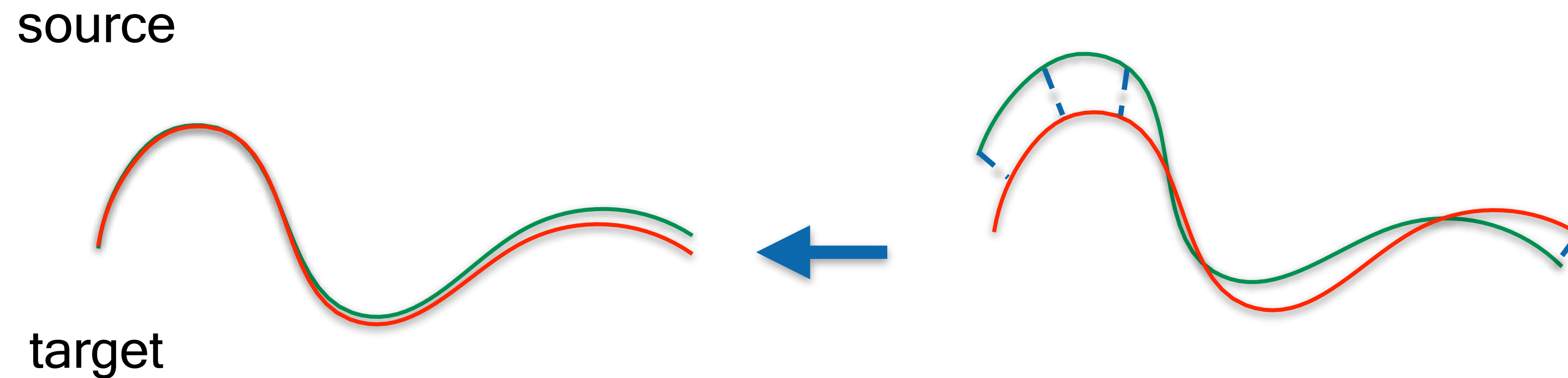
Update transformation by
minimizing the least square error

ICP correspondence

Iterate to Refine



Find correspondence



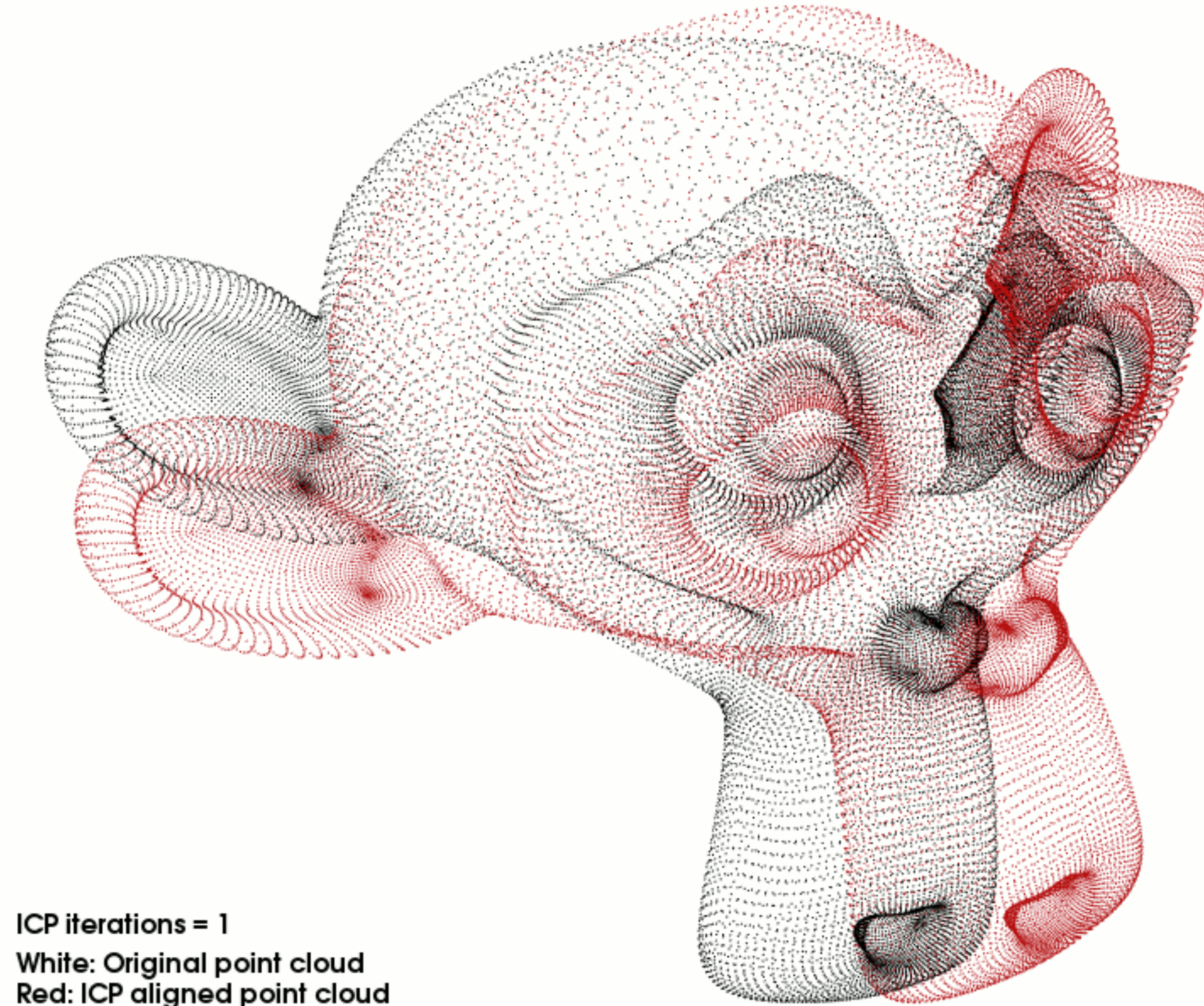
Update transformation

General ICP Algorithm

Starting from an initial transformation $T = (R, t)$

1. Find correspondence: for each point in the source point cloud transformed with current transformation, find the nearest neighbor in the target point cloud
2. Update the transformation by minimizing an objective function $E(T)$ over the correspondence
3. Go to step 1 until the transformation is not updated

Illustration



ICP iterations = 1
White: Original point cloud
Red: ICP aligned point cloud

Animation from <https://github.com/yassram/iterative-closest-point>

Improve ICP

- Objective functions
 - PointToPoint
 - PointToPlane (faster convergence, but requires normal computation)
- Outlier removal: abandon pairs of points with too large distance

Useful Libraries

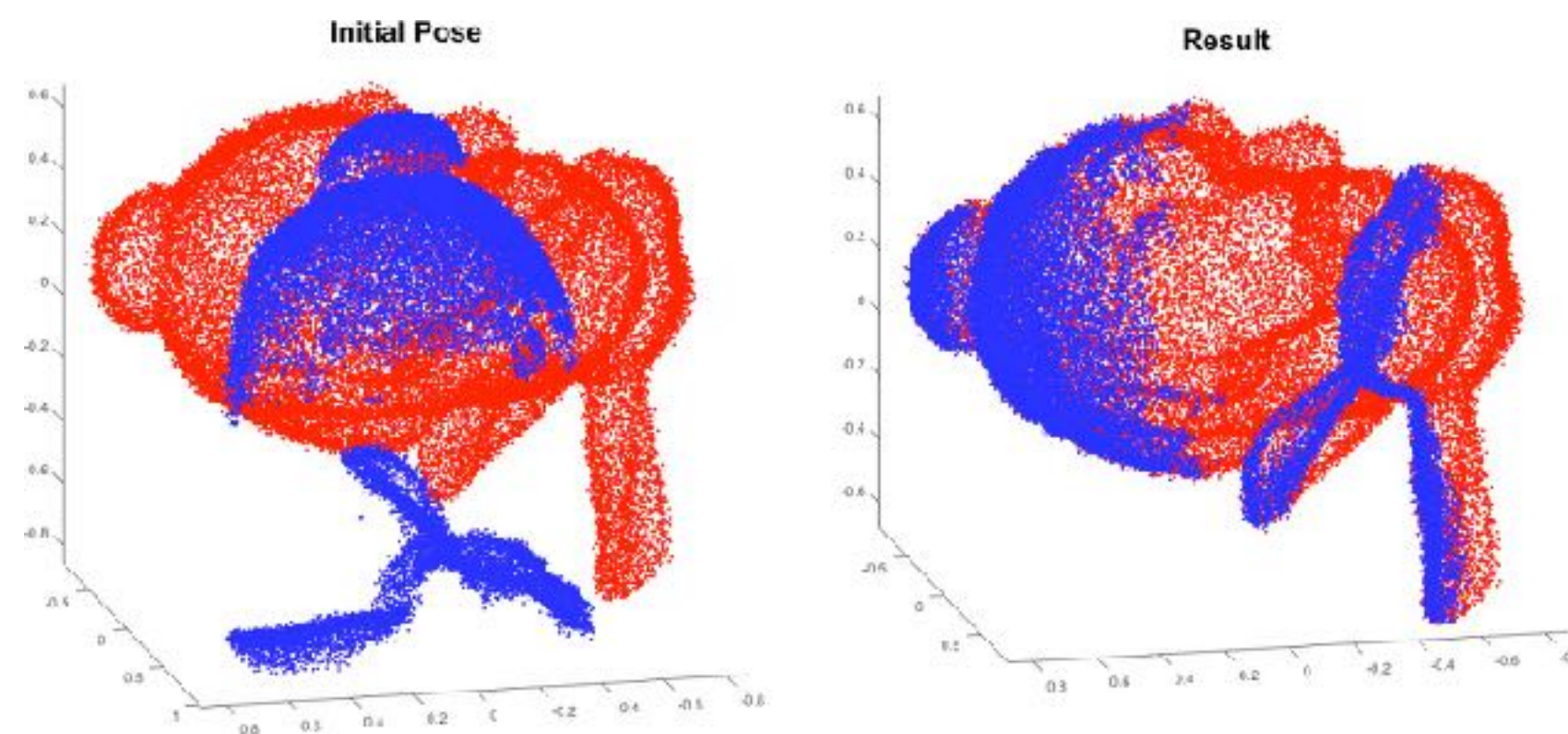
- Open3D
 - http://www.open3d.org/docs/release/tutorial/pipelines/icp_registration.html
 - <http://www.open3d.org/docs/release/tutorial/geometry/kdtree.html>
- PCL: https://pointclouds.org/documentation/group_registration.html

Limitation of ICP

- However, even with improvement
 - Easy to get stuck in local minima
 - Require a good initialization to work

Acquire the Initialization for ICP

- Go-ICP (correspondences based on features)
 - http://www.open3d.org/docs/release/tutorial/pipelines/global_registration.html
 - <https://github.com/yangjiaolong/Go-ICP>



- Teaser (more robust to outliers)
 - <https://github.com/MIT-SPARK/TEASER-plusplus>

Outline

- Introduction
- Rigid transformation estimation
 - Closed-form solution given correspondence
 - Iterative closet point (ICP)
- Learning-based approaches
 - Direct approaches
 - Indirect approaches

Two Categories of Approaches

- Direct: predict (R, t) directly
- Indirect: predict corresponding pairs $\{(p_i, q_i)\}$
 - points in the canonical frame $\{p_i\}$
 - points in the camera frame $\{q_i\}$
 - estimate (R, t) by solving

$$\min_{R, t} \sum_{i=1}^n \|Rp_i + t - q_i\|^2$$

Outline

- Introduction
- Rigid transformation estimation
 - Closed-form solution given correspondence
 - Iterative closet point (ICP)
- Learning-based approaches
 - Direct approaches
 - Indirect approaches

Direct Approaches

- Input: cropped image/point cloud/depth map of a single object
- Output: (R, t)



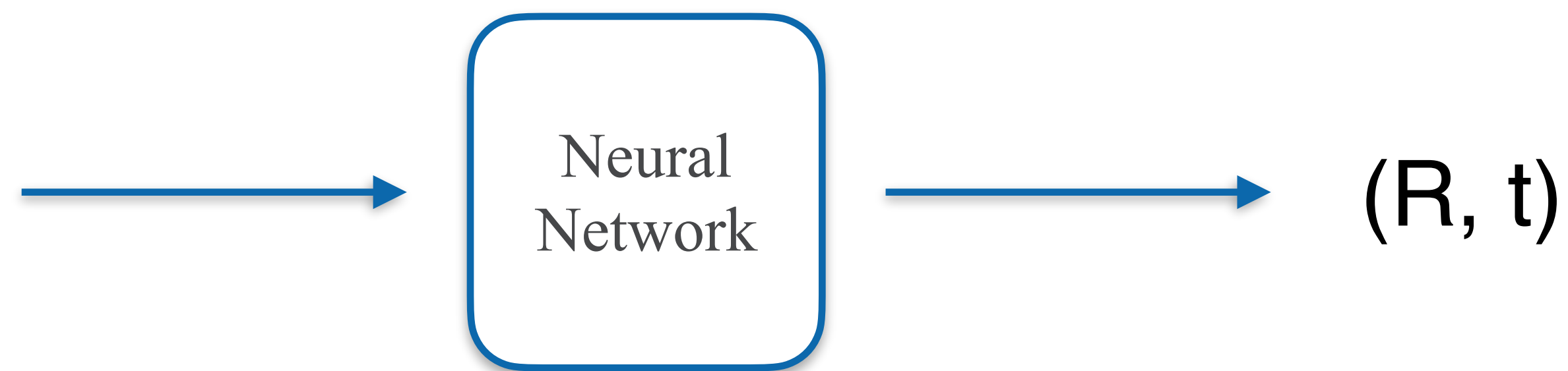
Image



Depth map



Point cloud



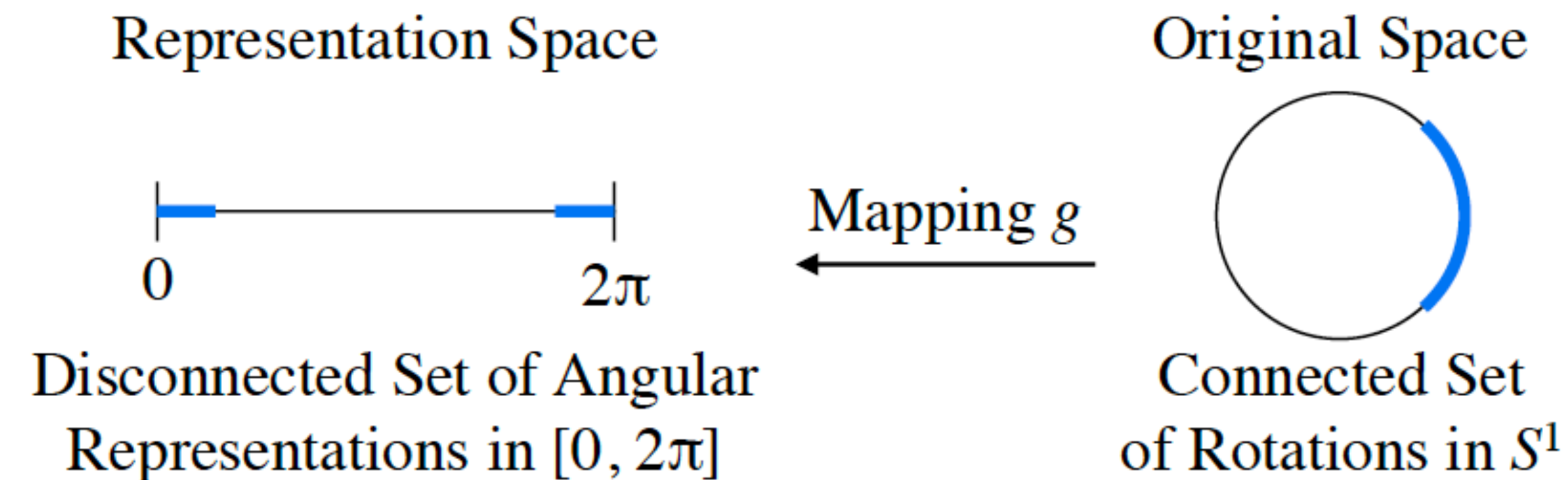
Challenges for Direct Approaches

- The choice of the representation of R
 - Recall rotation matrix, Euler angles, quaternion, axis-angle, ... (more will be introduced in this lecture)
 - Which is more learnable for neural networks?

Continuity: 2D Rotation Example

$$M = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

2D rotation can be parameterized by θ



However, the mapping is discontinuous due to the topology difference

Rotation Representations

Representation	Continuous parameterization	Unique for a rotation
Rotation Matrix	✓	✓
Euler Angle	No	No (gimbal lock)
Angle-axis	No	No (singularity)
Quaternion	No	No (double covering)

Note that neural networks are generally continuous

Fitting a discontinuous function is
not friendly to neural networks

Continuous Representation

- Next, we will introduce two continuous representations: 6D (vector) and 9D (vector)

6D Representation

- 6D representation: $[a_1^T, a_2^T]$, where $a_1, a_2 \in \mathbb{R}^{3 \times 1}$
- Convert $x = [a_1^T, a_2^T]$ to a rotation matrix $R = [b_1, b_2, b_3]$ through the Gram-Schmidt process, where $b_1, b_2, b_3 \in \mathbb{R}^{3 \times 1}$ with unit length

$$b_1 = \frac{a_1}{\|a_1\|}, b_2 \propto \frac{a_2 - \langle a_2, b_1 \rangle b_1}{\|a_2 - \langle a_2, b_1 \rangle b_1\|}, b_3 = b_1 \times b_2$$

- Convert a rotation matrix to 6D: $x = [b_1^T, b_2^T]$

9D Representation

- 9D representation (rotation matrix): $X \in \mathbb{R}^{3 \times 3}$
- Find the closest rotation matrix $R \in SO(3)$
 - Recall the Orthogonal Procrustes Problem
$$\operatorname{argmin}_R \|RP - Q\|_F, \text{ subject to } R^T R = \mathbf{I}$$
 - equivalent to $P = I$ and $Q = X$
- Implementation

Loss for Direct Approaches

- shape-agnostic: distance between (R, t) and (R_{GT}, t_{GT})
- shape-aware: distance between the same shape $X \in \mathbb{R}^{3 \times n}$ transformed by (R, t) and (R_{GT}, t_{GT}) respectively

Shape-agnostic Loss

- Rotation
 - Geodesic distance (angle difference, relative rotation error) on $SO(3)$: $\arccos[\frac{1}{2}(\text{tr}(R_{GT}R^T) - 1)]$
 - Mean square error over $\mathbb{R}^{3 \times 3}$: $\|R - R_{GT}\|_F^2$
 - L_p distance on representation x : $\|x - x_{GT}\|_p$
- Translation
 - L_p distance: $\|t - t_{GT}\|_p$

Challenges: Symmetry

- Symmetry introduces ambiguities of GT labels
- Require specially designed losses to tackle it



Examples of symmetric objects in YCB dataset

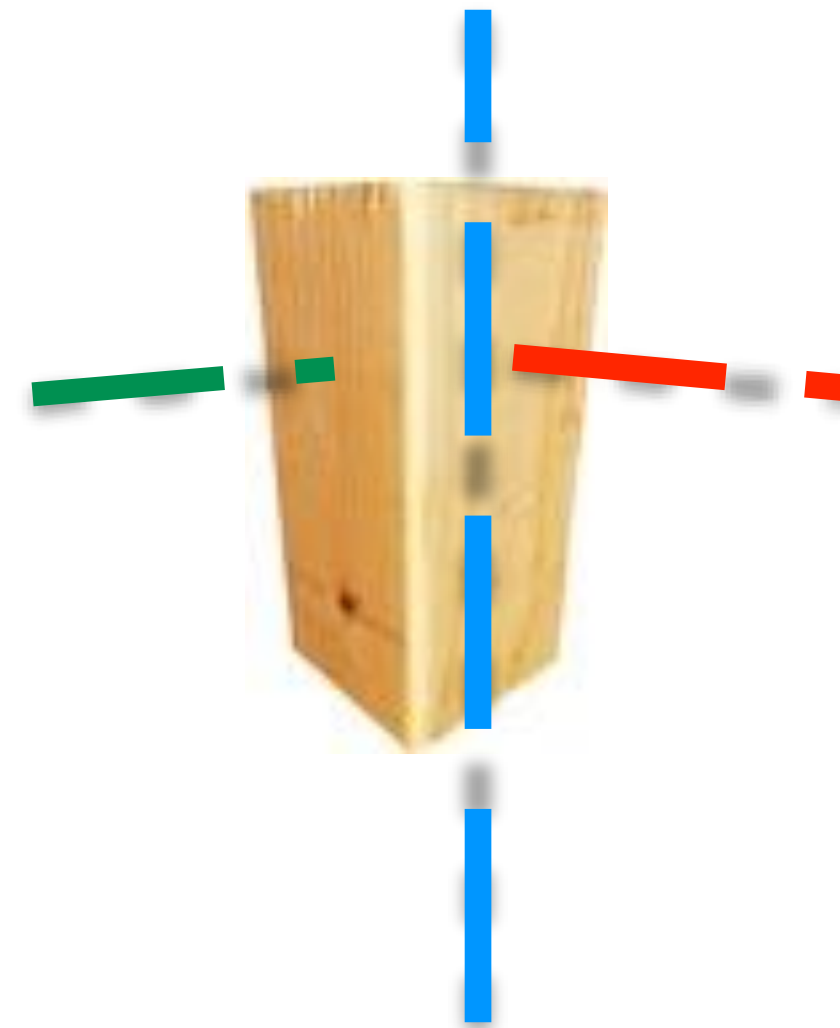
Rotational Symmetry



1 symmetry axis
symmetry order 2



1 symmetry axis
symmetry order infinite



3 symmetry axis
symmetry order (4, 2, 2)

Shape-agnostic Loss for Symmetric Objects

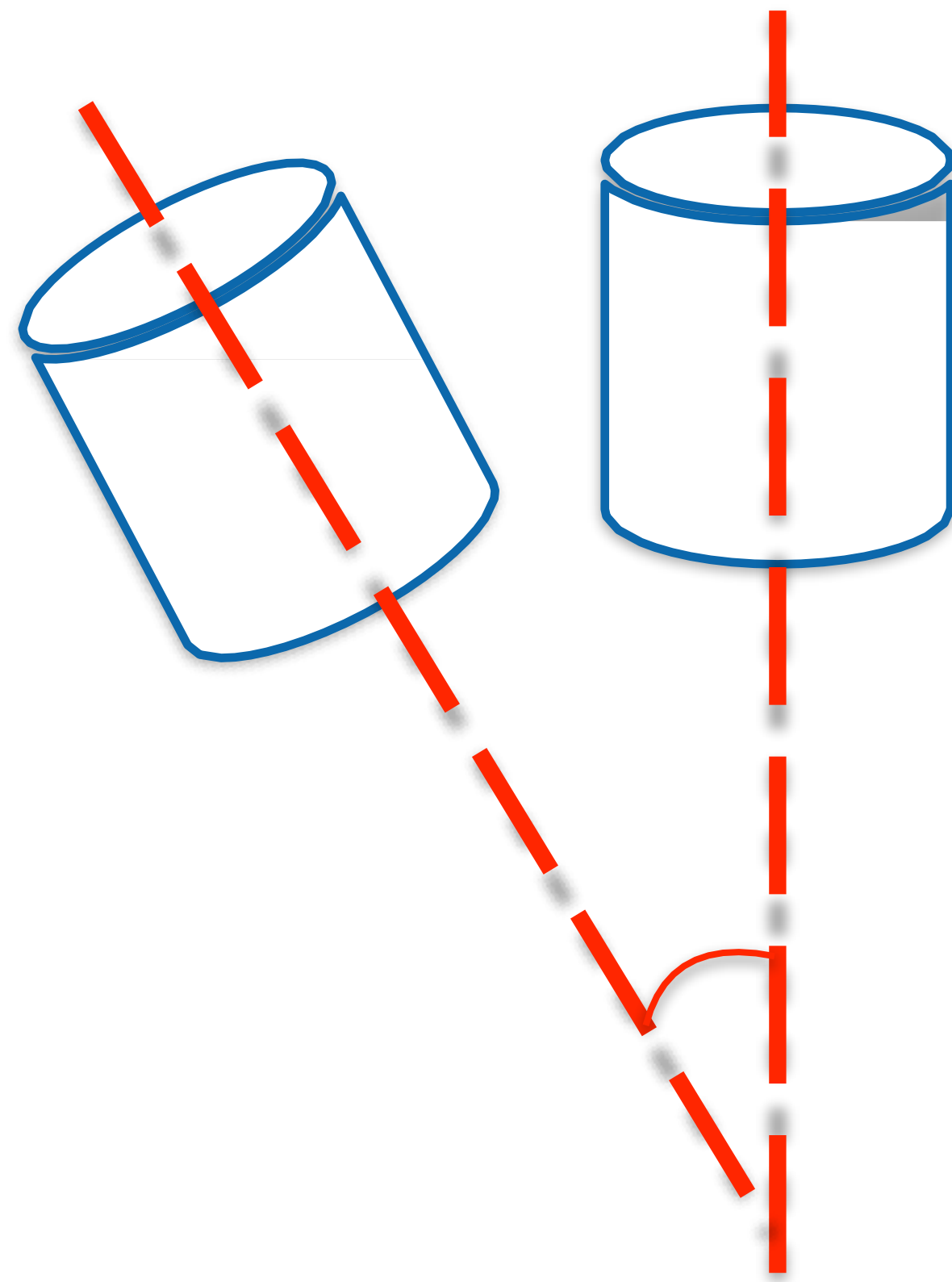
- Symmetry group: Multiple symmetry-equivalent GT rotations $\mathcal{R} = \{R_{GT}^1, R_{GT}^2, \dots, R_{GT}^n\}$ (n is the order of symmetry)

- *Min of N* loss (for finite order of symmetry)

$$\min_{R_{GT}^i \in \mathcal{R}} L(R_{GT}^i, R)$$

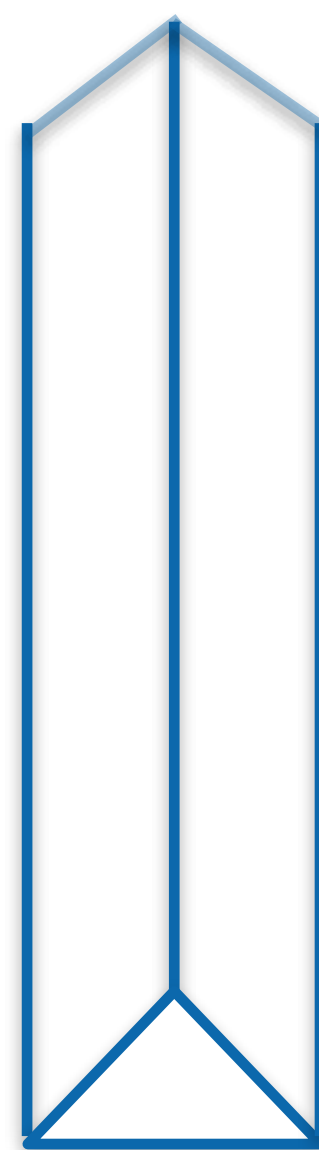
Q: How to deal with infinite order?

Shape-agnostic Loss for Symmetric Objects (Infinite Order)

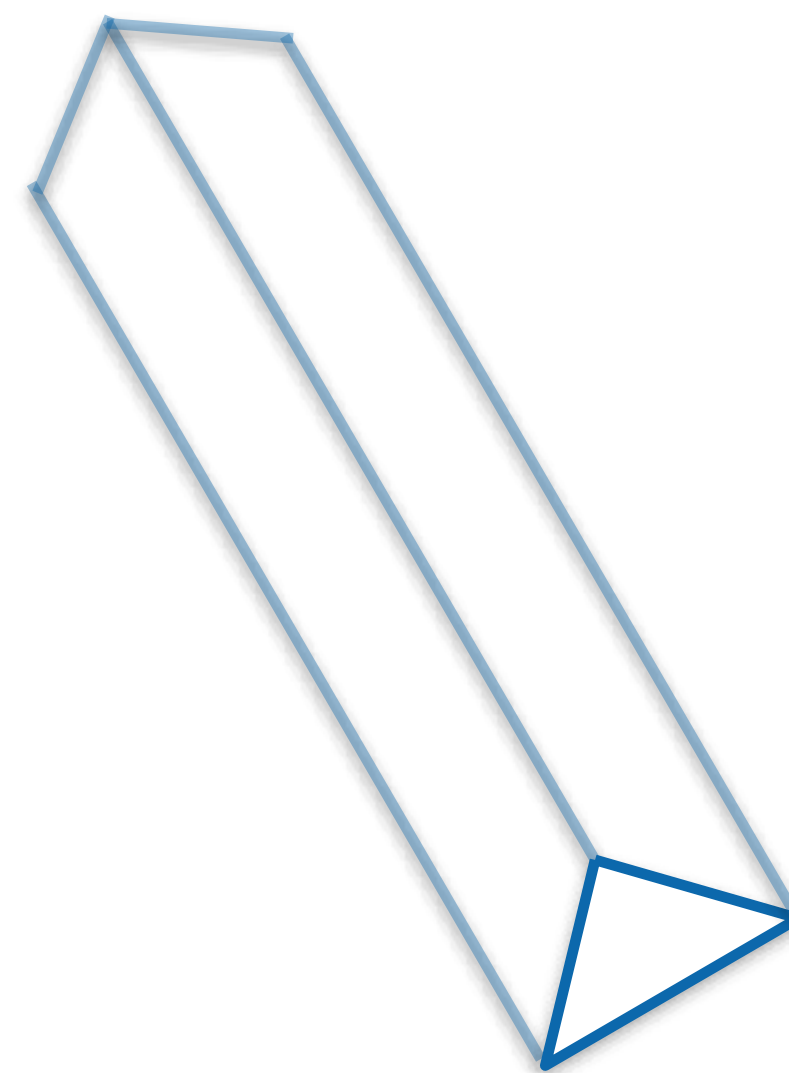


Use the angle between
two symmetry axes

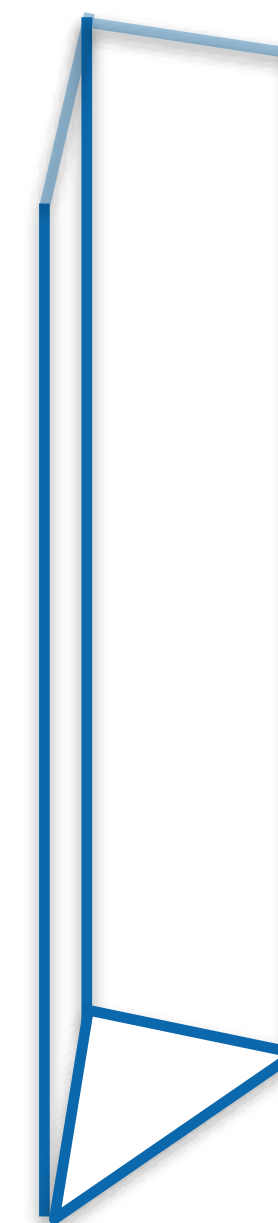
Motivation of Shape-aware Loss



target pose



predicted pose1



predicted pose2

Similar angle difference, but very different perception error

Shape-aware Loss

- Given a shape $X \in \mathbb{R}^{3 \times n}$, the loss for rotation can be the distance between RX and $R_{GT}X$
- e.g., $L = \|RX - R_{GT}X\|_F^2$ (per-point Mean Square Estimation (MSE))

Shape-aware Loss for Symmetry Objects

- With $\|RX - R_{GT}X\|_F^2$ as the distance metric, we can also apply the *Min of N* loss

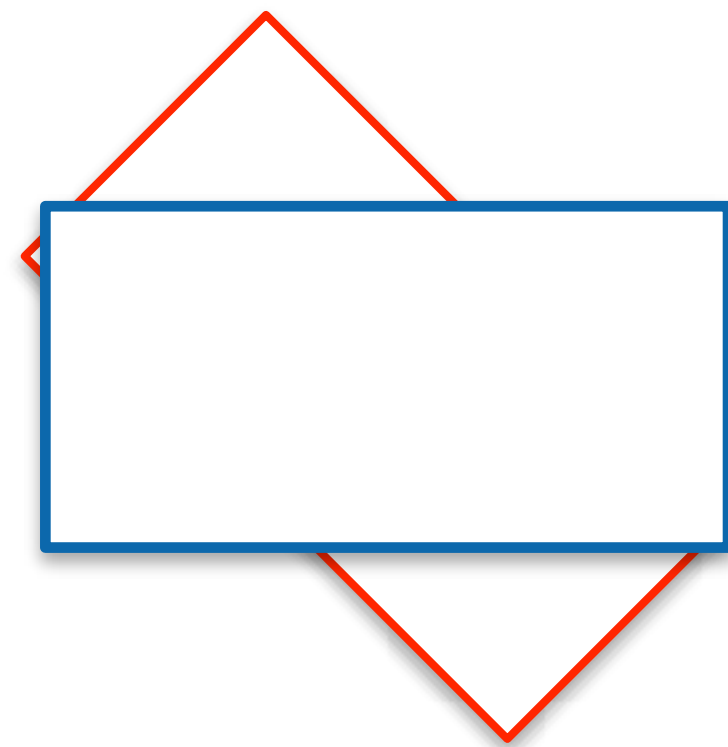
$$L = \min_{R_{GT}^i \in \mathcal{R}} \|RX - R_{GT}^i X\|_F^2$$

Shape-aware Loss for Symmetry Objects

- Recall the distance metrics for point clouds
 - Chamfer distance
 - earth mover distance
- Those distance metrics are compatible with symmetry (even infinite order)
- But requires a shape and might get stuck in local minima

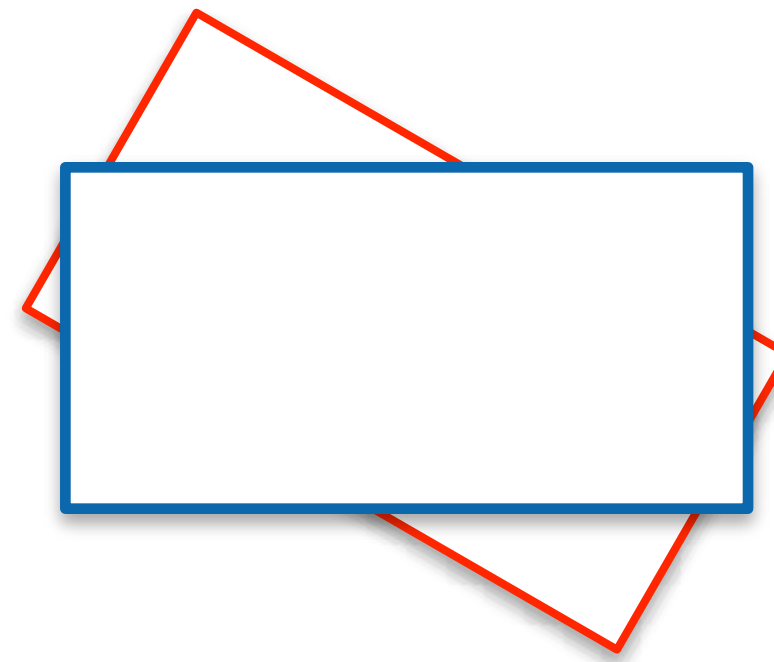
Example of Local Minima for Chamfer Distance

Rotate the red left a bit?



CD=2.02

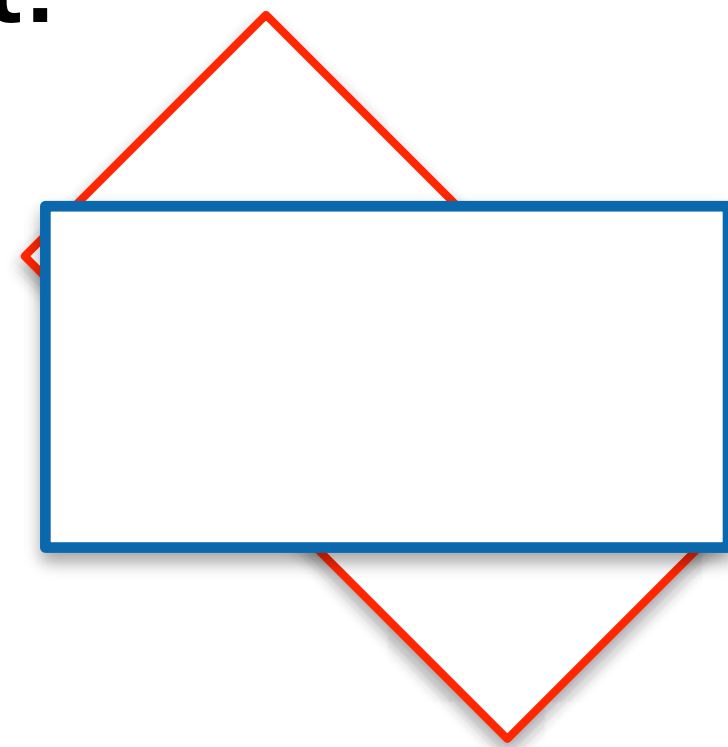
Example of Local Minima for Chamfer Distance



CD=2.05

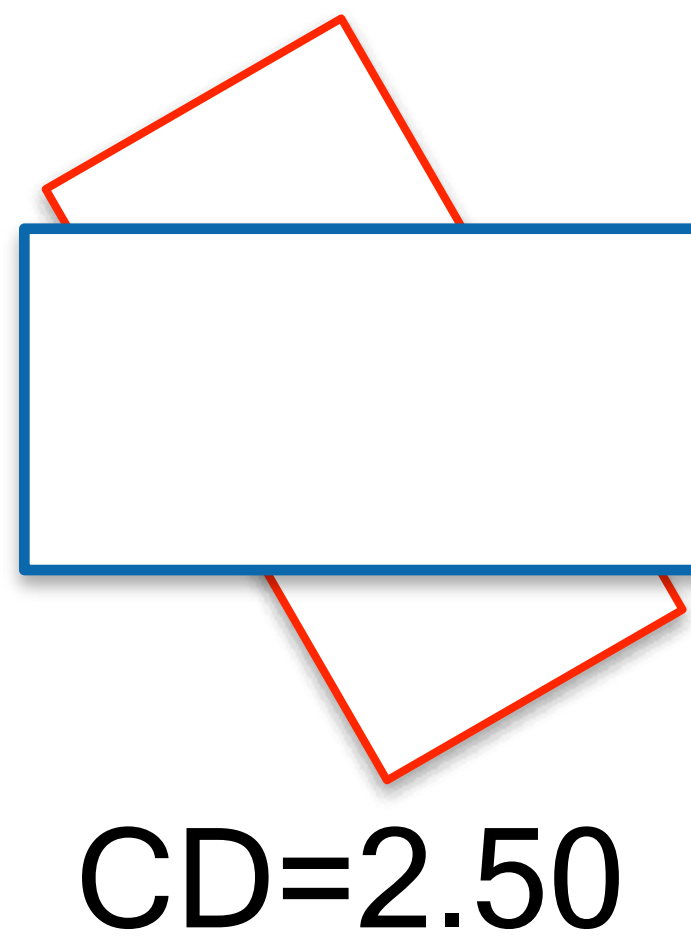
Example of Local Minima for Chamfer Distance

Reset, and try rotate right:

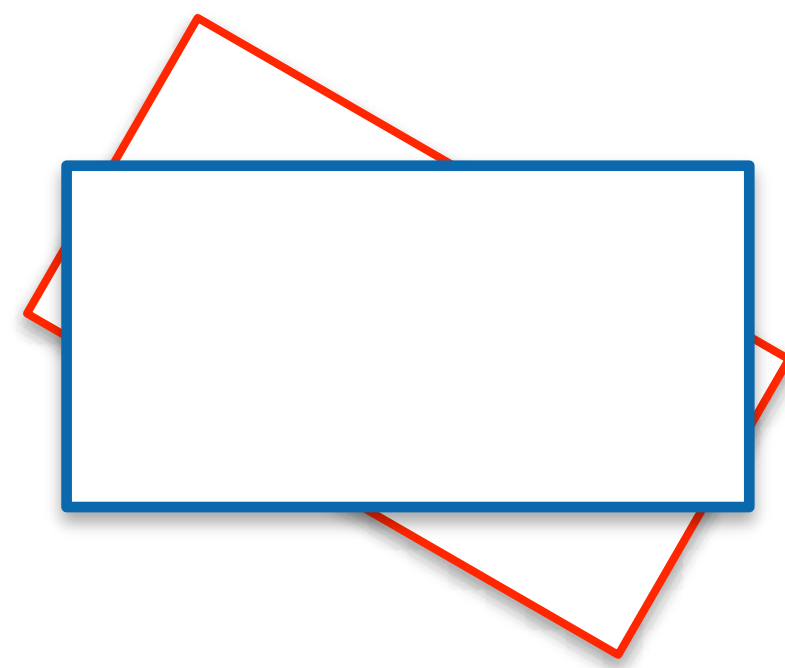


CD=2.02

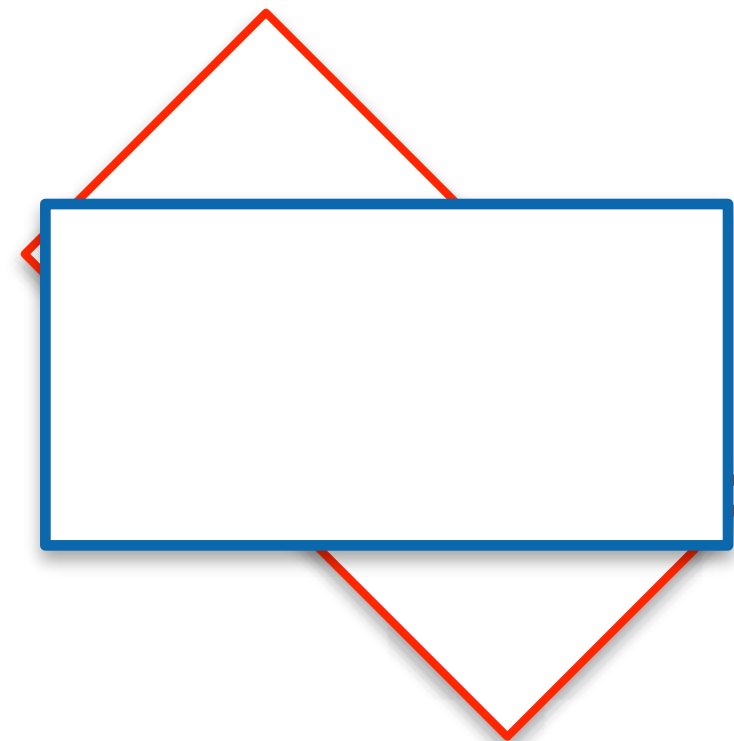
Example of Local Minima for Chamfer Distance



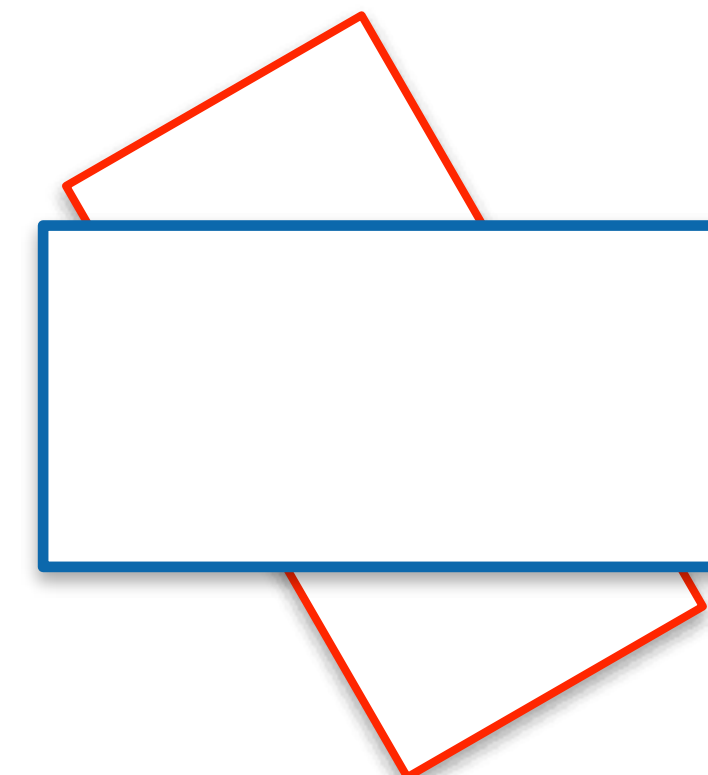
Example of Local Minima for Chamfer Distance



CD=2.05



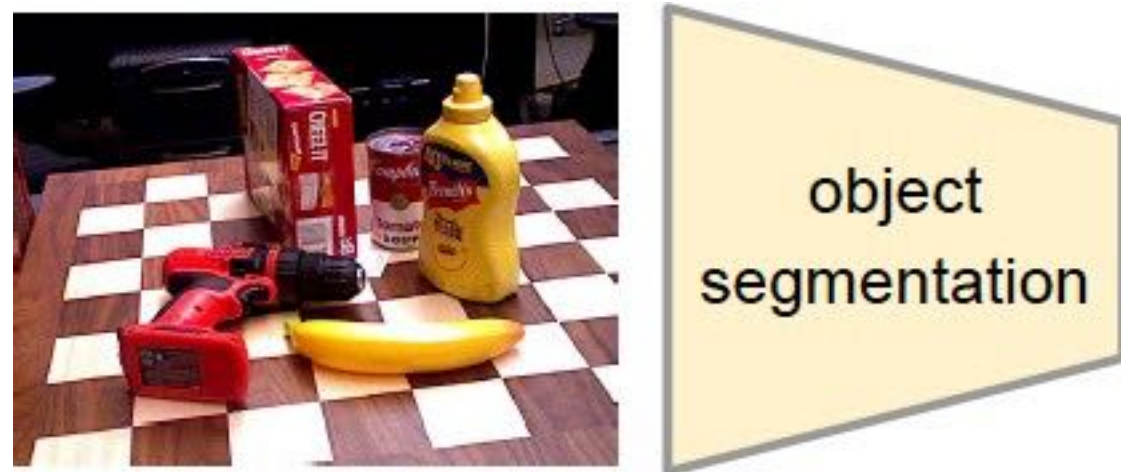
CD=2.02



CD=2.50

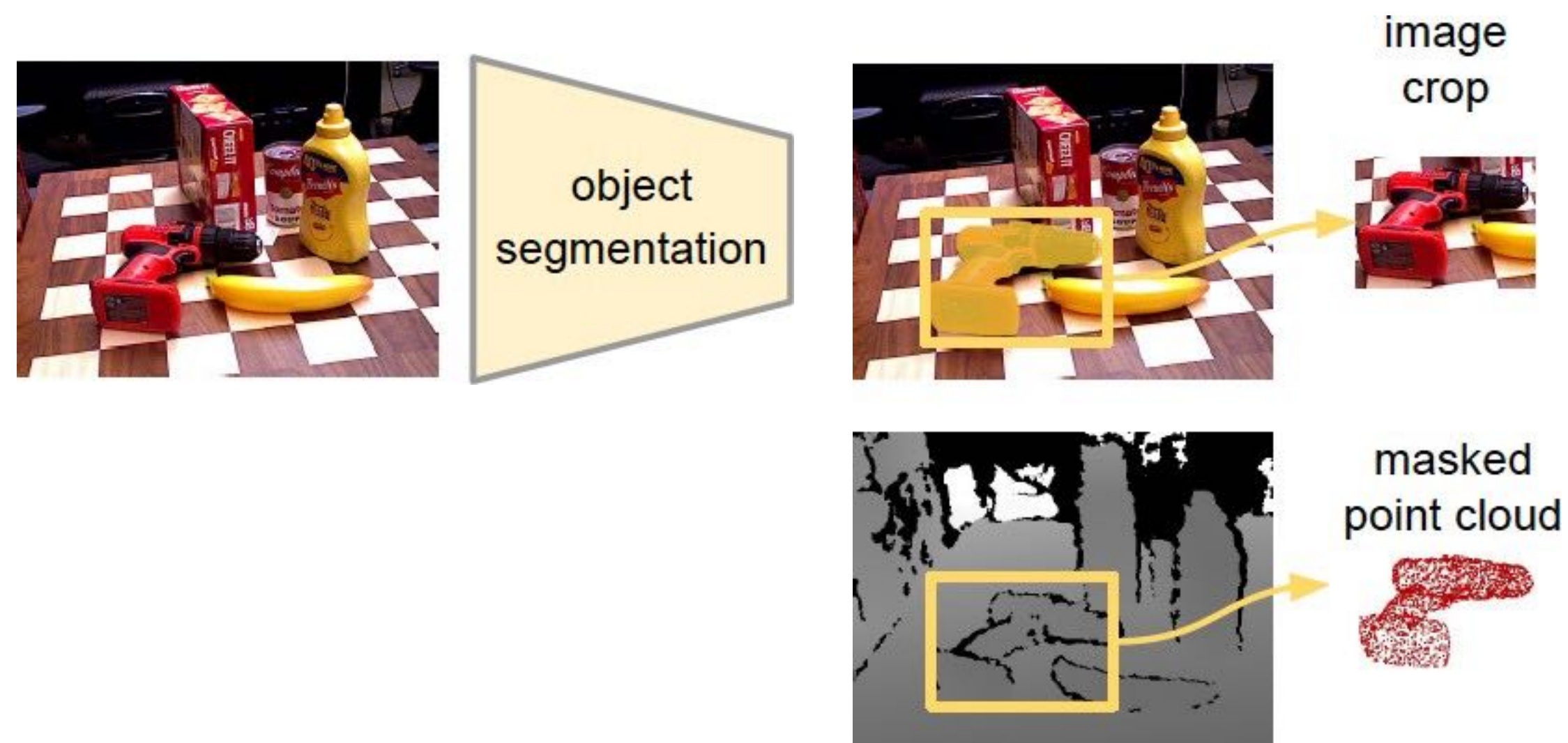
Note: No local minima for Min of N

Example: DenseFusion



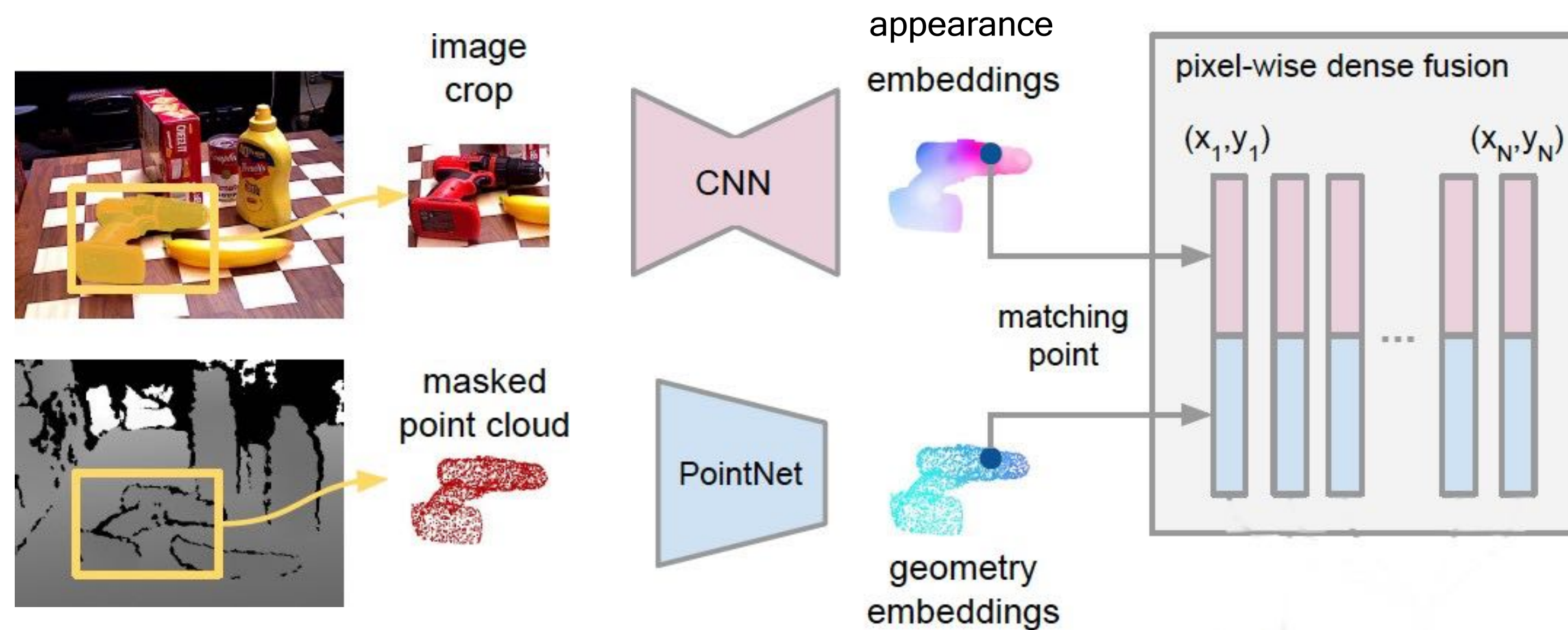
Segment objects

Example: DenseFusion



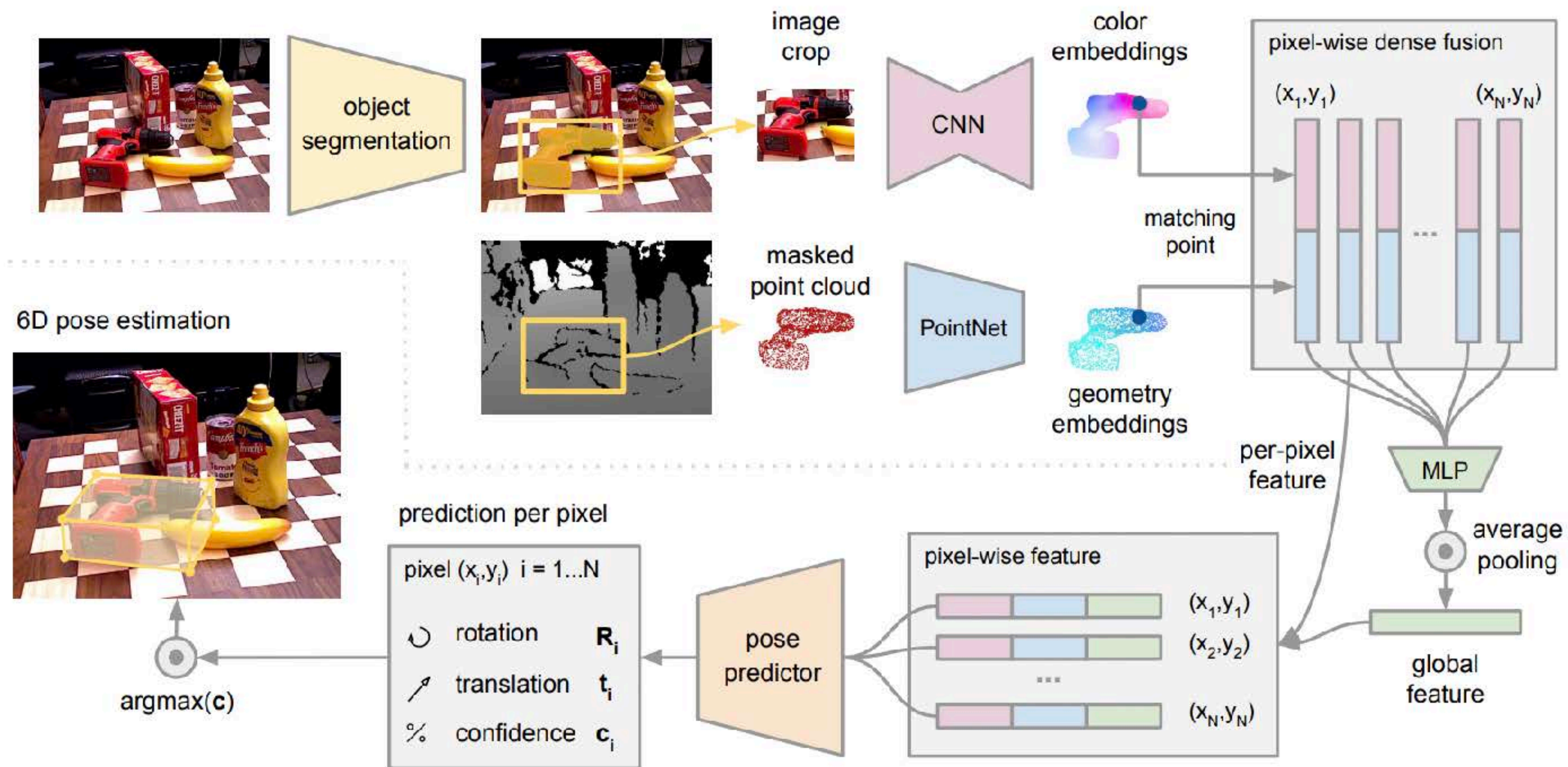
Get the image crop and the point cloud for an object

Example: DenseFusion



Encode appearance (image) with CNN
and geometry (point cloud) with PointNet

Example: DenseFusion



Predict poses from features

Example: DenseFusion

6D pose estimation



non-symmetric objects:
per-point MSE

symmetric objects:
chamfer distance

Min-of-N as alternative?

Compute shape-aware loss

Outline

- Introduction
- Rigid transformation estimation
 - Closed-form solution given correspondence
 - Iterative closet point (ICP)
- Learning-based approaches
 - Direct approaches
 - Indirect approaches

Indirect Approaches

- Input: cropped image/point cloud/depth map of a single object
- Output: corresponding pairs $\{(p_i, q_i)\}$
 - points in canonical frame $\{p_i\}$
 - points in camera frame $\{q_i\}$
 - estimate (R, t) by solving

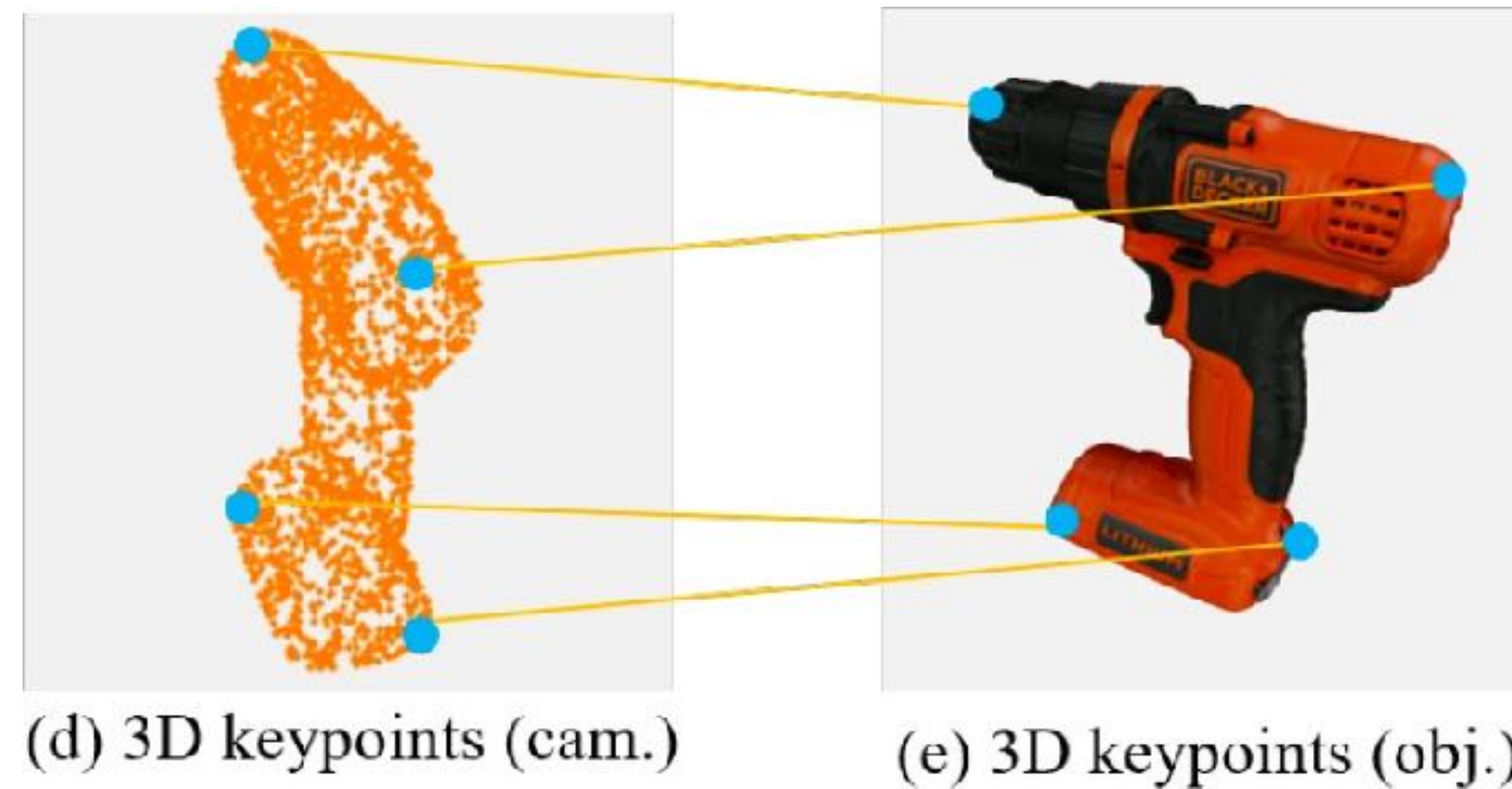
$$\min_{R, t} \sum_{i=1}^n \|Rp_i + t - q_i\|^2$$

Two Categories of Indirect Approaches

- If points in the **canonical frame** are known, predict their corresponding locations in the **camera frame**
- If points in the **camera frame** are known, predict their corresponding locations in the **canonical frame**

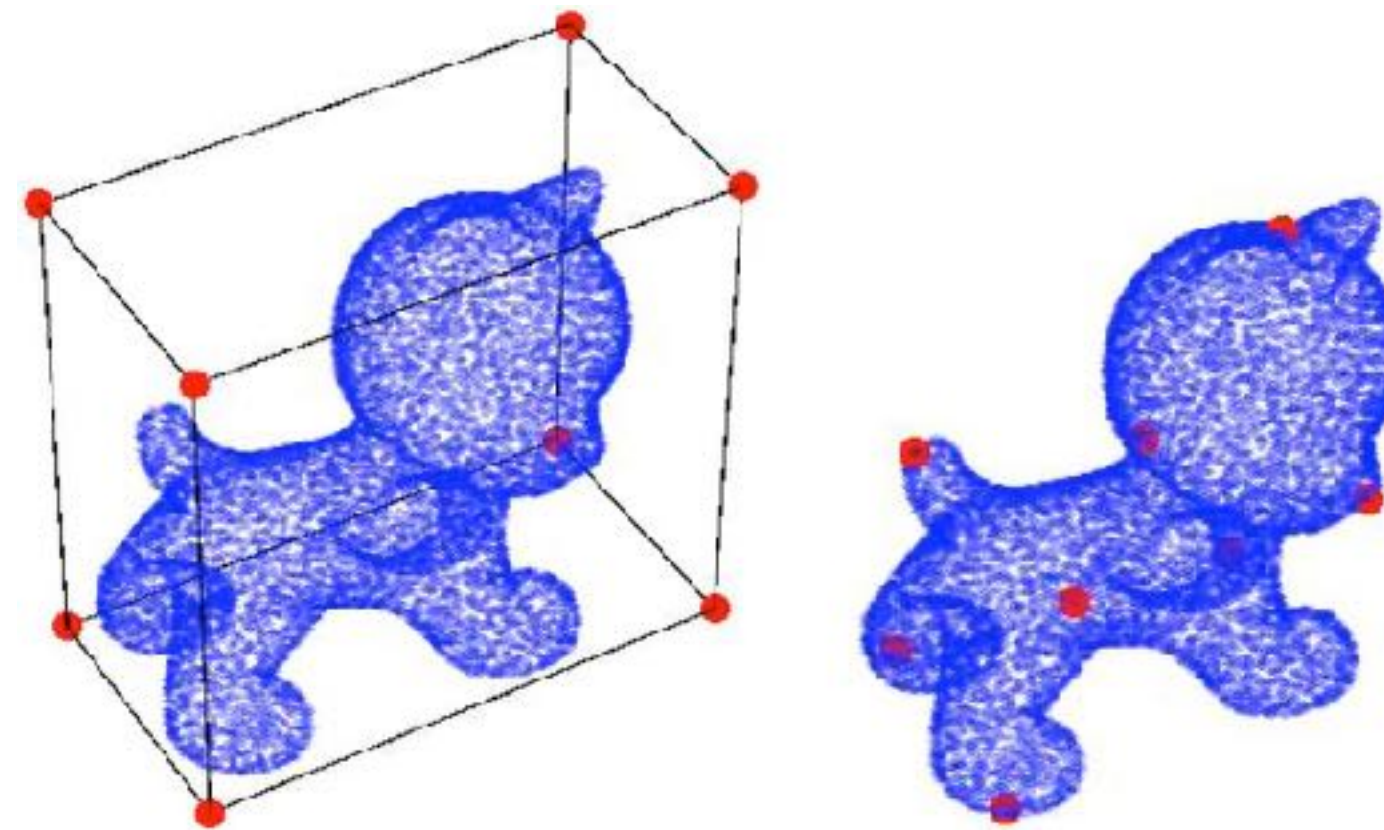
Given Points in the Canonical Frame, Predict Corresponding Location in the Camera Frame

- Recall: Three correspondences are enough
- Which points in the canonical frame should be given?
 - Choice by PVN3D: keypoints in the canonical frame

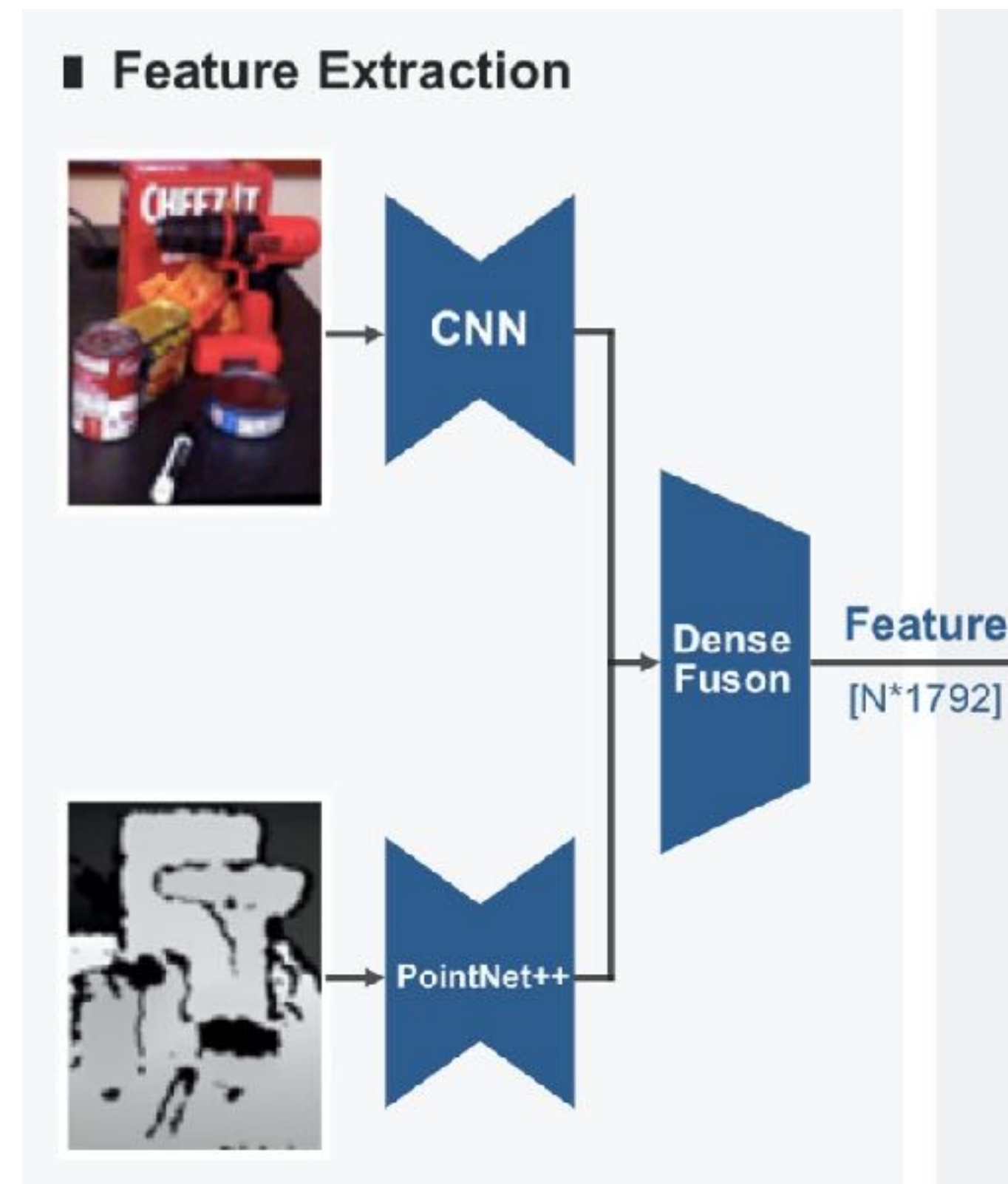


Keypoint Selection

- Option1: bounding box vertices
- Option 2: farthest point sampling (FPS) over CAD object model

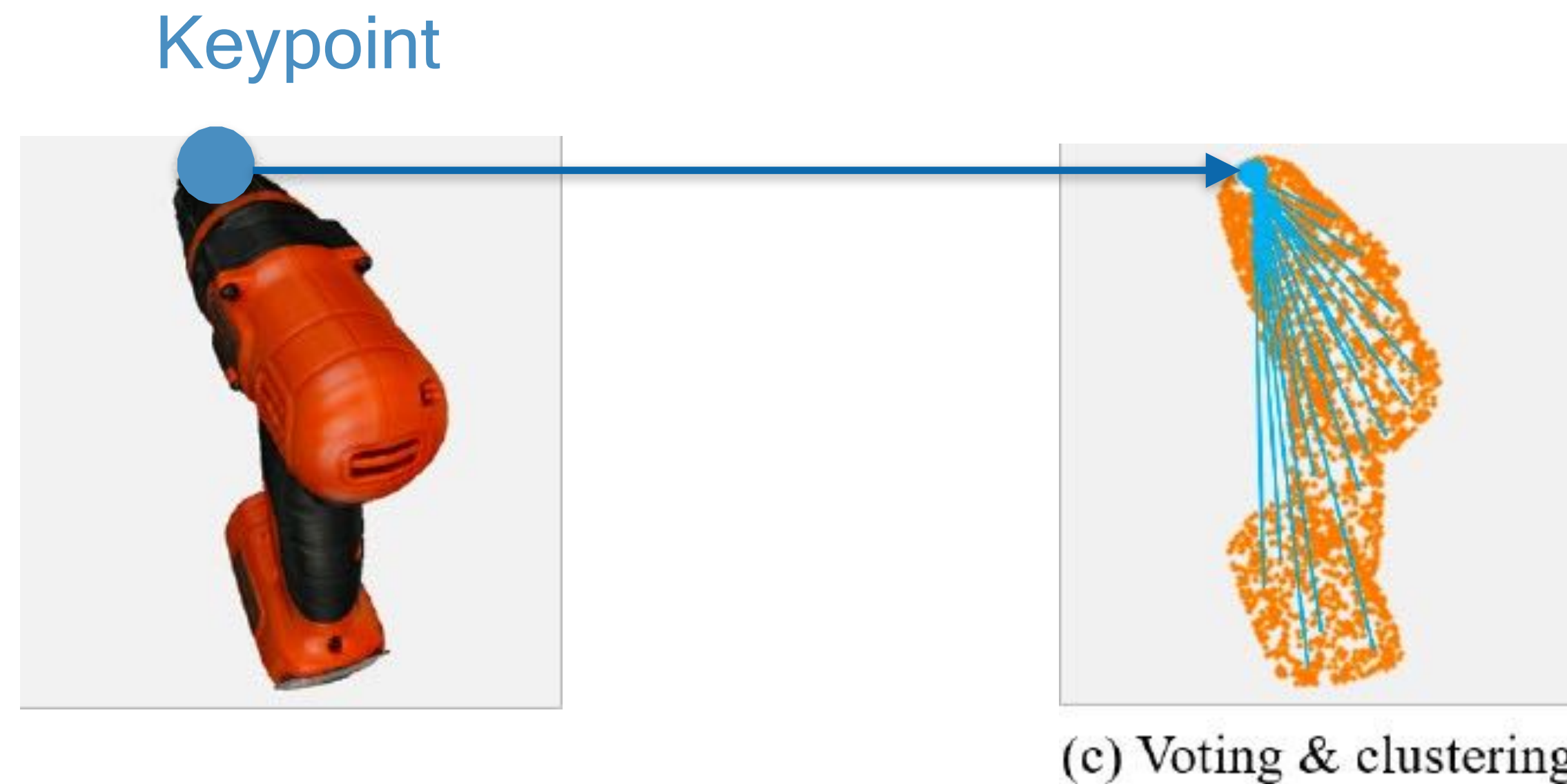


Example: PVN3D



Get point-wise features
by fusing color and geometry features

Example: PVN3D

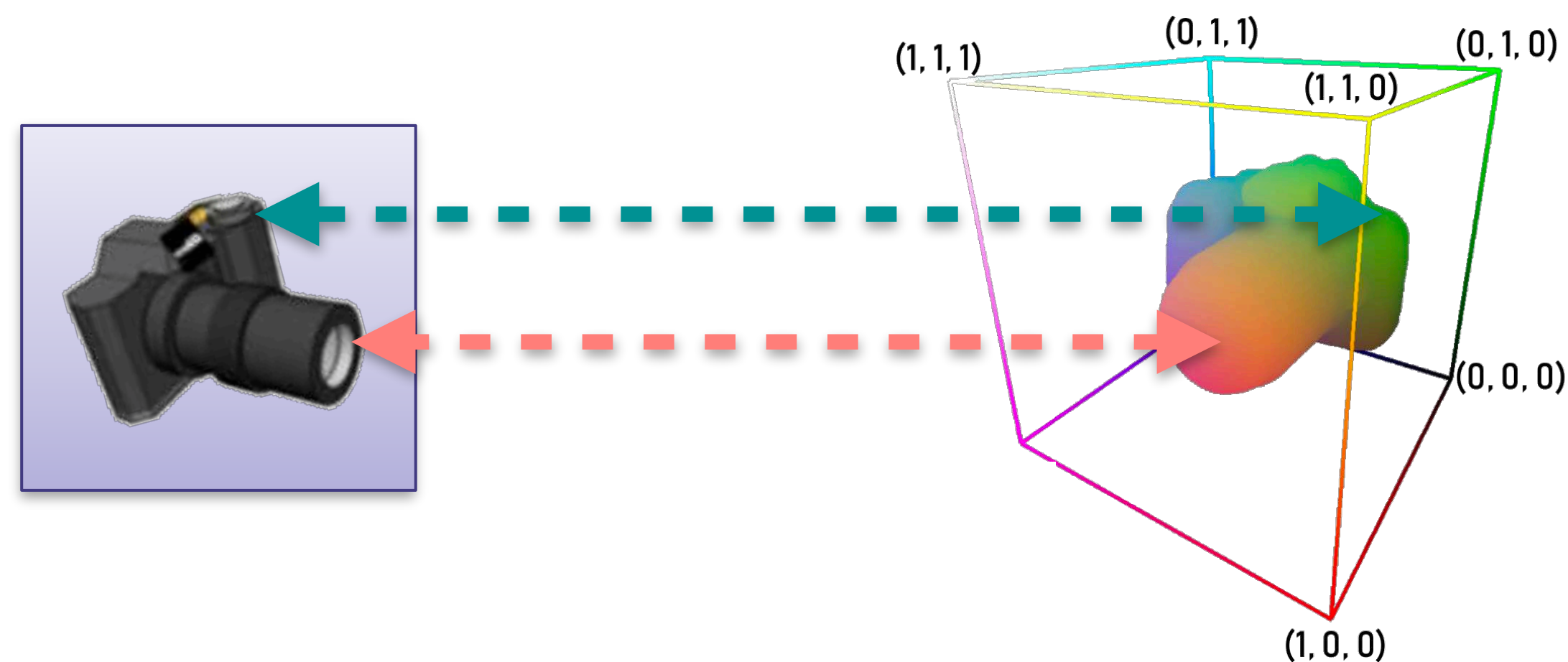


For each keypoint:

- **Voting:** for each point in the camera frame, predict its offset to the keypoint (in the camera frame)
- **Clustering:** find one location according to all the candidates

Given Points in the Camera Frame, Predict Corresponding Location in the Canonical Frame

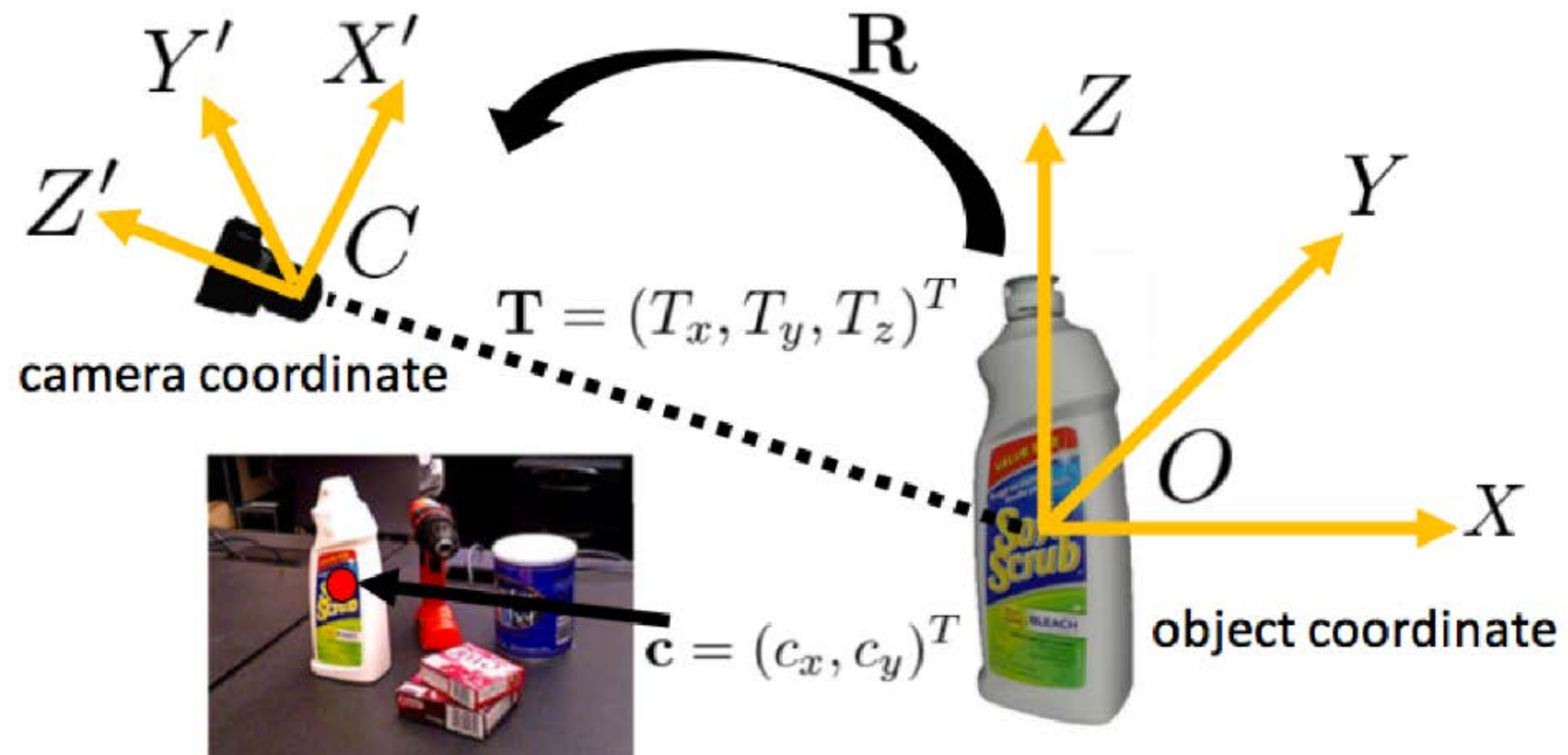
- Which points in the camera frame should be given?
 - Choice by NOCS: every point in the camera frame



3D point in the camera frame
(2D visible pixel with depth)

3D point in the (normalized)
canonical frame

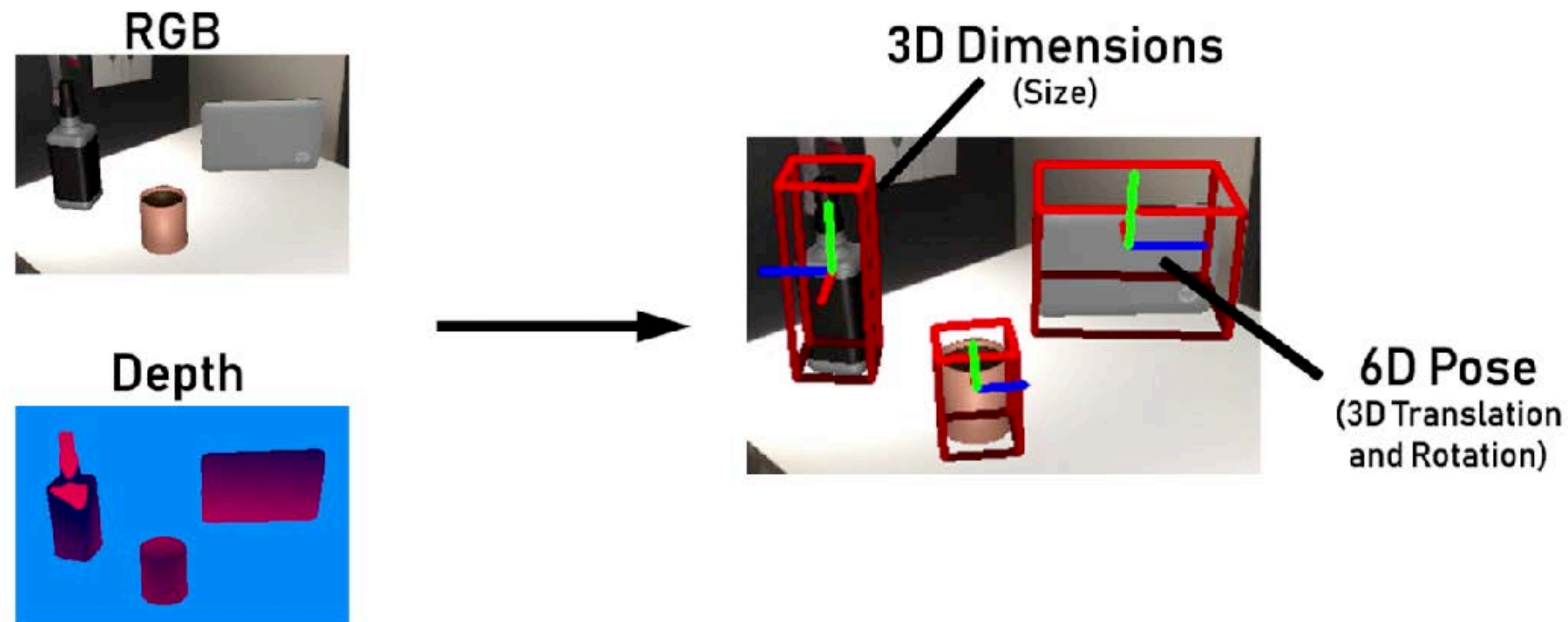
Instance-Level 6D Pose Estimation



- Instance-level: a small set of known instances
- Pose is defined for each instance according to their CAD model.
- Input: RGB/RGBD
- Output: 6D pose(s) of an object or all known objects

Category-Level 6D Pose Estimation

Detecting and estimating **6D pose** and **3D size** of **previously unseen** objects from certain categories from RGBD images.



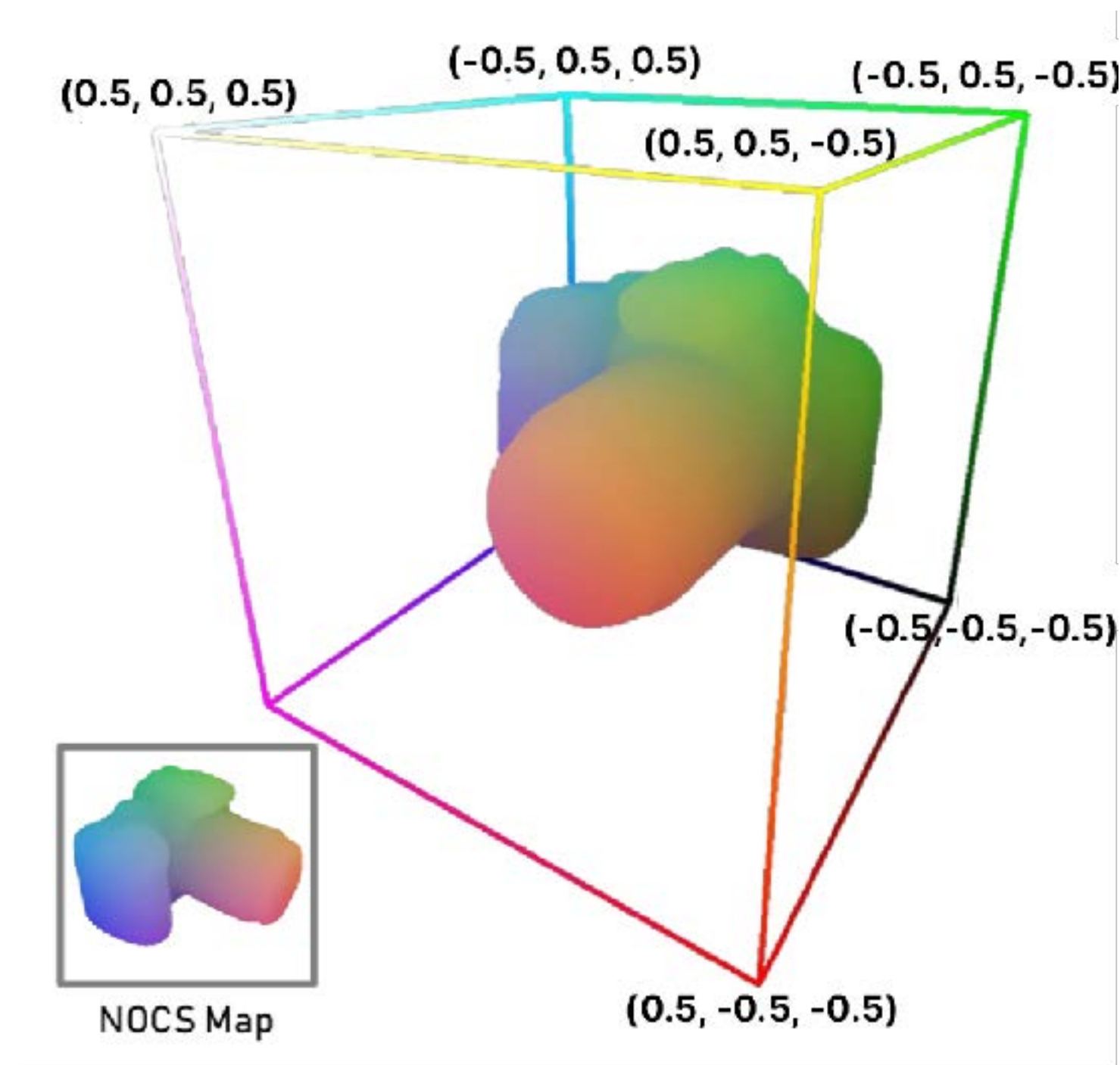
without knowing CAD models

Challenges

- Intra-category shape variations



Representation: Normalized Object Coordinate Space



RGB colors represent XYZ coordinates of shape.

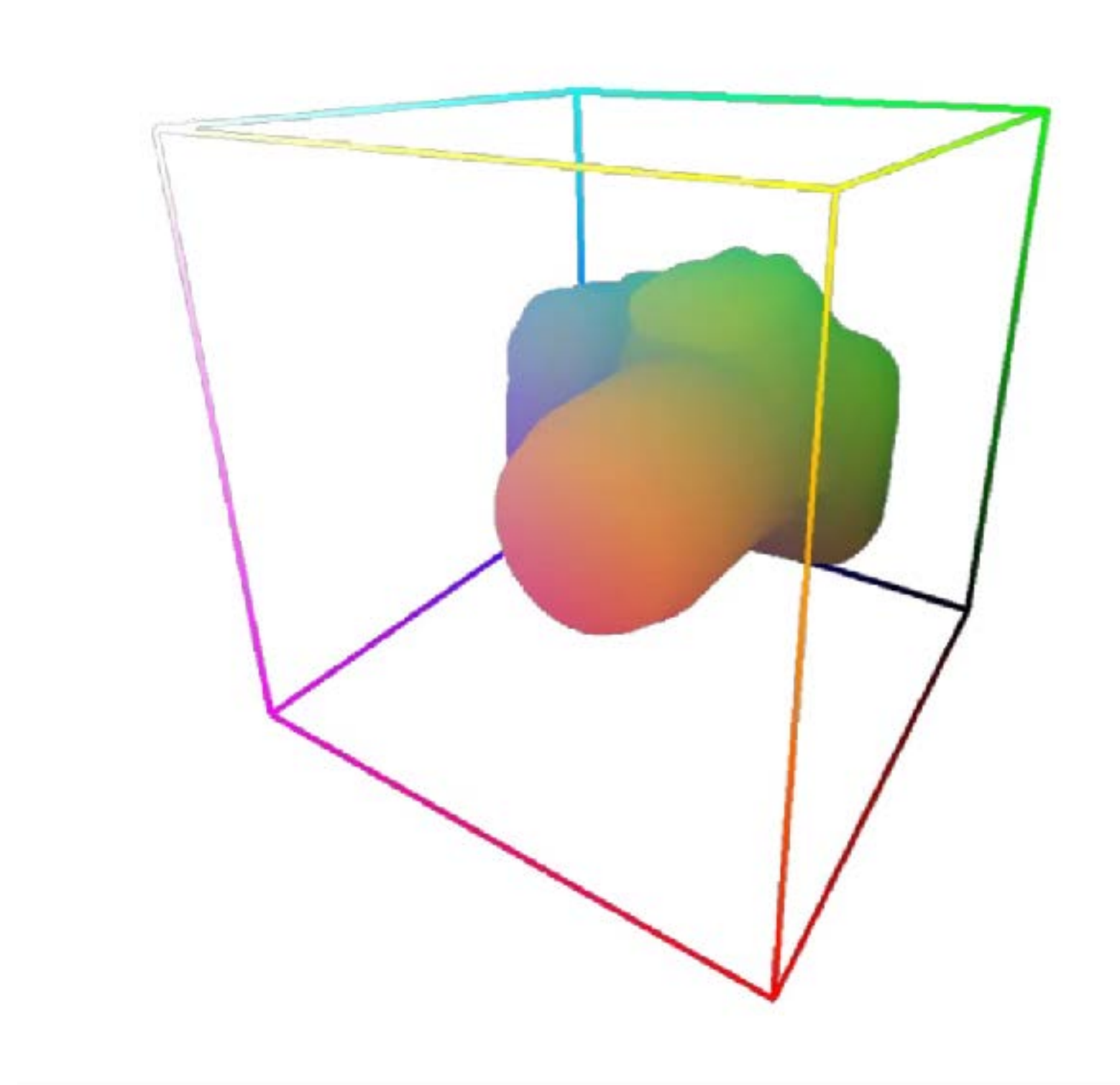
Representation: Normalized Object Coordinate Space

Step 1 (rotation normalization):
align object orientations



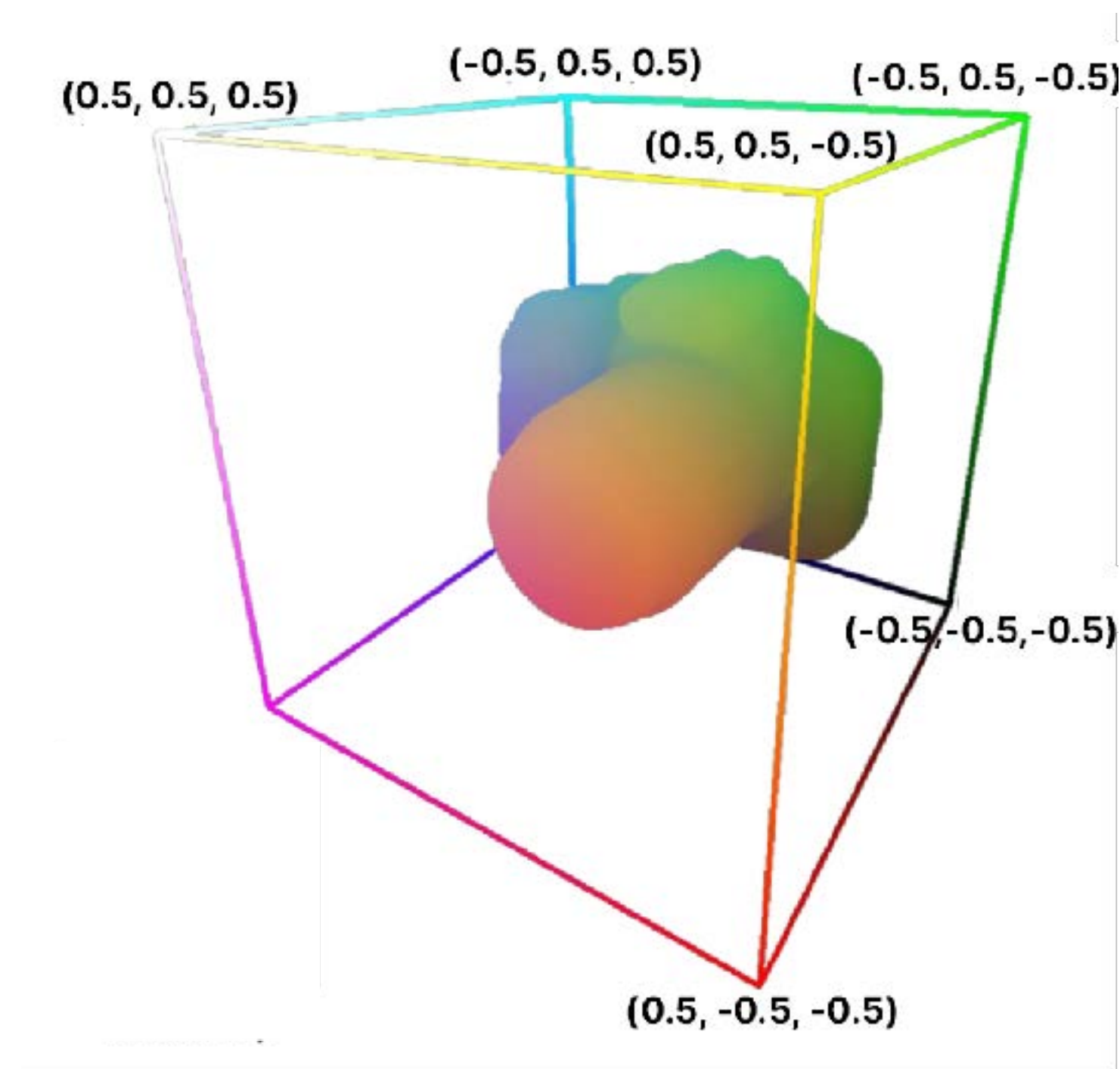
Representation: Normalized Object Coordinate Space

Step 2 (translation normalization):
zero-center the objects



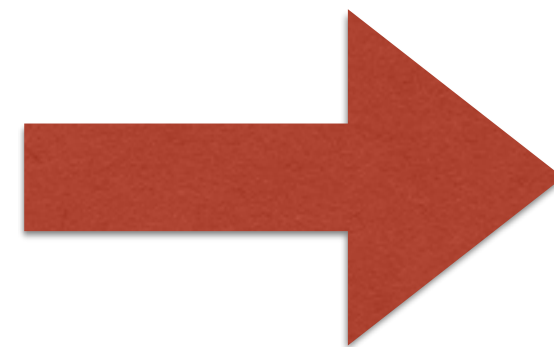
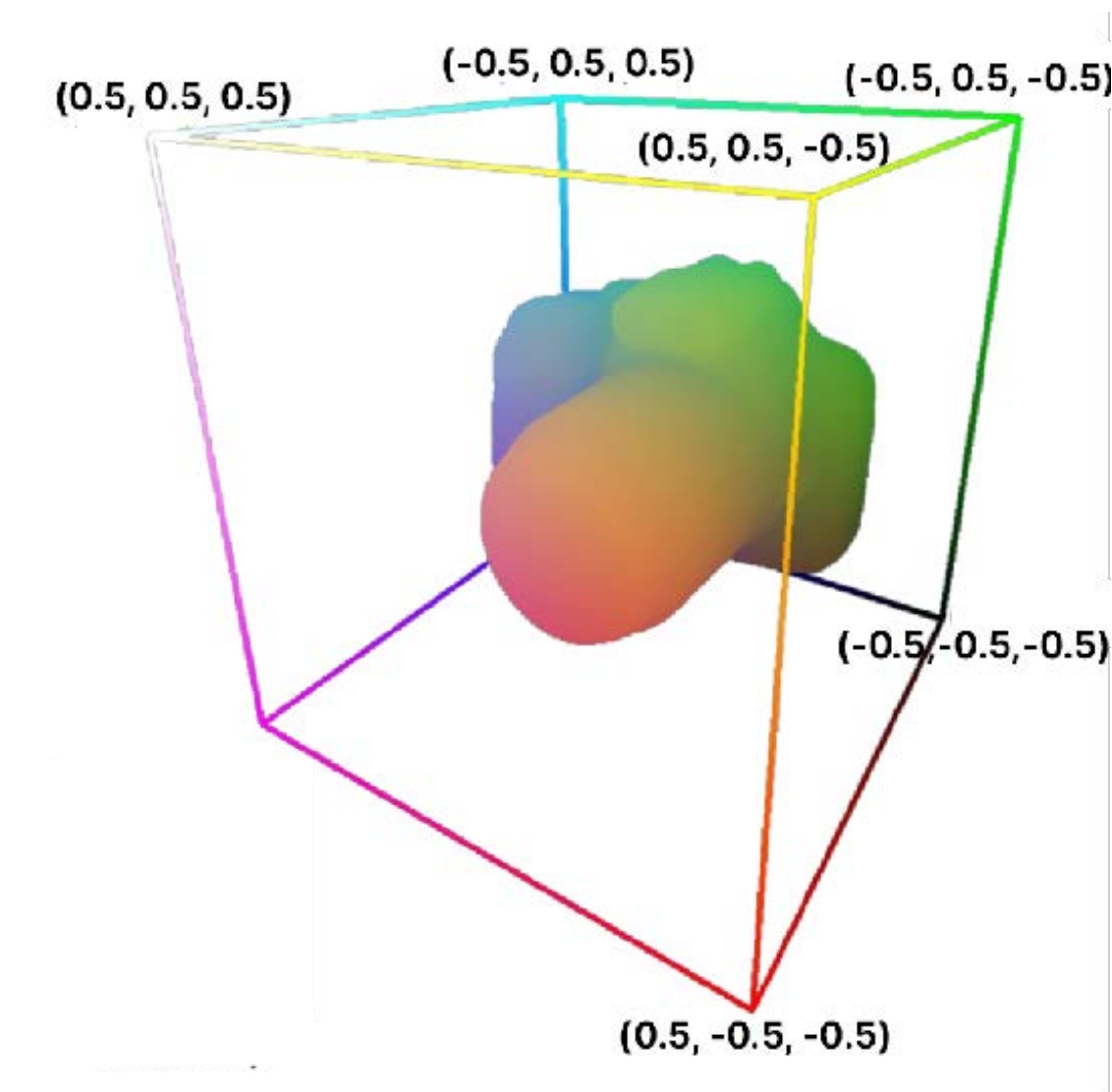
Representation: Normalized Object Coordinate Space

Step 3 (scale normalization):
uniformly normalize the scales

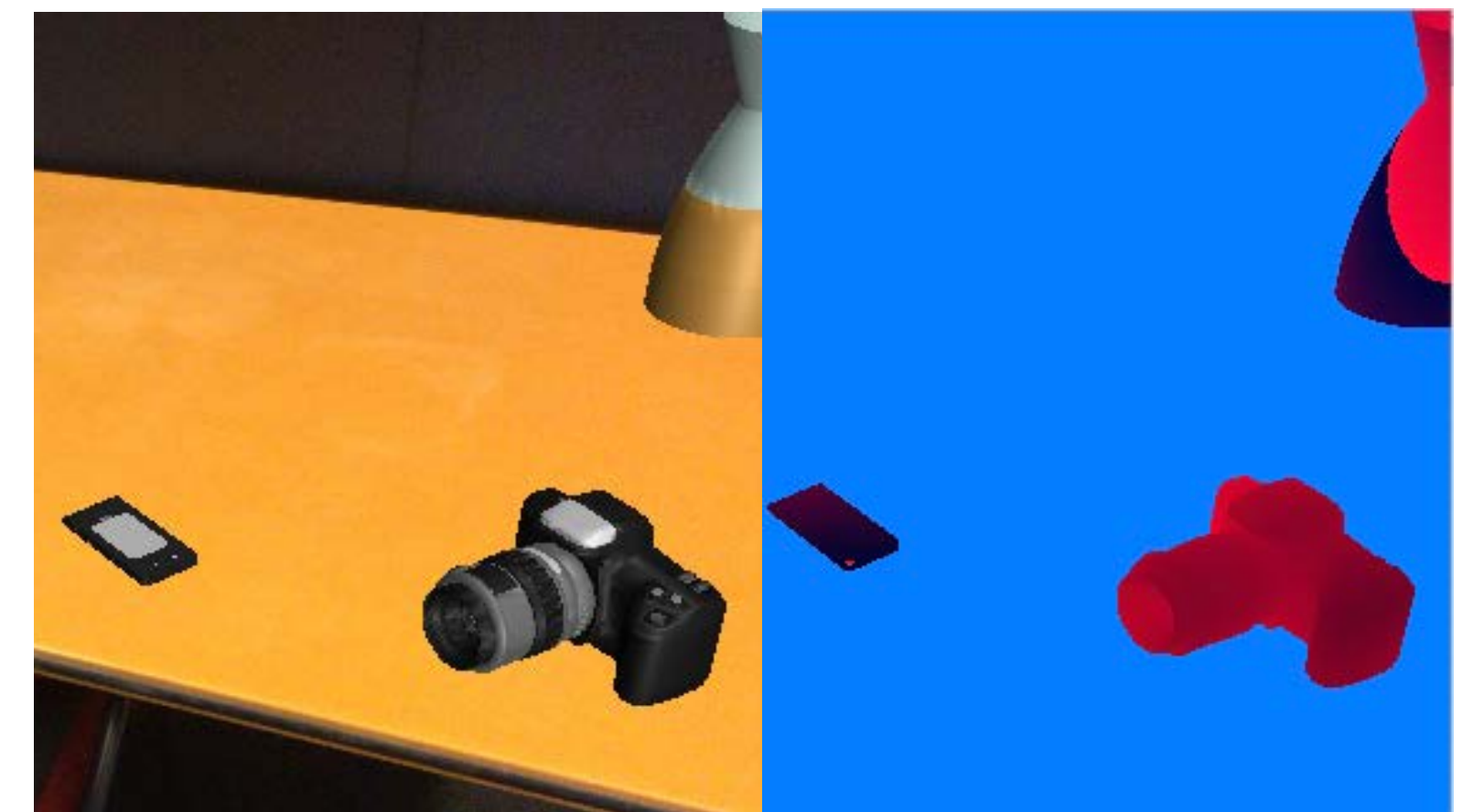


Definition of Category-Level 6D Pose

NOCS = Reference frame



RGBD



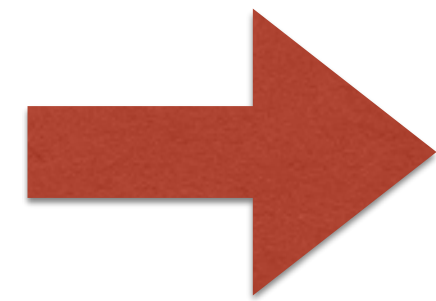
Category-level pose = transformation from NOCS to camera space

From Image to NOCS map to Pose



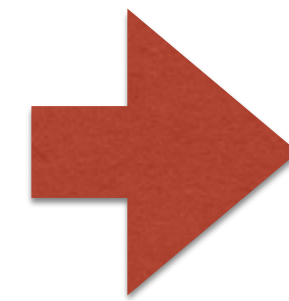
RGB Image

Per-pixel
predict



NOCS map

NOCS to point

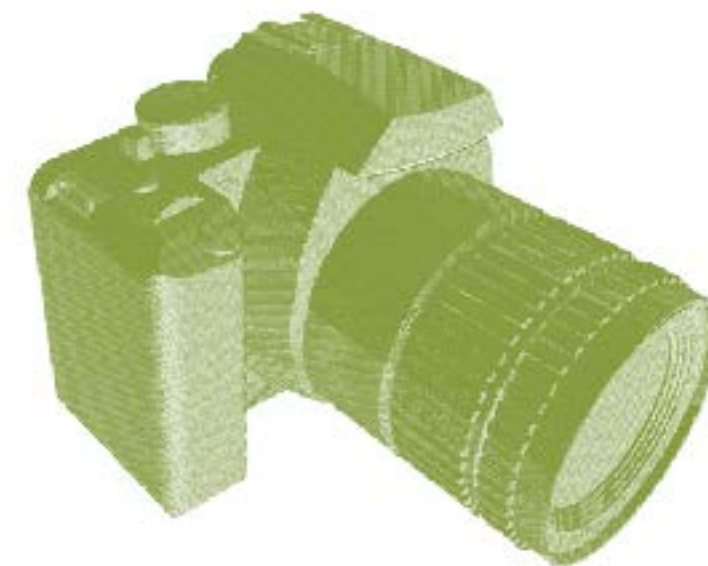
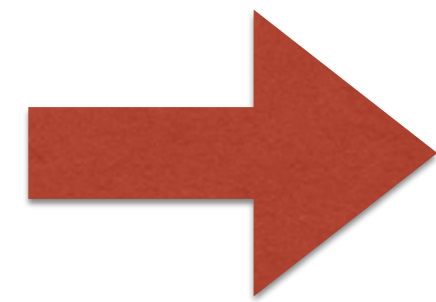


NOCS pts



Depth

Backproject



Depth pts



Pose fitting
w/ RANSAC

NOCS for Symmetric Objects

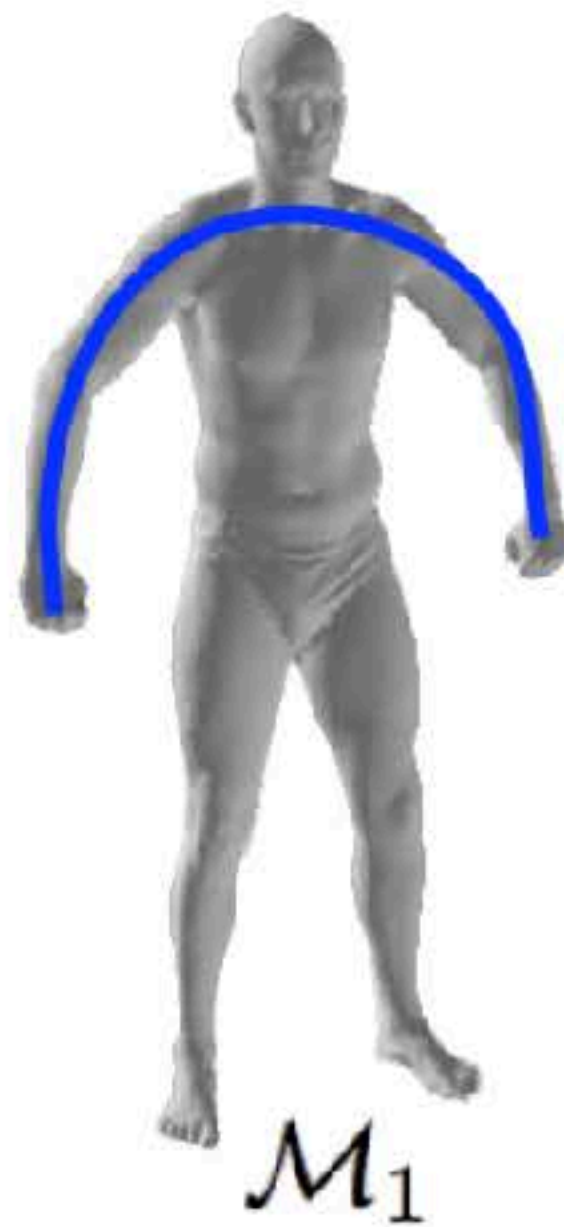
- Given equivalent GT rotations
 $\mathcal{R} = \{R_{GT}^1, R_{GT}^2, \dots, R_{GT}^n\}$ (finite symmetry order n),
we can generate n equivalent NOCS maps
- Similar to shape-agnostic loss in direct approaches,
we can use *Min of N* loss

Umeyama's Algorithm

- However, the target points in the canonical space of NOCS are normalized, and thus we also need to predict the scale factor
- Similarity transformation estimation (rigid transformation + uniform scale factor)
- Closed-form solution
 - Umeyama algorithm: <http://web.stanford.edu/class/cs273/refs/umeyama.pdf>
 - Similar to the counterpart without scale

Next Time

- 3D analysis - intrinsic shape understanding



isometry = length-preserving transform

